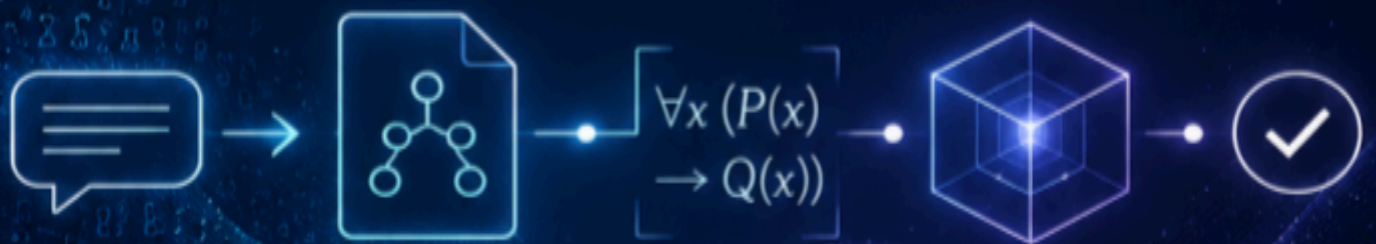


AUSFÜHRBARE NATÜRLICHE SPRACHE

Eine kritische Übersicht über formale Semantik,
abstrakte Interpretation, symbolische Ausführung
und symbolisch fundierte LLM-Architekturen



AUSFÜHRBARE NATÜRLICHE SPRACHE

Eine kritische Übersicht über formale Semantik, abstrakte
Interpretation, symbolische Ausführung und symbolisch fundierte
LLM-Architekturen

*Auf dem Weg zu einer neuro-symbolischen Laufzeitumgebung für lange
Dokumente, Regeln und agentische Systeme*



Copyright © [2026] Axiologic Research

Alle Rechte vorbehalten.

KDP Verlagsrechte von Outfinity SRL (Iași, Rumänien).

Kostenlos auf der Axiologic-Website (www.axiologic.net) verfügbar.

Anmerkung des Autors

Diese Arbeit wurde unter dem Dach von Axiologic Research als Teil von zwei miteinander verbundenen Forschungsprogrammen erstellt, eines der Künstlichen Intelligenz und das andere dem Ausbruch, beide von Sinică Alboaie geleitet. Um die Details zu erkunden, wie sich diese beiden Programme kreuzen und aufeinander aufbauen, laden wir die Leser ein, die Outfinity Venture Studio-Seite unter www.outfinity.ch zu besuchen. Während Sprachmo-

delle der Künstlichen Intelligenz zur Unterstützung der Texterzeugung verwendet wurden, stammen die grundlegende Philosophie, die thematische Struktur und der analytische Ansatz ausschließlich aus der jahrzehntelangen Untersuchung sozialer Phänomene, sozialer Technologien und echter harter Technologien, die sich aus verschiedenen europäischen und selbstfinanzierten Forschungsprojekten ergeben. Kürzlich haben wir im Rahmen des Achilles-Forschungsprojekts die generierte Literatur und ihre automatisierte Verifikation mit neurosymbolischen Ansätzen gründlich untersucht AI; alle kostenlosen Bücher, die der Öffentlichkeit zur Verfügung gestellt werden, sind Teil der Arbeitsreihe, die wir verwenden, um unsere neuesten Technologien zu testen.

Inhalt

Zusammenfassung.....	5
Vorwort: Umfang, Methode und kritische Position.....	8
1. Von fließender Sprache zu ausführbarer Semantik.....	11
1.1 Was Ausführung erfordert und wo die Vertrauensgrenze liegen muss.....	11
1.2 Die Formalisierungslücke: Warum ein korrekter Solver immer noch die falsche Antwort erzeugen kann.....	15
2. Die intellektuelle Abstammung der ausführbaren Sprache.....	18
2.1 Kontrollierte Sprachen, formale Semantik, Diskurs und natürliche Logik.....	18
2.2 Semantisches Parsing, ausführbare Abfragen, Programminduktion und frühe neurosymbolische Systeme.....	22
3. Abstrakte Interpretation und symbolische Ausführung für Sprache.....	26
3.1 Abstrakte Interpretation als Semantik der gebundenen Unsicherheit.....	26
3.2 Symbolische Ausführung, Constraint-Lösung, Modellprüfung und semantische Verfeinerung.....	30
4. Was die LLM-Ära tatsächlich erreicht hat.....	35
4.1 Autoformalisierung, programmgestütztes Denken, Logik, zeitliche Spezifikationen und Planung.....	35
4.2 LLM-gestützte Programmanalyse, symbolische Exploration, abstrakte Transformatoren und zertifizierte NLP.....	39
5. Warum ausführbare Sprachsysteme scheitern.....	43
5.1 Grounding, Identität, scheinbare Formalisierungen, Ontologien und Weltwissen.....	43
5.2 Pragmatik, Nichtmonotonität, Skala, Sicherheit und Bewertungsfehler.....	47
6. Architekturfamilien für ausführbare natürliche Sprache.....	51
6.1 Konkurrierende Designs: universelle Repräsentationen, Formalismus-Portfolios, kontrollierte Kompilierung und query-first-Systeme.....	51
6.2 Eine Referenzpipeline von Text zu Symbolen, Ausführung und eingeschränkten natürlichen Sprachantworten.....	56
7. LongTextJS, CircuitJS und nllAgent: Eine kritische Fallstudie.....	60
7.1 Was die öffentliche Architektur implementiert und warum ihre Grenzen wichtig sind.....	61
7.2 Unabhängige Bewertung: Stärken, aktuelle ernste Probleme und die erforderliche Evolution.....	65
8. Aufbau und Validierung einer neurosymbolischen Laufzeit.....	70
8.1 Implementierungsstrategie: Ein minimales, aber wissenschaftlich ehrliches ausführbares System für Sprache.....	70
8.2 Forschungsfragen, Benchmarks und ein Publikationsprogramm.....	77
Schlussfolgerung: von probabilistischer Form zu symbolischer Konsequenz.....	84
Literatur.....	88

Zusammenfassung

Große Sprachmodelle haben die natürliche Sprache zu einer effektiven Schnittstelle zur Berechnung gemacht, aber sie haben die natürliche Sprache nicht in dem starken Sinne ausführbar gemacht, in dem ein Programm, eine Datenbankabfrage, ein Übergangssystem oder eine logische Theorie ausführbar ist. Ein Modell kann eine überzeugende Erklärung erzeugen, während er den Umfang eines Quantifizierers ändert, eine Ausnahme auslöst, eine Prämisse erfindet oder die gleiche Regel an zwei Stellen unterschiedlich anwendet. Externe Tools verbessern die Zuverlässigkeit nur, wenn das Modell die Anforderung korrekt übersetzt hat. Ein Theorem-Prover kann die falsche Formalisierung perfekt beweisen; ein Python-Interpreter kann ein Programm ausführen, das versehentlich das Beispiel beantwortet, während er die Frage falsch darstellt; ein symbolischer Ausführer kann einen echten Pfad durch Software freilegen, obwohl die natürlichsprachliche Regel, die zum Konstruieren der Pfadbedingung verwendet wurde, falsch übersetzt wurde. Das schwierige Problem besteht daher nicht nur darin, nach einem LLM einen Löser hinzuzufügen. Es besteht darin, eine überprüfbare Kette von der Ausgangssprache bis zu geordneten Symbolen, von geordneten Symbolen bis hin zu ausführbaren formalen Objekten und von symbolischen Ergebnissen bis hin zur natürlichen Sprache zu konstruieren, ohne das endgültige Sprachmodell zu ändern, was etabliert wurde.

Dieses Buch untersucht die relevanten Traditionen als ein zusammenhängendes Forschungsproblem. Zu den früheren Grundlagen gehören kontrollierte natürliche Sprachen, modelltheoretische und Diskurssemantik, natürliche Logik, semantisches Parsen, Logikprogrammierung, Text-zu-Abfrage-Systeme, Programmsynthese aus Beispielen und Sprache, Regel-Engines, Entscheidungstabellen, zeitliche Logiken, automatisierte Planung und neuronal-symbolische Systeme. Die Grundlagen der Programmanalyse umfassen abstrakte Interpretation, symbolische und konkretische Ausführung, Datenflussanalyse, Modellprüfung, SAT/SMT/CSP-Lösung, Datalog und Fixpoint-Auswertung, Automaten- und Proof-Producing-Verifizierung. Die jüngste LLM-Literatur fügt programmgestütztes Denken, deklarative Löseraufforderung, Autoformalisierung, Übersetzung der Erstordnungslogik, temporal-logische Übersetzung, formale Planung, lösergesteuerte Selbstreparatur, LLM-gestützte Programmanalyse, LLM-gesteuerte symbolische Erforschung und die Synthese von verifizierten abstrakten Transformatoren hinzu. Über diese Arbeitsorgane hinweg erscheinen die stärksten Ergebnisse konsequent, wenn der Zielformalismus schmal genug ist, um eine explizite Semantik zu haben, und die akzeptierte Ausgabe von einer Komponente unabhängig vom LLM überprüft wird.

Besonderes Augenmerk legt die Erhebung auf die Unterscheidung zwischen zwei Verwendungen abstrakter Interpretation. Der erste, der bereits in NLP etabliert ist, überprüft ein neuronales Modell über eine formal definierte Familie von textuellen Störungen. Intervall- und relationale abstrakte Domänen können bescheinigen, dass sich die Vorhersage eines Klassifikators nicht unter jeder Substitution ändert, die durch eine bestimmte kontradiktorische Menge zulässig ist. Dies ist eine echte formale Garantie, aber es betrifft die Modellstabilität unter einem konstruierten Störungsverhältnis; es bescheinigt nicht, dass willkürliche menschliche Paraphrasen die Bedeutung bewahren, noch dass die ursprüngliche Antwort des Modells wahr ist. Die zweite Verwendung ist spekulativer und relevanter für ausführbare natürliche Sprache: die Behandlung der zulässigen Interpretationen eines Dokuments als Sammelsemantik und Rechenkonservativen kann, darf, kann, kann und unbekannte

Schlussfolgerungen über diesen Satz. Dies erfordert explizite Domänen für Identität, Zeit, Modalität, Quantität, Quellstatus, normative Kraft und Abdeckung. Es erfordert auch eine ehrliche bedingte Garantie: Die Analyse ist nur im Verhältnis zu dem Interpretationssatz, der Ontologie, der Grundpolitik und den ihm gelieferten Übertragungsregeln vernünftig.

Die symbolische Ausführung ist komplementär. Eine abstrakte Interpretation kann billig zeigen, dass ein Verstoß unter der aktuellen Abstraktion unmöglich ist oder dass ein Verstoß möglich bleibt. Die symbolische Ausführung kann dann versuchen, eine konkrete Interpretation und einen Pfad zu konstruieren, der sie verwirklicht. Wenn kein solcher Weg existiert, kann der gescheiterte Zeuge zeigen, welche Abstraktion oder semantische Wahl zu grob war. Dies deutet auf ein Forschungsmuster hin, das als Counter-Example-gesteuerte semantische Verfeinerung bezeichnet wird: Das Gegenbeispiel eines Löser, ein unzufriedener Kern oder ein fehlgeschlagener Beweis wird auf die genauen Quellpassagen und semantischen Verpflichtungen zurückgeführt, die es erzeugt haben; der LLM oder ein Mensch wird gebeten, nur diese Verpflichtungen zu überarbeiten; die symbolische Analyse wird wiederholt. Der LLM wird verwendet, um den Raum der Formalisierungen zu durchsuchen, während der symbolische Kernel bestimmt, was sich aus jeder akzeptierten Formalisierung ergibt.

Aktuelle Systeme zeigen wichtige Fragmente dieser Vision. PAL delegiert Verfahrensberechnung an Python. SatLM erzeugt deklarative Einschränkungen für eine Zufriedenheits-Engine. Logic-LM und LINC übersetzen Aufgaben des natürlichsprachlichen Denkens in formale Logik und verlassen sich auf symbolische Solver oder Theorem-Prover für die Deduktion. NL2TL und nl2spec übersetzen Anforderungen in zeitliche Spezifikationen, wobei nl2spec explizit die interaktive Korrektur auf der Unterformulaebene unterstützt. Formale Planungsarbeiten haben große Gewinne erzielt, wenn LLMs Planungsinstanzen konstruiert, die dann gelöst und von Sound-Tools überprüft werden. SymGPT übersetzt natursprachliche ERC-Regeln in eine eingeschränkte Grammatik und verwendet symbolische Ausführung, um konkrete Verstöße gegen intelligente Verträge zu finden, die Tausende von Regelverstößen und expliziten Angriffspfaden melden. In der anderen Richtung zerlegt AutoBug die Programmanalyse in symbolische Pfade mit LLM-Gehäuse, bleibt aber ungefähr; SAIL verwendet LLMs, um nach abstrakten Transformatoren zu suchen, während harte semantische und syntaktische Überprüfungen die globale Solidität bewahren. Diese Ergebnisse sind Beweise dafür, dass LLMs ein effektiver Vorschlag und Suchmechanismen für formale Kerne sind, nicht Beweise dafür, dass die probabilistische Generation selbst zu formaler Argumentation geworden ist.

Eine kürzlich durchgeführte systematische Überprüfung des neurosymbolischen NLP argumentiert, dass föderative Architekturen, in denen neuronale und symbolische Module autonom bleiben und über strukturierte Schnittstellen kommunizieren, oft größere Gewinne aufweisen als Architekturen, die nur symbolische Verzerrungen in neuronale Netze injizieren. Seine Autoren warnen auch davor, dass der scheinbare Vorteil noch nicht schlüssig ist, da föderative und injektive Systeme in der Regel bei verschiedenen Aufgaben bewertet werden. Dieses Buch nimmt die gleiche disziplinierte Position ein. Das architektonische Argument für die Trennung ist stark, weil es die Vertrauensgrenze klärt, eine unabhängige Wiedergabe erlaubt und die symbolische Komponente unfassbar hält; die empirische Behauptung, dass eine Familie eine andere universell übertrifft, bleibt unbewiesen [Chatzikyriakidis and Lappin 2026].

Das Buch bietet auch eine kritische Fallstudie der öffentlichen NaturalLanguageLinterAgent oder nllAgent, Architektur und seiner LongTextJS- und CircuitJS-Komponenten. Die

Implementierung trennt ein unveränderliches dokumentenseitiges Programm von einem wiederverwendbaren theoretisch-seitigen Programm. Es bewahrt exakte Quellanker, versionierte Beobachtungstypen, Herstelleridentitäten, Abdeckung, Lücken und epistemischen Status. Schaltungszusammenstellung leitet die tatsächlich von einer Regel geforderten Beobachtungen ab, die Veröffentlichung prüft, ob kompatible Hersteller existieren, Laufzeitkompatibilitätsprüfungen, ob das konkrete Dokument ausreichende Beobachtungen und Abdeckungen ergab, und ein unabhängiger Prüfer kontrolliert, ob ein Kandidatenfund veröffentlicht wird. Das Lernen findet in einem Einweg-Arbeitsbereich statt und kann nicht automatisch veröffentlicht werden. Dies sind ungewöhnlich ernste Governance-Entscheidungen für ein LLM-gestütztes Dokumentenanalysesystem. Das öffentliche Repository zeichnet auch ungelöste Einschränkungen auf, anstatt sie zu verbergen. Das aktuelle seriöse Register identifiziert die Inkrementalität des begrenzten Schedulers, die unvollständige Verbreitung von Interpretationsalternativen über jeden Ausführungspfad, die konservative Kompatibilitätsanalyse für undurchsichtiges Verhaltensverhalten, die minimale Abfrageplanung und zeitliche Indexierung, die bewusst enge kontrollierte Erzeugung und die Mutationsunterstützung, die szenariospezifisch bleibt. Das sind glaubwürdige Forschungsbeschränkungen: Sie betreffen das semantische und ausführende Substrat, nicht nur fehlende Benutzeroberflächenpolitik.

In den Suchanfragen, die für diese Ausgabe durchgeführt wurden, wurde keine unabhängige Peer-Review-Bewertung oder technische Überprüfung von nllAgent durch Dritte durchgeführt. Dementsprechend ist die Fallstudienbewertung explizit die eigene architektonische Überprüfung des Autors, die im öffentlichen Endlager, der Dokumentation, dem ausführbaren Umfang und selbsterklärten ernsthaften Fragen begründet ist. Das Urteil ist günstig über die Trennung von Beweisen, Theorie, Verifizierung und Veröffentlichung, aber vorsichtig über semantische Vollständigkeit. Die stärksten aktuellen Umsetzungsansprüche betreffen begrenzte strukturelle und kontrolliert-englische Kontrollen. Allgemeine Identitätsauflösung, reichhaltige Ereignissemantik, Wissen der aktuellen Welt, breiter Domaintransfer und statistisch unterstützte Qualitätsansprüche bleiben eher Forschungsarbeit als etablierte Fähigkeiten.

Der zentrale Vorschlag des Buches ist eine multiformalistische, ausführbare, natursprachliche Laufzeit. Ein LLM schlägt zunächst eine kleine Reihe von kandidatensemantischen Strukturen vor, die jeweils mit exakten Quellspannen verknüpft sind und gegen eine ausgewählte Ontologie getippt sind. Ein symbolischer Erdungskern lässt nur Objekte zu, deren Identität, Herkunft, Umfang, Zeit, Modalität und Beweise explizite Verträge erfüllen. Ein Router wählt ein oder mehrere formale Ziele nach der Frage aus: relationale oder Datalog-Bewertung für Schließungs- und Dokumentabfragen; Entscheidungstabellen für Richtlinien; SAT/SMT/CSP für Einschränkungen; Automaten für geordnete Muster; Modellprüfung für zeitliches Verhalten; symbolische Ausführung für Pfadzeugen; und Beweisassistenten oder Theoremprüfer für deduktive Ansprüche. Die formelle Ausführung führt zu einem kanonischen Ergebnis, das akzeptierte Fakten, abgelehnte Kandidaten, Unsicherheit, Zeugen, Beweisobjekte, Deckungsgrenzen und Annahmen enthält. Ein abschließender LLM kann dieses Ergebnis erklären, aber jeder Satz in der Erklärung muss durch die kanonische Struktur lizenziert und auf Beweise oder eine verifizierte Ableitung zurückzuführen sein.

Ausführbare natürliche Sprache sollte daher nicht als Ausführung einer uneingeschränkten Prosa beworben werden. Es sollte als kontrollierter Zusammenstellungs-, Erdungs-, Ausführungs- und Realisierungsprozess über einen oder mehrere explizite semantische Kandidaten definiert werden. Die Garantie ist bedingt, aber nützlich: Angesichts der angegebenen Quellenrevision, Ontologie, Erdung, Interpretationssatz, formale Semantik und

Ausführungsgrenzen folgt das gemeldete Ergebnis und kann wiedergegeben werden. Das Hauptproblem der offenen Forschung ist nicht der Löser. Es ist der Bau, der Vergleich, die Verfeinerung und die Validierung der Formalisierungen, auf denen der Löser arbeitet.

Vorwort: Umfang, Methode und kritische Position

Der Ehrgeiz der ausführbaren natürlichen Sprache ist leicht auszudrücken und leicht zu überschätzen. Ein Benutzer schreibt eine Regel, Politik, wissenschaftliche Hypothese, Verfahren oder Frage in der gewöhnlichen Sprache; ein System versteht es; Berechnung folgt; das Ergebnis wird als gewöhnliche Sprache zurückgegeben. Zeitgenössische Sprachmodelle lassen den Anfang und das Ende dieser Pipeline gelöst erscheinen. Sie können uneingeschränkte Prosa lesen, Code und Formeln generieren, Tools aufrufen und Ausgänge fließend erklären. Die gefährliche Illusion ist, dass die Mitte verschwunden ist. In der Tat enthält die Mitte fast jedes schwierige Problem: entscheiden, welche Bedeutung zählt, identifizieren Objekte über Passagen hinweg, Unterscheidung expliziter Beweise von Hintergrundannahmen, Bewahrung von Modalitäten und Ausnahmen, Auswahl eines geeigneten Formalismus, Nachweis, dass das erzeugte Artefakt gut geformt ist, und Verhinderung der endgültigen Erklärung, mehr als die durchgeführte Ausführung zu sagen.

Eine nützliche Erhebung kann kein Katalog kleiner Systeme sein, der in kurzen isolierten Unterabschnitten dargestellt wird. Das Feld ist gerade deshalb fragmentiert, weil die gleichen Wörter für unterschiedliche Operationen verwendet werden. "Symbolische Ausführung" kann die Erkundung von Pfaden im Königsstil über symbolische Programmeingaben, die Ausführung einer generierten logischen Form auf einer Wissensbasis oder einfach die Verwendung eines externen Interpreters bedeuten. "Formale Verifikation" kann sich auf die Richtigkeit eines Solver-Ergebnisses, die Korrektheit eines kompilierten Programms, die Genauigkeit der natürlichen Sprachübersetzung oder die Robustheit eines neuronalen Netzwerks unter einer vordefinierten Störungsfamilie beziehen. "Erden" kann bedeuten, Wörter mit Datenbankspalten, Entitäten zu verbinden, um Spannen zu dokumentieren, Vorschläge zum Weltwissen oder Roboteranweisungen für Wahrnehmungsobjekte. Eine ernsthafte Behandlung muss zeigen, wie sich diese Schichten verhalten und wo Garantien aufhören.

Die Erhebungsmethode folgt daher eher den Mechanismen als den Marketingkategorien. Für jede Arbeit stellt die Analyse fünf Fragen. Welches Artefakt wird aus der Sprache hergestellt? Welche Semantik hat dieses Artefakt unabhängig vom LLM? Wie basiert das Artefakt auf der Quelle oder Umgebung? Welche Komponente bestimmt tatsächlich das Ergebnis? Was ist und ist nicht garantiert, wenn das System erfolgreich ist? Papiere werden als Beweis für ihre gemeldeten Aufgaben behandelt, nicht als Beweis für eine allgemeine Architektur. Starke numerische Ergebnisse werden einbezogen, wenn sie den Mechanismus verdeutlichen, aber sie werden nicht in Domänen mit unterschiedlicher semantischer Struktur übertragen. Der Erhebungshorizont ist der 2. August 2026 und umfasst begutachtete Literatur, offizielle Verfahren, primäre Forschungsarbeiten und das öffentliche nllAgent-Repository und die Dokumentation.

Der Begriff „Review“ wird sorgfältig verwendet. Es gibt mittlerweile mehrere umfassende Erhebungen, die jedoch jeweils nur einen Teil des End-to-End-Problems abdecken. Kuhns kontrolliert-natürlichsprachige Umfrage organisiert Sprachen nach Präzision, Ausdruckskraft, Natürlichkeit und Einfachheit [Kuhn 2014]. Hamilton und Kollegen bewerten, ob neurosymbolisches NLP tatsächlich Argumentation, Transfer, Interpretierbarkeit, Dateneffizienz und Out-of-Distribution-Verallgemeinerung geliefert hat, während neuere Umfragen aufgabenorientierte und daten- und kennungsgesteuerte neurosymbolische Systeme [Hamilton et al. 2024; Delvecchio et al. 2025; Wang et al. 2025b] klassifizieren. Untersuchungen zur

symbolischen Ausführung und LLM-gestützten Programmanalysen beschreiben die technischen Grenzen der Pfadexploration und die wachsende Rolle der gelernten Komponenten [Baldoni et al. 2018; Wang et al. 2025a]. Eine spezielle Übersicht untersucht LLMs als Formalisierer für die automatisierte Planung [Tantakoun et al. 2025], und die Autoformalisierungserhebung konzentriert sich auf die Übersetzung informeller Mathematik in maschinenüberprüfbare Formen [Weng et al. 2025]. Eine Zwei-Wege-Roadmap für Requirements Engineering untersucht sowohl den Einsatz von LLMs, um formale Methoden zugänglich zu machen, als auch den Einsatz formaler Methoden zur Eindämmung der LLM-produzierten Anforderungen [Ferrari and Spoletini 2025]. Eine fokussierte Umfrage unter fünfunddreißig Papieren bildet die formalisierte Formalisierung von Softwareanforderungen und Spezifikationen, einschließlich der Arbeit, die auf Dafny, C und Java abzielt; sein Umfang bestätigt, dass die Übersetzungsunterstützung schneller wächst als die End-to-End-semantische Sicherheit [Beg et al. 2025]. Der systematische Bericht von Frontiers 2026 entwickelt eine neue Taxonomie für neurosymbolische NLP und plädiert für mehr Aufmerksamkeit für föderative Systeme [Chatzikyriakidis and Lappin 2026].

Keine befragte Überprüfung vereint kontrollierte Sprache, semantische Erdung, abstrakte Interpretation, symbolische Ausführung, Formalismusauswahl, Proof-Carrying-Ergebnisse und eingeschränkte Regeneration als eine Vertrauensarchitektur. Diese Schlussfolgerung ist eher eine Literatursynthese als ein Anspruch auf erschöpfende Nichtexistenz: Die Terminologie unterscheidet sich bereichsweise und relevante Arbeiten werden über Computerlinguistik, Programmiersprachen, Datenbanken, formale Methoden, Software-Engineering, Planung und Robotik verteilt. Wenn keine unabhängige Überprüfung besteht, insbesondere für nllAgent, bezeichnet der Text seine Urteile als ursprüngliche Analyse, anstatt eine externe Billigung zu implizieren.

Die Position des Buches ist eigensinnig, aber konservativ. Es lehnt die Idee ab, dass einem großen Modell als der letzte Grund nur deshalb vertraut werden sollte, weil seine Prosa einem Beweis ähnelt. Es lehnt auch die gegenteilige Idee ab, dass nur ein handgeschriebener kompletter semantischer Parser eine nützliche formale Ausführung unterstützen kann. In offenen Domänen ist ein LLM oft der beste verfügbare Mechanismus, um Ereignisse, Beziehungen, formale Schemata und Kandidatenzuordnungen vorzuschlagen. Die architektonische Frage ist, wie man diese Macht einschränkt. Die vertretbarste Antwort ist ein föderiertes System mit expliziten Zwischenartefakten, typisierten und versionierten Schnittstellen, unabhängiger symbolischer Ausführung, konservativen unbekannten Zuständen und widerspielbaren Beweisen.

Diese Position hat eine wichtige Konsequenz. Der Zweck des Systems besteht nicht immer darin, eine endgültige Formalisierung zu erreichen. Manchmal ist das richtige Artefakt eine Reihe von Alternativen. Ein Pronomen kann sich auf zwei mögliche Akteure beziehen; ein juristischer Satz kann konkurrierende Spielräume zulassen; eine Anforderung kann unvollständig sein; zwei Quellen können anderer Meinung sein; ein Dokument kann nicht sagen, ob eine Ausnahme gilt. Eine symbolische Laufzeit, die willkürlich eine Interpretation wählt, ist weniger streng als eine LLM, die offen Mehrdeutigkeit angibt. Die Ausführbarkeit wird wertvoll, wenn sie berechnen kann, was unter jeder zulässigen Lesung folgt, was unter mindestens einem folgt, welche Schlussfolgerungen von einer umstrittenen Wahl abhängen und welche zusätzlichen Beweise die Wahl lösen würden.

Die vorgeschlagene Architektur behandelt die Unsicherheit daher strukturell. Vertrauensbewertungen können Kandidaten einordnen, aber sie verwandeln eine vorgeschlagene Interpretation nicht in eine zertifizierte Tatsache. Der Status eines Artikels zeichnet auf, wie er bekannt wurde: direkt extrahiert aus einem deterministischen Parser, der von einem Modell vorgeschlagen wurde, für ein Szenario angenommen, abgeleitet durch eine Regel, von einem Prüfer überprüft oder von einer Person bestätigt. Die Abdeckung erfasst, ob eine Domain vollständig untersucht, beprobt, abgerufen oder nicht verarbeitet wurde. Eine negative Schlussfolgerung ist nur zulässig, wenn die entsprechende Recherche im Rahmen einer genannten Richtlinie abgeschlossen ist. Dies ist der Unterschied zwischen einem zuverlässigen Symbolsystem und einem fließenden Klassifizierer mit einem JSON-förmigen Ausgang.

Die Diskussion bleibt eher zugänglich als nachweislich schwer. Abstrakte Interpretation, symbolische Ausführung, Modellprüfung und logische Semantik werden durch ihr operatives Wesen und die damit verbundenen Verpflichtungen erklärt. Die formale Notation erscheint nur dort, wo sie eine Unterscheidung klärt, dass Prosa verschwimmen könnte. Ziel ist es nicht, dekorative Mathematik zu präsentieren. Es geht darum, Systemdesignern, Rezensenten und Forschern genügend konzeptionelle Präzision zu geben, um zu entscheiden, wo eine vorgeschlagene Garantie gültig ist, wo sie bedingt ist und wo sie nur ehrgeizig ist.

Die daraus resultierende These lässt sich deutlich feststellen. LLMs sollte plausible formale Formen entdecken; deterministische und formal eingeschränkte Komponenten sollten Erdung, Zusammenstellung, Ausführung und Akzeptanz kontrollieren; LLMs kann dann das kanonische Ergebnis als Sprache realisieren, aber nur unter einem Vertrag, der nicht unterstützte Ergänzungen verhindert. Dies ist keine vollständige Lösung für den Sinn. Es ist ein glaubwürdiger Weg von der probabilistischen Sprachverarbeitung zu begrenzter, ausführbarer und überprüfbarer Argumentation.

1. Von fließender Sprache zu ausführbarer Semantik

1.1 Was Ausführung erfordert und wo die Vertrauensgrenze liegen muss

Das Wort "execute" trägt eine stärkere Verpflichtung als "Interpretieren", "Antwort" oder "Vernunft". Ein Programm ist ausführbar, weil seine Operationen eine definierte Wirkung auf einen Zustand haben. Eine Datenbankabfrage ist ausführbar, weil die relationalen Operatoren bestimmen, welche Tupel ausgewählt, verbunden, gruppiert oder ausgeschlossen werden. Ein Planungsproblem ist ausführbar, weil Handlungen Vorbedingungen und Auswirkungen haben und ein Planer nach einer Sequenz sucht, die ein Ziel erreicht. Eine logische Theorie ist im weiteren Sinne ausführbar, weil ein Inferenzverfahren bestimmt, ob eine Schlussfolgerung folgt, ob ein Modell existiert oder ob Einschränkungen zufriedenstellend sind. In jedem Fall ist die entscheidende Operation außerhalb der Rhetorik, die zur Beschreibung der Aufgabe verwendet wird. Ein Ergebnis kann aus einem Artefakt und einer Semantik reproduziert werden.

Eine Antwort von LLM ist anders. Das Modell bildet einen Kontext einer Verteilung über Fortsetzungen ab. Es kann Muster verinnerlichen, die Algorithmen ähneln, und es kann viele Aufgaben richtig lösen, aber die Regel, die bei einem bestimmten Schritt angewendet wird, ist nicht separat inspizierbar oder garantiert konsistent verwendet. Chain-of-Thought-Prompting kann die Leistung verbessern, aber eine Kette von Sätzen ist kein Beweisobjekt, nur weil sie als Schritte organisiert ist. Selbstkonsistenzproben mehrere Fortsetzungen, nicht mehrere formal definierte Ausführungen. Die Werkzeugverwendung kann einen Wert exakt machen, während die Interpretation, die den Werkzeugaufruf erzeugt hat, nicht verifiziert bleibt. Die entscheidende Unterscheidung besteht zwischen einem Modell, das ein ausführbares Artefakt vorschlägt, und einem Modell, dessen eigene Generation als Ausführung behandelt wird.

Ausführbare natürliche Sprache beginnt daher mit der Materialisierung. Das System muss ein Objekt mit einer deklarierten Grammatik, einem Typsystem, einer Semantik und einer Quellkarte erstellen. Dieses Objekt könnte eine relationale Abfrage, eine logische Theorie, eine Zustandsmaschine, eine Reihe von zeitlichen Einschränkungen, eine Entscheidungstabelle, ein Datalog-Programm, ein endlicher Automat oder ein typisiertes Diagramm mit registrierten Operatoren sein. Die besondere Wahl hängt von der Aufgabe ab. Wichtig ist, dass das Verhalten des Objekts nicht mehr durch die Next-Token-Verteilung des Sprachmodells bestimmt wird. Es kann unabhängig voneinander abgelehnt, ausgeführt, getestet, verglichen und wiedergegeben werden.

Diese Anforderung ist stärker als gewöhnliche strukturierte Ausgabe. Ein Modell kann syntaktisch gültige JSON aussenden, die erfundene Entitäten oder inkonsistente Felder enthält. Ein Schema validiert die Form, nicht die Wahrheit. In ähnlicher Weise kann ein Modell legale SQL erzeugen, die die falschen Spalten verbindet, eine gültige Logik erster Ordnung, die einen Quantifizierer umkehrt, oder eine zeitliche Formel, die die entgegengesetzte Präzedenzbeziehung ausdrückt. Eine nützliche Vertrauensgrenze ist daher nicht einfach „nach der JSON-Generierung“. Es liegt nach der Quellenerkennung, Typprüfung, semantischer Kompatibilität und formaler Zusammenstellung. Jedes Symbol, das die Antwort beeinflusst, muss eine zulässige Herkunft und eine definierte Rolle haben.

Die Architektur trennt natürlich vier Behörden. Die Quellbehörde bestimmt, welche Texte, Daten oder Beobachtungen tatsächlich geliefert wurden. Die semantische Behörde bestimmt, welche Arten, Beziehungen, Modalitäten und Identitätsrichtlinien zulässig sind. Die

Regelbehörde bestimmt, welche Zwänge oder Transformationen die Theorie darstellen, die ausgeführt wird. Die Ausführungsbehörde bestimmt, ob ein Kandidatenergebnis unter dieser Theorie folgt. Ein LLM kann allen vier helfen, aber es sollte sie nicht stillschweigend zusammenführen. Wenn das gleiche Modell eine Prämisse erfindet, eine Ontologie auswählt, eine Regel anwendet und das Urteil schreibt, gibt es keine sinnvolle Trennung zwischen Beweisen und Schlussfolgerungen.

Die Quellenautorität beginnt mit stabilen Bytes und Spannen. Ein Dokumentenanalysesystem sollte die genaue Überarbeitung, Struktur und Position jedes zitierten Fragments beibehalten. Es sollte ein Zitat von einer Zusammenfassung und eine Zusammenfassung von einer Schlussfolgerung unterscheiden. Wenn ein Modell vorschlägt, dass ein Satz eine Verpflichtung ausdrückt, muss die Beobachtung auf den Satz hinweisen und das Modell, die Aufforderung, das Schema und die Alternativen beibehalten, die die Klassifizierung erstellt haben. Wenn das Zitat in der Quelle nicht zu finden ist, wird die Beobachtung abgelehnt und nicht als plausible Rekonstruktion akzeptiert. Das klingt nach weltlicher Provenienztechnik, ist aber für jede spätere Garantie Voraussetzung.

Semantische Autorität wird normalerweise auf eine obere Ontologie und Domain-Pakete verteilt. Eine universelle Ontologie ist attraktiv, weil sie Interoperabilität verspricht, aber sie neigt entweder dazu, zu abstrakt zu werden, um nützliche Fragen zu entscheiden, oder zu groß und umstritten, um zu regieren. Ein besseres Design definiert eine kleine gemeinsame Schicht für Quellspannen, Erwähnungen, Entitäten, Ereignisse, Zeit, Welten, Status, Provenienz, Alternativen, Abdeckung und Lücken. Domain-Pakete definieren dann Konzepte wie klinische Intervention, regulatorische Verpflichtung, narratives Objekt ereignis, Software API Call oder Finanztransaktion. Jedes Paket muss nicht nur Feldnamen, sondern auch semantische Verpflichtungen und Tests angeben. Ein Typ namens "Zulassung" ist nicht sinnvoll, es sei denn, das System gibt an, ob er ein Dokument, eine Handlung, einen Staat, eine Autoritätsentscheidung oder eine Kombination bezeichnet.

Die Regelbehörde sollte getrennt von der Auslegung von Dokumenten versioniert werden. Eine Politik, eine Lintertheorie oder ein wissenschaftliches Modell ändert sich mit einer anderen Geschwindigkeit als die Quelle, die es analysiert. Wenn Regeln unsichtbar in Aufforderungen eingebettet sind, wird es schwierig zu wissen, ob zwei Runs die gleiche Theorie anwenden. Versionierte Regelartefakte erlauben Benchmark-Veröffentlichung, semantische Diffs und Rollback. Sie erlauben auch mehrere Theorien über die gleiche Dokumentdarstellung. Ein Manuskript kann auf Terminologiekonsistenz, kausale Kohärenz und Zitationsunterstützung überprüft werden, ohne es zu einer anderen undurchsichtigen Aufforderung für jede Regel zu kompilieren.

Die Vollstreckungsbehörde sollte so klein und mechanisch wie praktisch sein. Bei relationalen Abfragen kann es sich um einen deterministischen Bewerter handeln. Für Einschränkungen, ein SMT oder CSP Solver. Für das zeitliche Verhalten, ein Modell-Checker. Für pathspezifische Programmeigenschaften, ein symbolischer Ausführer. Für Beweisverpflichtungen, ein Theorem-Prüfer plus ein kleiner Proof-Checker. Für eine benutzerdefinierte Schaltungssprache kann es sich um einen eingeschränkten Dolmetscher mit registrierten Operatoren und unabhängigen Prüfern handeln. Die Akzeptanzkomponente sollte einen LLM nicht fragen, ob das Ergebnis „richtig aussieht“. Es sollte einen Zeugen mit der Quelle und Theorie vergleichen.

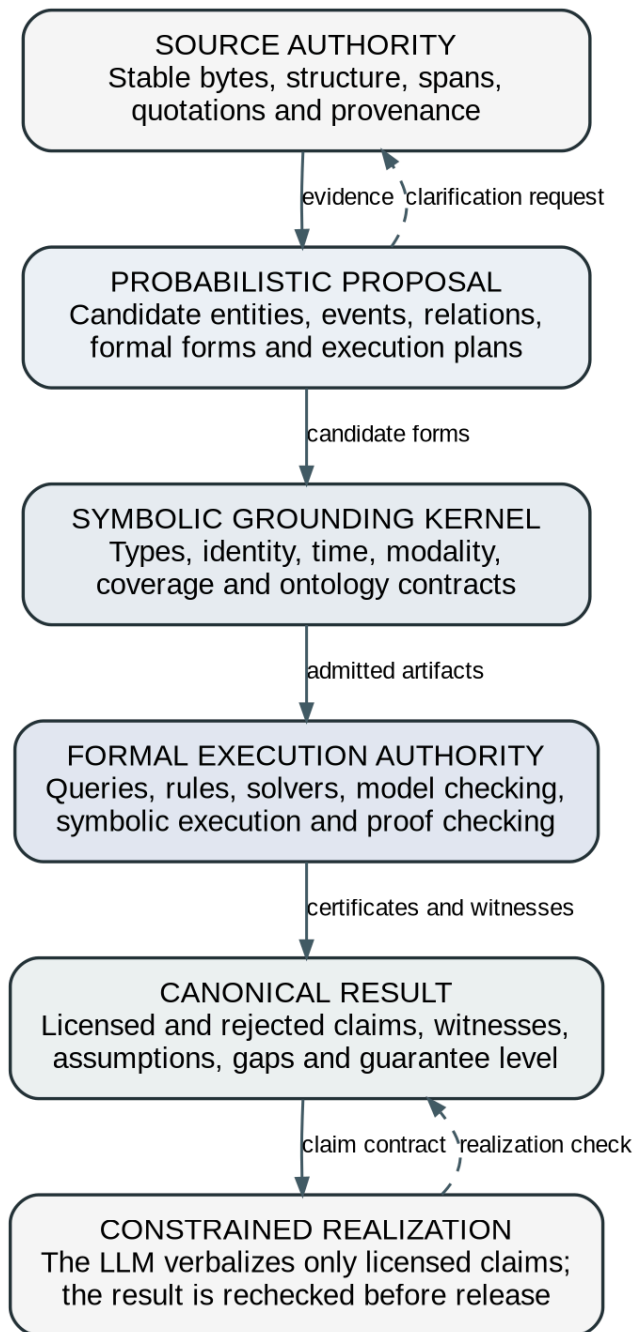


Abbildung 1. Vertrauensgrenze für ausführbare natürliche Sprache. Das LLM schlägt semantische Formulare vor, während die Quellenerdung, die formelle Ausführung, die Annahme und die endgültige Anspruchslizenzierung unabhängig prüfbar bleiben.

Eine nützliche Möglichkeit, Systeme zu klassifizieren, ist der Ort des endgültigen Orakels. In einem rein neuronalen System ist das LLM das Orakel. In einem werkzeuggestützten System berechnet ein Tool einen Wert, aber das LLM kann das Orakel bleiben, ob der Werkzeugaufwurf die Frage darstellt. In einem von Lösern unterstützten System ist der Löser das Orakel für die Folgen einer erzeugten Formalisierung. In einem evidenzgeordneten System bestimmen der Löser und der Erdungskern gemeinsam das akzeptierte Ergebnis. In einem Proof-Carrying-System kann sogar die Ausgabe des Solvers von einem kleineren vertrauenswürdigen Kernel überprüft werden. Jeder Schritt verengt den Raum, in dem probabilistischer Fehler das Urteil stillschweigend ändern kann.

Dies bedeutet nicht, dass jede Anwendung eine Theorem-Prover-Level-Zusicherung benötigt. Ein Schreibassistent kann leichte deterministische Schaltkreise verwenden, während ein Regulierungs- oder Sicherheitssystem formale Beweisartefakte erfordern kann. Das wichtige Prinzip ist, dass die Garantie dem Mechanismus entsprechen sollte. Eine deterministische Regex-Prüfung kann das Vorhandensein eines wörtlichen Musters bescheinigen, nicht das Fehlen eines Konzepts. Ein modellgestützter Ereignisextraktor kann eine vorgeschlagene Kontinuitätswarnung unterstützen, nicht eine mechanisch bestimmte Darstellung der narrativen Identität. Ein Löser kann Unzufriedenheit mit Einschränkungen und nicht die Vollständigkeit der natürlichsprachlichen Übersetzung feststellen. Rigor besteht weitgehend darin, sich zu weigern, das Vertrauen über diese Grenzen hinweg zu übertragen.

Die stärkste Architektur ist daher föderativ. Die neuronale Komponente wird nicht in ein symbolisches Netzwerk aufgelöst, und die symbolische Komponente wird nicht auf einen differenzierbaren Regularizer reduziert. Sie bleiben getrennte Systeme mit einer typisierten Schnittstelle. Der LLM ist verantwortlich für Breite, Hypothesengenerierung, semantische Normalisierung und Erholung von ungewöhnlicher Sprache. Der formelle Kern ist für Zulässigkeit, Hinrichtung, Invarianten, Zeugen und Reproduzierbarkeit verantwortlich. Die Überprüfung von Chatzikiyiakidis und Lappin aus dem Jahr 2026 nennt verwandte Systeme föderativ und beobachtet, dass sie oft starke Fortschritte bei Aufgaben zeigen, bei denen die Wahrnehmung von der Schlussfolgerung getrennt werden kann. Die Autoren betonen auch, dass Vergleiche durch Aufgabenunterschiede weiterhin verunsichert sind. Das architektonische Argument für den Verband ist dennoch unabhängig von der Benchmark-Überlegenheit: Separate Module machen Ansprüche überprüfbar und ermöglichen es einer Komponente, eine andere [Chatzikiyiakidis and Lappin 2026] abzulehnen.

Die endgültige natürlichsprachliche Ausgabe erfordert eine eigene Grenze. Sobald ein Löser ein Ergebnis zurückgibt, ist es verlockend, die Spur zu einem LLM zu liefern und eine polierte Antwort anzufordern. Ohne Einschränkungen kann das Modell Hintergrundklärungen hinzufügen, Motive schließen oder Unsicherheit in Sicherheit bringen. Ein kanonisches Ergebnis sollte daher die Behauptungen aufzählen, die verbalisiert werden können, ihren Status, ihre Beweise, Annahmen und ihren Umfang. Der Renderer kann Wortlaut und Organisation wählen, aber er sollte mit der kanonischen Struktur überprüft werden. Eine praktische Methode ist die Realisierung auf Anspruchsebene: Das Modell erzeugt Sätze, die mit Ergebniskennungen verbunden sind; ein Prüfer stellt sicher, dass jeder Satz von einem zulässigen Anspruch mit sich gebracht wird oder umschreibt; nicht unterstütztes Material wird entfernt oder als Kommentar gekennzeichnet. In hochsicheren Einstellungen kann eine vorgefertigte kontrollierte Sprache der uneingeschränkten Prosa vorzuziehen sein.

Die in diesem Buch vorgeschlagene Vertrauensgrenze kann ohne Code zusammengefasst werden. LLMs kann semantische Formen, Ontologie-Mappings, Formalismusentscheidungen, Reparaturen und Erklärungen vorschlagen. Sie stellen keine Beweise fest, schließen eine Open-World-Suche, bescheinigen die Identität, entscheiden über die Zufriedenheit oder genehmigen eine endgültige Behauptung. Diese Funktionen gehören expliziten symbolischen Verträgen an. Diese Trennung bewahrt den einzigartigen Wert von Sprachmodellen und verhindert, dass ihre Flüssigkeit mit Beweisen verwechselt wird.

1.2 Die Formalisierungslücke: Warum ein korrekter Solver immer noch die falsche Antwort erzeugen kann

Die zentrale Schwäche der meisten neurosymbolischen Argumentationssysteme ist nicht der Inferenzmotor. SAT-Solver, SMT-Löser, Datenbank-Engines, Theorem-Provers, Modellprüfer und symbolische Ausführer sind oft viel zuverlässiger als das Sprachmodell, das ihnen vorausgeht. Die Schwäche liegt in der Formalisierungslücke: der Unterschied zwischen dem, was die Quelle im Kontext bedeutet, und dem formalen Objekt, das zur Ausführung eingereicht wird. Ein System kann perfekt von der formalen Eingabe bis zur formalen Ausgabe gesund sein, während es von der natürlichen Sprache bis zur Antwort unerschrocken ist.

Betrachten Sie einen einfachen Grundsatzsatz: "Ein Prüfer kann eine Veröffentlichung erst genehmigen, nachdem der Sicherheitsbericht vollständig ist, es sei denn, der Vorfallkommandant autorisiert eine Notfallausnahme." Ein Formalisierer muss mindestens den Akteur, die Handlung, das Objekt, die zeitliche Bedingung, den Abschlusszustand, die Ausnahme, die Autorität und den Umfang von "nur danach" bewahren. Sie muss die Erlaubnis von der Verpflichtung unterscheiden. Sie muss entscheiden, ob die Ausnahme nur die zeitliche Bedingung entfernt oder die Genehmigung generell genehmigt. Es muss „den Sicherheitsbericht“ mit dem richtigen Artefakt und „dem Vorfallkommandanten“ mit einer Identität oder Rolle verbinden. Wenn eine dieser Entscheidungen falsch ist, kann ein Löser die falsche Richtlinie korrekt durchsetzen.

Quantifier-Bereich ist eine klassische Fehlerquelle. "Jeder Forscher überprüfte ein Papier" kann bedeuten, dass jeder Forscher mindestens ein Papier überprüft hat, nicht unbedingt dasselbe. "Ein Papier wurde von jedem Forscher überprüft" hat eine andere Struktur. Negation schafft ähnliche Probleme: „nicht jede Probe wurde übergeben“ bedeutet nicht, dass keine Probe bestanden wurde. Bedingungen, Vergleiche, Modalitäten, zeitliche Beziehungen, Anaphoren und Voraussetzung schaffen Unterscheidungen, die die oberflächenebene Translation löschen kann. Formale semantische Traditionen entwickelten ausgeklügelte Mechanismen für diese Phänomene, aber die Sprache der offenen Domäne fügt Weltwissen, Genrekonventionen und implizite Ziele hinzu, die keine einzelne Repräsentation vollständig erfasst.

Die Lücke zeigt sich auch in der semantischen Parsierung von den Anmerkungen. Ein System, das nur auf eine Frage und Endantwort trainiert wird, kann ein Programm finden, das zufällig die richtige Antwort auf die Trainingsumgebung hervorbringt. Solche sporigen Programme sind ein bekanntes Problem beim schwach überwachten semantischen Parsen [Pasupat and Liang 2016; Guu et al. 2017; Goldman et al. 2018]. Die Ausführung unterscheidet das beabsichtigte Programm nicht von einem zufälligen, wenn beide die gleiche Bezeichnung liefern. Text-to-SQL-Systeme stehen vor dem gleichen Risiko: Eine Abfrage kann aufgrund der aktuellen Datenwerte die richtigen Zeilen zurückgeben, während sie die falsche Verknüpfung oder den falschen Filter enthält. Die Ausführungsgesteuerte Dekodierung entfernt Programme, die Zwischenbeschränkungen abstürzen oder widersprechen, aber keine semantische Genauigkeit [Wang et al. 2018] nachweisen können.

Die Formalisierungslücke wird am besten in getrennte Verpflichtungen zerlegt. Die lexikalische Erdung fragt, welche Prädikate oder Felder Wörtern und Phrasen entsprechen. Die Referenzierung fragt, welche Erwähnungen die gleiche Entität bezeichnen. Die strukturelle Grundlage fragt, welche Argumente, Zielfernräume und Abhängigkeiten zusammengehören.

Zeitliche Erdung verbindet Phrasen mit Momenten, Intervallen, Kalendern und Reihenfolge. Modale Erdung unterscheidet Fakten, Möglichkeiten, Verpflichtungen, Genehmigungen, Absichten, gemeldete Ansprüche und Kontrafakten. Die ontologische Erdung bestimmt, ob ein Konzept zum ausgewählten Domänenmodell gehört. Evidential Grounding verbindet jede Prämisse mit einer Quellspanne, einem Sensor, einer Datenbankzeile oder einer deklarierten Annahme. Jede Verpflichtung kann selbständig scheitern.

Das System muss auch entscheiden, wie viel es bedeutet, zu formalisieren. Eine totale semantische Darstellung uneingeschränkter Prosa ist für die meisten Aufgaben weder praktisch noch notwendig. Die Ausführungsregel schafft Nachfrage. Eine Terminologieschaltung benötigt möglicherweise nur exakte Spannen und normalisierte Begriffe. Eine zeitliche Compliance-Regel kann Ereignisse, Akteure, Termine und Ausnahmen erfordern. Eine kausale Analyse kann Behauptungen, Interventionen, Ergebnisse und Unsicherheit erfordern. Die nachfrageorientierte Formalisierung reduziert die Kosten und begrenzt die Anzahl der nicht unterstützten Verpflichtungen. Es schafft auch ein explizites Abdeckungsproblem: Wenn eine Regel ein Konzept erfordert, das die verfügbaren Hersteller nicht verwirklichen können, muss der Lauf anhalten oder unbekannt zurückgeben, anstatt sich stillschweigend zu nähern.

Mehrdeutigkeit sollte als Alternativen dargestellt werden, nicht sofort zusammengebrochen. Angenommen, "Alex sagte Jordan, dass sie gehen sollten" erscheint in einem Verfahren. Das Pronomen kann sich auf Alex, Jordan oder eine Gruppe beziehen, abhängig von Kontext und Sprachkonventionen. Ein Modell kann Kandidaten einordnen, aber ein hohes Vertrauen ist kein Beweis. Wenn das Urteil der nachgelagerten Regel unter allen plausiblen Bindungen gleich ist, ist die Zweideutigkeit irrelevant und die Ausführung kann fortgesetzt werden. Wenn sich das Urteil ändert, sollte das Ergebnis die Abhängigkeit aufdecken und eine Klärung oder einen zusätzlichen Kontext anfordern. Dies ist ein Ort, an dem abstrakte Interpretation konzeptionell nützlich wird: Sie kann Schlussfolgerungen über eine Reihe von Interpretationen berechnen, ohne jede Kombination in vollen Details aufzuzählen.

Eine vollständige Kartierung ist oft unmöglich, weil natürliche Sprache offen ist. Ein Dokument kann Fakten auslassen, die wahr sind, allgemein bekannt sind oder sich auf institutionelle Praxis verlassen. Closed-World-Gründe behandeln nicht aufgezeichnete Fakten als falsch; Open-World-Gründe behandeln sie als unbekannt. Beide Politiken sind in unterschiedlichen Kontexten geeignet. Ein Datenbankinventar kann für eine bestimmte Tabelle und Zeit geschlossen werden. Eine Sammlung abgerufener E-Mails ist im Allgemeinen kein vollständiger Nachweis, dass keine Benachrichtigung gesendet wurde. Die Erdungsschicht muss daher die Abdeckung an Beobachtungen und Suchanfragen anhängen. Negative Schlussfolgerungen erfordern ein Deckungstoken, das die Domäne, die Quellrevision, den Umfang, die Methode, die Ausschlüsse und die Vollständigkeitsrichtlinie angibt.

Externes Wissen führt zu weiteren Komplikationen. Ein allgemeines LLM enthält umfangreiche, aber nicht versionierte und potenziell veraltete Informationen. Ein symbolisches System kann Modellgedächtnis nicht als zeitlose Tatsache behandeln. Weltwissen sollte durch quellengebundene, datierte, gerichtsbarkeitsspezifische und inhaltsbezogene Pakete eingeben. Eine Packung kann angeben, dass eine Rechtsvorschrift in einem bestimmten Gerichtsstand während eines bestimmten Intervalls wirksam war oder dass eine wissenschaftliche Konstante aus einer benannten Quelle stammte. Konflikte zwischen den Packungen sollten sichtbar bleiben. Fiktionale oder hypothetische Welten müssen ohne Kontamination durch aktuelle

Weltfakten wählbar sein. Die nllAgent-Dokumentation behandelt solche Pakete korrekt als Grenze, anstatt so zu tun, als ob der Speicher eines Gründungsmodells formale Autorität bietet.

Die Treue der Formalisierung kann getestet werden, wenn auch nicht allgemein bewiesen. Eine Methode ist die bidirektionale Überprüfung: Übersetzen Sie das formelle Objekt zurück in die kontrollierte Sprache und vergleichen Sie es mit der Quelle. Ein anderer ist kontrastive Tests: Generieren Sie minimale Textänderungen, die ein formales Element ändern und den Rest bewahren sollten. Ein dritter ist der Erweiterungsvergleich: Fragen Sie konkurrierende Formalisierungen, um Unterscheidungsbeispiele oder Abfrageergebnisse zu generieren. Ein viertes ist die Paraphrasenstabilität: Semantisch äquivalente Formulierungen sollten äquivalenten oder nachweislich verwandten Artefakten zusammengestellt werden. Ein Fünftel ist eine Gegenbeispiel-Erklärung: Wenn der Löser einen Zeugen produziert, kann jede Abhängigkeit zur Quelle umfassen und fragen, ob das Szenario sprachlich zulässig ist. Eine sechste ist die menschliche Überprüfung, die sich auf die kleinen Entscheidungen konzentriert, die das Ergebnis betreffen, und nicht auf das gesamte formelle Programm.

Interaktive Systeme wie nl2spec veranschaulichen den Wert der lokalen Reparatur. Anstatt den Benutzer zu bitten, eine ganze zeitliche Formel zu verstehen oder umzuschreiben, verknüpft die Schnittstelle Unterformeln mit Quellfragmenten, so dass eine falsche Komponente [Cosler et al. 2023] korrigiert werden kann. Solver-Feedback kann eine ähnliche Rolle spielen. Logic-LM verwendet Syntax- und Ausführungsfehler, um die Reformalisierung von [Pan et al. 2023] zu veranlassen. Fortschrittlichere Systeme können unzufriedene Kerne oder Gegenbeispiele anstelle von generischen Fehlermeldungen verwenden. Die Kerneinsicht ist, dass formales Feedback die sprachliche Frage einschränken sollte.

Die Formalisierungslücke setzt auch End-to-End-„Soundness“-Ansprüche. Ein System kann sich als Satz der Form erweisen: Wenn jede akzeptierte Beobachtung genau ihre Quelle darstellt und die Ontologie und die Regeln korrekt sind, dann folgt das Urteil. Das ist wertvoll. Es sollte nicht umgeschrieben werden, da „das Urteil aus dem Dokument bewiesen ist“, es sei denn, die Beobachtungs- und Erdungsverpflichtungen wurden ebenfalls erfüllt. Die Garantie sollte ihre Annahmen und Unbekannten benennen. Ein Klangsicherheits-Hüllraum ist ehrlicher als ein binäres Vertrauenslabel.

Dieses Buch behandelt Formalisierung daher als einen Zusammenstellungsprozess mit mehreren Zwischenartefakten und nicht mit einer einzigen Aufforderung. Die Quelle ist segmentiert und verankert. Kandidatenbeobachtungen werden unter Schemata erstellt. Identitäts- und Zeitalternativen sind vertreten. Die Ontologie-Kompatibilität wird überprüft. Es wird ein Zielformalismus gewählt. Der Compiler erzeugt eine ausführbare Objekt- und Quellkarte. Der Testamentsvollstrecker produziert einen Zeugen, Beweis, Modell oder Misserfolg. Das Ergebnis wird wiedergegeben und gerendert. In jeder Phase hat ein Fehler eine bestimmte Kategorie und eine mögliche Reparatur. Die Architektur beseitigt keine semantische Unsicherheit; sie verhindert, dass Unsicherheiten in fließender Leistung verborgen werden.

Es folgt die wichtigste Schlussfolgerung der Forschung. Solvergestützte LLM-Systeme sollten separat auf Formalisierungstreue und formale Ausführung bewertet werden. Eine hohe letzte Antwort kann falsche Programme verbergen. Eine niedrige Punktzahl kann durch eine gute Formalisierung mit einem unzureichenden Löser oder durch eine schlechte Formalisierung mit einem perfekten Löser verursacht werden. Benchmarks benötigen Gold oder zulässige formale Strukturen, Beweisausrichtungen und semantische Minimalpaare, nicht nur Antworten.

Bis diese Trennung zum Standard wird, werden viele Behauptungen über „formales Denken mit LLMs“ schwer zu interpretieren bleiben.

2. Die intellektuelle Abstammung der ausführbaren Sprache

2.1 Kontrollierte Sprachen, formale Semantik, Diskurs und natürliche Logik

Die Idee, dass eine menschenlesbare Sprache die mechanische Inferenz unterstützen kann, datiert große Sprachmodelle um Jahrzehnte. Frühere Arbeiten näherten sich dem Problem aus der entgegengesetzten Richtung: Anstatt ein statistisches Modell zu bitten, uneingeschränkte Prosa zu interpretieren, entwarfen die Forscher Sprachen oder semantische Theorien, die die Mehrdeutigkeit begrenzen, um die Übersetzung in die Logik zu unterstützen. Diese Traditionen sind unerlässlich, weil sie zeigen, was ein ausführbares Sprachsystem erhalten muss, auch wenn ein LLM die Front-End-Analyse durchführt.

Kontrollierte Natursprachen sind der klarste Präzedenzfall. Sie schränken Vokabeln, Grammatik oder Interpretation ein, während sie eine Oberfläche beibehalten, die der gewöhnlichen Sprache ähnelt. Attempto Controlled English ist das bekannteste Beispiel. Sätze werden in Diskursrepräsentationsstrukturen und dann in formale Logik übersetzt, die Abfragen und Inferenzen ermöglicht und gleichzeitig für Nicht-Logiker [Fuchs and Schwitter 1996; Fuchs et al. 2008] lesbar bleibt. Kontrollierte Sprachen wurden für die Wissensrepräsentation, technische Spezifikationen, semantische Wikis, Geschäftsregeln und Anforderungen verwendet. Kuhns Umfrage zeigt, dass die Familie vielfältig ist: Einige Sprachen sind eher auf menschliche Klarheit als auf formale Präzision ausgelegt, während andere Natürlichkeit gegen deterministische Semantik [Kuhn 2014] eintauschen.

Die dauerhafte Lektion ist nicht, dass uneingeschränkte Sprache durch einen kontrollierten Dialekt ersetzt werden sollte. Es ist, dass ein ausführbares System eine Normalisierungsschicht benötigt, deren Semantik strenger ist als gewöhnliche Prosa. Ein LLM kann als Übersetzer von der offenen Sprache in eine kontrollierte semantische Form dienen, aber die kontrollierte Form muss eine unabhängige Grammatik und Interpretation haben. Ohne diese Grenze wird „kontrollierte natürliche Sprache“ nur zu einem bevorzugten Schreibstil und bietet keine Rechengarantie.

Kontrollierte Sprachen zeigen auch die Bedeutung des Autorenfeedbacks. Wenn sich ein Satz außerhalb des kontrollierten Fragments befindet, lehnt ein traditionelles System ihn ab oder erzeugt eine explizite Zweideutigkeit. Moderne LLM-Schnittstellen neigen dazu, das Gegenteil zu tun: Sie wählen ein plausibles Lesen aus und fahren fort. Ablehnung wird oft als Usability-Defekt behandelt, aber in einem hochsicheren System ist es eine Form der epistemischen Integrität. Eine ausführbare Sprachschnittstelle der nächsten Generation soll die Flexibilität der LLM-Paraphrase mit der Ehrlichkeit eines kontrollierten Compilers kombinieren. Es kann mehrere normalisierte Messwerte vorschlagen, die Unterscheidungen erklären und nur fragen, wann die Unterscheidung das Ergebnis ändert.

Die formale Semantik bietet eine tiefere theoretische Abstammung. Die modelltheoretische Semantik repräsentiert Bedeutung in Bedingungen, unter denen Ausdrücke in einem Modell wahr sind. Die Kompositionalität verbindet die Interpretation eines komplexen Ausdrucks mit den Bedeutungen seiner Teile und deren syntaktischer Kombination. Montague-artige Semantik, typisierte Lambda-Calculi, generalisierte Quantifizierer und verwandte Frameworks bieten

präzise Berichte über Umfang, Argumentstruktur und logische Operatoren. Sie zeigen, dass die natürliche Sprachbedeutung nicht auf eine flache Liste von Triples reduziert werden kann. Quantifizierer, Negationen, Intensional-Verben, Modalität und Kontext können das logische Verhalten scheinbar einfacher Phrasen verändern.

Für ausführbare natürliche Sprache trägt die formale Semantik zwei Anforderungen bei. Erstens muss die Zwischendarstellung die Operatorstruktur und nicht nur die Einheiten und Beziehungen beibehalten. "Jeder", "einige", "nein", "nur", "es sei denn", "möge", "muss" und "früher" sind keine Anmerkungen um einen Satz; Sie bestimmen den Raum der Modelle. Zweitens sollte die semantische Konstruktion typisch sein. Ein zeitliches Intervall, eine Person, eine Organisation, ein Vorschlag und eine Verpflichtung können nicht frei ersetzt werden, nur weil ihre Oberflächenbezeichnungen einander ähneln. Typprüfung ist eine der günstigsten Möglichkeiten, um inkohärente Formalisierungen abzulehnen, bevor Sie sich auf einen Solver berufen.

Diskursrepräsentationstheorie und verwandte dynamisch-semantische Ansätze adressieren Phänomene, die die Satz-für-Satz-Logik schlecht handhabt. Ein Diskurs führt Referenzen ein, trägt sie über Sätze hinweg, aktualisiert den Kontext und schafft Barrierefreiheitsbeschränkungen für Pronomen und Beschreibungen. Boxer- und DRS-Parser operationalisierten diese Ideen für eine breitflächige semantische Analyse [Bos 2008; Liu et al. 2018]. Die Parallel Meaning Bank und die gemeinsamen DRS-Parsing-Aufgaben ermöglichten es, Sprachbezogene Bedeutungsdarstellungen in allen Sprachen [Abzianidze et al. 2019] zu bewerten. Diese Bemühungen zeigen sowohl Fortschritt als auch Schwierigkeit: Variable Bindung, Sinneszuweisung, Umfang und Diskursverhältnisse sind wesentlich schwieriger als die flache Extraktion.

Ein ausführbares System auf Dokumentenebene sollte von der dynamischen Semantik lernen, ohne dass für jede Quelle ein monolithisches DRS erforderlich ist. Es braucht persistente Diskursobjekte, umgefasste Entitätshypothesen, Ereignisse, Welten und Aktualisierungsbeziehungen. Sie muss eine Erwähnung von einer Entität und einem Kandidaten-Identitätslink von einer etablierten unterscheiden. Es muss zitierte, hypothetische, negierte und fiktive Kontexte bewahren. Ein flacher Wissensgraph, der gleiche Zeichenfolgen zu einem Knoten zusammenführt, macht systematische Fehler in langen Dokumenten. Namen wiederholen, Aliase treten auf, Rollen ändern sich und Dokumente beschreiben gegenseitig inkonsistente Welten.

Die praktischen Kosten des vollständigen formalen semantischen Parsings erklären, warum sich viele Systeme in Richtung flacherer Darstellungen bewegten. Die semantische Rollenkennzeichnung identifiziert die Prädikat-Argument-Struktur. Die abstrakte Bedeutungsdarstellung stellt Begriffe und Beziehungen in einem Graphen dar. Die Informationsextraktion identifiziert Entitäten, Ereignisse und Beziehungen. Diese Darstellungen sind nützlich, aber ihre Semantik ist oft unvollständig für die Ausführung. Sie können Quantifiziererbereich, Modalität, zeitliche Verankerung und den Unterschied zwischen behauptetem und gemeldetem Inhalt auslassen. Eine ausführbare Sprachlaufzeit sollte solche Strukturen als evidenzpflichtige Beobachtungen und nicht als vollständige logische Form behandeln. Ein Diagramm kann viele lokale Regeln unterstützen und dabei ungelöste Aspekte explizit lassen.

Natürliche Logik bietet einen weiteren wichtigen Kompromiss. Anstatt Sätze in eine uneingeschränkte Formsprache zu übersetzen, verstehen natürlich-logische Systeme durch

strukturierte Transformationen in der näheren Oberflächenform. MacCartney und Manning entwickelten Inferenzbeziehungen auf der Grundlage von Monotonität, lexikalischer Einbindung und Projektivität [MacCartney and Manning 2007, 2009]. NaturalLI verwendete später Natural-Logic-Operatoren für effiziente Inferenz über Text [Angeli and Manning 2014]. LangPro kombiniert eine formelle Grammatik, lexikalisches Wissen und ein Theorem, das die natürliche Sprachinferenz [Abzianidze 2017] beweist.

Natürliche Logik ist attraktiv, weil viele Dokumentenfragen damit verbundene Beziehungen betreffen, die erfasst werden können, ohne ein großes Weltmodell zu konstruieren. Das Ersetzen von "Hund" durch "Tier" ist in einem nach oben gerichteten Kontext gültig, aber nicht unbedingt unter Negation oder universellen Einschränkungen. Diese Berechnungen sind transparenter als ein Einbettungsähnlichkeits-Score. Sie können Widerspruchsprüfungen, taxonomische Argumentation und kontrollierte Schlussfolgerungen unterstützen. Natürliche Logik ist jedoch kein universeller Vollstrecker. Es handhabt bestimmte Monotonizität und lexikalische Beziehungen gut, drückt aber natürlich keine willkürlichen Verfahren, arithmetischen, Planungs- oder komplexen zeitlichen Verhaltens aus. In einer Multiformalismus-Architektur sollte es sich um einen Motor unter mehreren handeln.

Die semantischen Traditionen klären auch, warum „Erdung“ mehrere Ebenen hat. Die formale Semantik kann angeben, wie ein Ausdruck zu Wahrheitsbedingungen beiträgt, aber sie bestimmt nicht automatisch, welches reale Objekt ein Name bezeichnet oder ob eine Quelle glaubwürdig ist. DRS kann einen Diskurs referentieren, aber Identitätslösung bleibt eine empirische Aufgabe. Natürliche Logik kann lexikalisches Nacheinkommen propagieren, aber die lexikalische Beziehung kann domänenabhängig sein. Die Ausführbarkeit erfordert daher eine Partnerschaft zwischen sprachlicher Struktur, Domänenontologie und Evidenzmanagement.

Eine nützliche Architektur kann mindestens drei semantische Produkte unterscheiden. Das erste ist ein Quellmodell, das genaue Struktur, Spannweiten und Zitate enthält. Die zweite ist eine Interpretationsschicht, die typisierte Erwähnungen, Ereignisse, Beziehungen, Modalitäten und Alternativen enthält. Die dritte ist eine ausführbare Theorie, die ausgewählte Interpretationen verwendet, um eine bestimmte Frage zu beantworten. Formale semantische Techniken beeinflussen in erster Linie die zweite Schicht. Kontrollierte Sprachen und Theoremprüfer verbinden die zweite und dritte. Provenienz und Abdeckung verbinden sich beide zurück zum ersten.

Die frühere Literatur warnt auch davor, anzunehmen, dass eine Repräsentation jeder Aufgabe dienen kann. DRS ist ausdrucksstark für Diskurs, aber teuer zu produzieren und zu argumentieren. Logik erster Ordnung ist gut verstanden, aber unangenehm für zeitliche, probabilistische oder verächtliche Phänomene. Beschreibungslogiken bieten eine aussehbare Ontologie-Vermutung, schränken jedoch die Ausdruckskraft ein. Die natürliche Logik bleibt der Sprache nahe, ist aber spezialisiert. Controlled English verbessert die Vorhersehbarkeit durch Einschränkung der Eingabe. Die richtige Schlussfolgerung besteht nicht darin, einen Gewinner zu wählen, sondern genügend semantische Informationen zu bewahren, um eine Aufgabe in den Formalismus zu reduzieren, der dazu passt.

LLMs verändert die Wirtschaftlichkeit dieses Designs. Sie können Kandidatenparsen generieren, unkontrollierte Sprache in kontrollierte Formen umschreiben, Ontology-Mappings vorschlagen und formale Alternativen für Benutzer erklären. Sie machen es praktisch, semantische Interpretationen auf Dokumenten zu versuchen, die handschriftliche Grammatiken ablehnen würden. Sie beseitigen nicht die Notwendigkeit der semantischen Unterscheidungen,

die durch frühere Arbeiten entwickelt wurden. Im Gegenteil, diese Unterscheidungen werden zu den Verträgen, nach denen LLM Vorschläge beurteilt werden.

Ein wiederkehrender Fehler in zeitgenössischen Systemen besteht darin, einen LLM-generierten Triple Store als eine vollständige semantische Darstellung zu behandeln. Triples sind bequem für Abruf- und Graphabfragen, aber sie kodieren nicht selbst Umfang, Provenienz, Modalität, Zeit, Negation, Identitätsalternativen oder Abdeckung. Eine ausführbare semantische Schicht sollte reichhaltiger, aber dennoch endlich und betriebsbereit sein. Es kann eine Assertion als typisiertes Objekt mit Teilnehmern, zeitlichem Kontext, Polarität, Modalität, Welt, Beweis, Produzent und Status speichern. Es kann alternative Bindungen darstellen, ohne eine zu wählen. Sie kann erklären, welche Dimensionen fehlen. Dies ist weniger elegant als eine einzige logische Form, aber es ist mehr treu, wie reale Systeme Bedeutung erlangen.

Der tiefste Beitrag der vor-LLM-Linie ist daher die konzeptionelle Disziplin. Die menschliche Lesbarkeit bedeutet keine semantische Determination. Die formale Determination wird durch Einschränkung, Tippen, expliziten Kontext und Ablehnung von nicht unterstützten Konstruktionen erreicht. Die Bedeutung des Dokuments ist dynamisch und umgespannt. Inferenz in der Nähe von Oberflächensprache ist möglich, aber spezialisiert. Mehrere formale Darstellungen sind legitim, weil sie verschiedene Fragen beantworten. Diese Lektionen definieren die Architektur, die ein LLM-basierter Compiler wiederherstellen muss, anstatt zu umgehen.

2.2 Semantisches Parsing, ausführbare Abfragen, Programminduktion und frühe neurosymbolische Systeme

Semantisches Parsen wandelt natürlichsprachliche Äußerungen in formale Objekte um, die gegen eine Datenbank, Wissensdatenbank, Umgebung oder einen Interpreter ausgeführt werden können. Es ist der nächstgelegene historische Vorgänger der hier vorgeschlagenen ausführbaren Sprachpipeline. Das Feld umfasst Fragen zur Beantwortung von Tabellen und Wissensdiagrammen, Text-zu-SQL, Anweisungen nach unten, Synthese mit regelmäßigem Ausdruck, Interpretation von Roboterbefehlen und Programmgenerierung. Seine Erfolge zeigen die Kraft der äußeren Ausführung; ihre Misserfolge zeigen, warum die Ausführung allein kein Verständnis schafft.

Frühe semantische Parser stützten sich auf Grammatiken, Lexikone und überwachte logische Formen. Sie könnten transparente Darstellungen erzeugen, erforderten aber teure Anmerkungen und wurden oft schlecht über Domänen übertragen. Die schwache Aufsicht reduzierte die Annotationsbelastung durch das Training von den Bezeichnungen: Eine Frage wird mit einer Antwort gepaart, und das System sucht nach einem Programm, das es ergibt. Dieser Ansatz ermöglichte breitere Datensätze, aber er verursachte das Problem mit dem falschen Programm. Viele Programme können versehentlich die gleiche Antwort zurückgeben, insbesondere in kleinen Tabellen oder Wissensdatenbanken. Das Trainingssignal belohnt dann das Verhalten, ohne die beabsichtigte Semantik [Pasupat and Liang 2016] zu identifizieren.

Das Problem ist strukturell, nicht nur statistisch. Nehmen wir an, die Antwort auf „Welche Stadt war Gastgeber der Veranstaltung 2012?“ ist London. Ein Programm, das die Reihe mit dem Jahr 2012 auswählt, ist vorgesehen. Ein Programm, das die erste Zeile auswählt, kann auch London in der Trainingstabelle zurückgeben. Mehr Daten können zufällige Korrelationen reduzieren, können aber nicht garantieren, dass das gelernte Programm die Sprache widerspiegelt. Abstract-Beispiele, Ausrichtungsmodelle, Typbeschränkungen und ausführungsbasierte Filterungen wurden entwickelt, um falsche Kandidaten [Goldman et al. 2018; Lee et al. 2023] zu unterdrücken. Diese Techniken sind auch für die LLM Formalisierung wichtig. Ein Solver-Ergebnis sollte als ein Test einer Formalisierung behandelt werden, nicht als sein semantisches Zertifikat.

Neuronale symbolische Maschinen verbinden neuronale Programmgenerierung mit einem symbolischen Lisp-Interpreter [Liang et al. 2017]. Die neuronale Komponente schlägt Programme vor, während der Testamentsvollstrecker ungültige Kandidaten ablehnt und die Denotierungen zurückgibt. Mou und Kollegen gekoppelten die verteilte neuronale Ausführung explizit mit der symbolischen Ausführung für natursprachliche Abfragen und berichteten über Verbesserungen in den Bereichen Genauigkeit, Effizienz und Interpretierbarkeit [Mou et al. 2017]. Diese Systeme etablierten ein dauerhaftes Muster: neuronale Modelle behandeln die Sprachvariabilität; symbolische Interpreten erzwingen eine betriebliche Bedeutung.

Text-to-SQL machte das Muster kommerziell wichtig. Eine Benutzerfrage wird in eine Abfrage zusammengestellt, deren Ausführung Daten zurückgibt. Die Ausführungsgesteuerte Dekodierung kann teilweise oder vollständige SQL-Kandidaten ausführen und solche verwerfen, die Fehler oder unmögliche Ergebnisse [Wang et al. 2018] erzeugen. Schema-Verknüpfung schränkt Namen und Beziehungen ein. Moderne LLMs verbessern die Erzeugung erheblich, aber die domänenübergreifende Verallgemeinerung, mehrdeutige Fragen, datenbankspezifische Konventionen und unbeantwortbare Anfragen bleiben schwierig. Die

wichtigste Lektion für ausführbare natürliche Sprache ist, dass die formale Umgebung selbst nützliche Einschränkungen bietet. Spaltentypen, Fremdschlüssel und Abfragefehler geben Feedback, das der reinen Sprachmodellierung fehlt.

Text-zu-SQL zeigt jedoch auch die Grenzen einer einzigen letzten Genauigkeitsmetrik. Exakte String-Match kann gleichwertige Abfragen ablehnen. Die Genauigkeit der Ausführung kann semantisch falsche Abfragen akzeptieren, die das gleiche Ergebnis zurückgeben. Eine strenge Bewertung erfordert eine Äquivalenz unter verschiedenen Datenbankinstanzen oder sorgfältig konstruierten Gegenbeispielen. Dies ist direkt analog zur Bewertung der Regelformalisierung: Ein generiertes formales Programm sollte auf Unterscheidungsszenarien getestet werden, nicht nur auf einer beobachteten Antwort.

Die Programmsynthese aus der natürlichen Sprache verallgemeinert die Idee. Systeme generieren reguläre Ausdrücke, Code-Snippets, Datentransformationen oder kleine domänenspezifische Programme. Beispiele können den Kandidatenraum einschränken, und Typsysteme können unsinnige Programme beseitigen. Die neuronale Erzeugung von regulären Ausdrücken zeigte, dass Language-to-Program-Mapping auch mit begrenztem Domänenwissen funktionieren kann, während sie auch Sprödigkeit veranschaulicht, wenn Beschreibungen unterspezifiziert sind [Locascio et al. 2018]. Programmgestützte LLMs bewegte später die Berechnung außerhalb des Modells, aber das grundlegende Syntheseproblem blieb: Das Modell muss ein Programm wählen, dessen Semantik der Anforderung entspricht.

Ein robuster Compiler sollte daher mehrere Formen von Beweisen suchen. Natürliche Sprachbeschreibungen schränken die Absicht ein. Eingabe-Ausgabe-Beispiele schränken das Erweiterungsverhalten ein. Typen schränken zulässige Vorgänge ein. Domänenschemata beschränken den Wortschatz. Testet Sonden-Kantenfälle. Ein generiertes Programm, das all diese Einschränkungen erfüllt, ist glaubwürdiger als eines, das nur ausgeführt wird. Dieses Multi-Evidenz-Prinzip kann auf Dokumentenregeln angewendet werden: Textautorität, positive und negative Beispiele, erwartete Befunde und semantische Mutationen sollten alle eine Schaltung vor der Veröffentlichung prägen.

Die Logikprogrammierung bietet ein weiteres ausführbares Ziel. Datalog-Regeln berechnen Konsequenzen für einen Fixpoint und unterstützen eine inkrementelle Bewertung. Prolog und verwandte Sprachen repräsentieren Beziehungen und Suche. Die abstrakte Interpretation wird seit langem zur Analyse von Logikprogrammen verwendet, die eine konzeptionelle Brücke zwischen deklarativer Semantik und Programmanalyse zeigen. Für natürlichsprachliche Anwendungen eignet sich die Logikprogrammierung gut für monotone Schließungen: transitive Beziehungen, Taxonomien, Abhängigkeitsausbreitung und Richtlinienregeln ohne komplexe Standardeinstellungen. Es ist weniger geeignet, wenn das Denken stark von Unsicherheit, inkonsistenten Quellen oder nichtmonotonen Ausnahmen abhängt, es sei denn, es wurde mit expliziten Mechanismen erweitert.

Entscheidungstabellen und Produktionsregelsysteme werden in akademischen Erhebungen häufig übersehen, sind aber von hoher Relevanz. Viele Politiken sind keine tiefen logischen Theorien; Sie sind endliche Kombinationen von Bedingungen und Ergebnissen. Eine Entscheidungstabelle macht Abdeckung, Konflikte und fehlende Kombinationen sichtbar. Es kann zu ausführbaren Prädikaten zusammengestellt und systematisch getestet werden. Natürliche Sprache kann verwendet werden, um die Tabelle zu autorisieren oder zu erklären, während die Tabelle die maßgebliche Semantik bleibt. Für das Verkleinern von

Unternehmensdokumenten können Entscheidungstabellen sicherer und einfacher zu validieren sein als die willkürliche Logik erster Ordnung.

Zeitliche und verfahrenstechnische Anweisungen erfordern reichere Ziele. Finite-State-Maschinen, Verhaltensbäume, Planungssprachen und zeitliche Logiken stellen geordnete Aktionen und Bedingungen dar. Robotik verwendet seit langem Sprachparser, die ausführbare Primitive komponieren. Neue neurosymbolische Manipulationssysteme setzen dieses Muster fort: Ein Sprachparser bildet eine Anleitung einem Programm ab, dessen Komponenten neuronale Wahrnehmung und symbolische Zählung, räumliches Denken oder Aktionssteuerung umfassen können. Der Erfolg solcher Systeme hängt von einer eingeschränkten Fähigkeitsbibliothek und einer geordneten Umgebung ab, nicht von der uneingeschränkten semantischen Abdeckung [Tziafas and Kasaei 2026].

Probabilistische Logikprogrammierung und differenzierbare neurosymbolische Systeme versuchen eine engere Integration. DeepProbLog, Logic Tensor Networks und verwandte Systeme kombinieren gelernte Prädikate mit der logischen Struktur [Manhaeve et al. 2018; Badreddine et al. 2022]. Sie sind wertvoll, wenn Unsicherheit und Lernen intrinsisch sind, und sie können Einschränkungen in das Training einbringen. Die Differenzierbarkeit ist jedoch nicht gleichbedeutend mit formaler Sicherheit. Fuzzy-Wahrheitswerte und weiche Strafen können Modelle leiten, ohne diskrete Beweisverpflichtungen zu erfüllen. Für ausführbare natürliche Sprache können solche Systeme semantische Kandidaten schätzen oder einordnen, aber eine Akzeptanzgrenze mit hoher Sicherheit profitiert immer noch von einem separaten symbolischen Checker.

Der neurosymbolische NLP-Test 2026 unterscheidet injektive Systeme, die symbolische Informationen in neuronale Modelle zusammenstellen, von föderativen Systemen, die autonome Komponenten erhalten. Es wird berichtet, dass injektive Systeme die Literatur dominieren, aber oft bescheidene Gewinne bei breiten NLU-Aufgaben aufweisen, während föderative Systeme größere Gewinne bei Argumentationsaufgaben aufweisen. Die Autoren warnen ausdrücklich, dass der Vergleich durch verschiedene Benchmarks verwechselt wird und dass die Out-of-Domain-Verallgemeinerung nach wie vor ein ernstes Problem [Chatzikyriakidis and Lappin 2026] ist. Die ausgewogene Interpretation ist, dass die Föderation architektonisch attraktiv für die Wirtschaftlichkeit und Aufgabenzerlegung ist, aber noch nicht empirisch als universelle Skalierungsstrategie bewiesen.

Die semantisch-parsende Linie verdeutlicht auch die Rolle einer Zwischensprache. Eine Darstellung muss ausdrucksstark genug sein, um eine aufgabenrelevante Bedeutung zu erfassen, aber begrenzt genug, um zuverlässig zu kompilieren. Wenn es zu nah an Surface-Text ist, bleibt die Ausführung implizit. Wenn es zu allgemein ist, werden Übersetzung und Solver-Komplexität unüberschaubar. Beiser und Penz beschreiben dies als ein Zwischensprachproblem in zeitgenössischen neurosymbolischen Systemen: Die Wahl eines Formalismus kann sich bewegen, anstatt die Schwierigkeit [Beiser and Penz 2025] zu lösen. Eine Multiformalismus-Architektur reduziert den Druck auf eine Sprache, führt aber eine neue Anforderung an typisiertes Routing und formalismusübergreifende Konsistenz ein.

Ein nützlicher semantischer Drehpunkt sollte daher nicht vorgeben, die endgültige Logik zu sein. Es sollte Beweise und aufgabenneutrale Unterscheidungen – Wesenheiten, Erwähnungen, Ereignisse, Beziehungen, Zeit, Modalität, Mengen, Welten, Herkunft, Alternativen und Abdeckung – bewahren und gleichzeitig Senkungen in spezialisierte Testamentsvollstrecker ermöglichen. Einige Fragen können direkt über den Pivot mit

relationalen Operationen beantwortet werden. Andere benötigen eine kompilierte zeitliche Formel, ein Zwangssystem oder eine Zustandsmaschine. Der Pivot ist wertvoll, weil er das Sprachverständnis von einem bestimmten Löser trennt und gleichzeitig genügend Informationen für die quellenbasierte Ausführung beibehält.

Das Forschungsprogramm vor LLM erzielte einen erheblichen, aber domänengebundenen Erfolg. Es zeigte sich, dass natürliche Sprache in ausführbaren Programmen zusammengestellt werden kann, dass symbolische Dolmetscher die Systematizität verbessern, dass Typen und Schemata Fehler reduzieren und dass schwache Überwachung semantische Abkürzungen erzeugt. Es zeigte sich auch, dass eine breite Abdeckung und exakte Semantik in Spannung sind. LLMs verbessern die Abdeckung und die Kandidatengenerierung erheblich, aber sie erben jedes strukturelle Problem, das durch semantisches Parsen identifiziert wird. Die richtige Synthese besteht darin, die frühere Maschine nicht zu verwerfen. Es soll LLMs vor und um ihn herum platzieren, mit formalen Schnittstellen, die alte Lektionen in neuem Maßstab durchsetzbar machen.

3. Abstrakte Interpretation und symbolische Ausführung für Sprache

3.1 Abstrakte Interpretation als Semantik der gebundenen Unsicherheit

Die abstrakte Interpretation wurde als allgemeine Theorie zur Berechnung sicherer Approximationen des Programmverhaltens [Cousot and Cousot 1977] entwickelt. Ein Programm kann unendlich viele Ein- und Ausgaben haben, was eine erschöpfende Ausführung unmöglich macht. Anstatt jeden Zustand aufzuzählen, stellt eine Analyse Sätze von Zuständen durch abstrakte Eigenschaften wie Intervalle, Zeichen, mögliche Objekttypen, Erreichen von Definitionen oder Alias-Relationen dar. Abstrakte Vorgänge sind so konzipiert, dass jedes konkrete Verhalten, das für die Eigenschaft relevant ist, dargestellt bleibt. Die Analyse kann unmögliche Verhaltensweisen enthalten und daher Fehlalarme melden, aber eine Geräuschüberangleichung lässt nicht stillschweigend ein Verhalten aus, das die konkrete Semantik zulässt.

Die operative Idee ist einfach. Eine konkrete Ausführung verfolgt exakte Werte. Eine Intervallanalyse könnte verfolgen, dass eine Variable zwischen Null und Zehn liegt. Wenn das Programm fünf hinzufügt, erzeugt die abstrakte Transferoperation ein Intervall zwischen fünf und fünfzehn. In einer Filiale werden die Staaten nach dem Zustand verfeinert. Bei einer Zusammenführung werden Informationen verbunden. Für Schleifen und rekursive Prozesse sucht die Analyse nach einem Fixpoint. Die Erweiterung kann die Konvergenz durch Präzisionsopfer erzwingen, während die Verengung einige Details danach wiedererlangen kann. Der mathematische Rahmen liefert ein diszipliniertes Vokabular für Präzision, Klang, Konvergenz und Verfeinerung.

Die Anwendung des Rahmens auf die Sprache erfordert eine konkrete Semantik. Das angenäherte Objekt kann nicht in einem undefinierten Sinne „Bedeutung“ sein. Für eine begrenzte Aufgabe kann die konkrete Domäne die Menge von semantischen Strukturen sein, die unter einer bestimmten Grammatik, Ontologie, Erdungspolitik, Quellenrevision und Hintergrundtheorie zulässig sind. Jede Struktur enthält mögliche Entitäten, Ereigniszuweisungen, zeitliche Beziehungen, Modalitäten und Quellverbindungen. Mehrdeutigkeit schafft mehrere Strukturen. Fehlende Informationen lassen die Dimensionen unbeschränkt. Widersprüchliche Quellen können alternative Welten und nicht eine inkonsistente Union erzeugen.

Ein abstrakter semantischer Zustand fasst diese Menge zusammen. Es kann aufzeichnen, dass sich ein Pronomen auf eine von zwei Personen beziehen kann, dass ein Ereignis innerhalb eines Zeitintervalls eintritt, dass eine Verpflichtung unter einer Interpretation vorliegt und unter einer anderen fehlt, oder dass eine Größe innerhalb eines Bereichs liegt. Eine Muss-Fakte gilt in jeder vertretenen Interpretation. Eine mögliche Tatsache gilt in mindestens einem. Eine nicht-fakte gilt in keiner. Unbekannt weist darauf hin, dass dem System die Darstellung, Abdeckung oder Rechenressourcen fehlen, die zur Entscheidung erforderlich sind. Diese Vier-Wege-Auszeichnung ist informativer als eine Wahrscheinlichkeit allein und ehrlicher, als jede Frage in wahr oder falsch zu zwingen.

Betrachten Sie ein Grundsatzdokument, in dem es heißt: „Der Betreiber sendet die Benachrichtigung nach der Validierung. Wenn der Notfallmodus aktiv ist, kann der Supervisor

auf die Wartezeit verzichten. " Angenommen, es ist unklar, ob sich „die Wartezeit“ auf den Zeitraum vor der Validierung oder auf einen späteren Zeitraum bezieht. Eine einzelne Parse trifft eine spröde Wahl. Ein abstrakter Zustand kann zwei zeitliche Modelle beibehalten. Wenn beide implizieren, dass diese Mitteilung der Validierung folgen muss, ist diese Schlussfolgerung eine Muss. Wenn nur eine Kündigung sofort zulässt, kann ein sofortiger Versand möglich sein. Eine Compliance-Regel kann nur bei Bedarf erkennen, welches Urteil von der mehrdeutigen Phrase abhängt und eine Klärung verlangen.

Der Hauptvorteil ist nicht, dass abstrakte Interpretation die Sprache magisch versteht. Es ist, dass es eine prinzipientreue Ausführungssemantik gibt, sobald eine begrenzte Reihe von Lesungen konstruiert wurde. Es sagt dem System, wie man Alternativen aggregiert, ohne die Möglichkeit mit Notwendigkeit zu verwechseln. Es unterstützt auch modulare Domains. Identität, Zeit, Quantität, Modalität, Provenienz und normativer Status können jeweils ihre eigene Abstraktion haben und dann durch reduzierte Produkte kombiniert werden, die Informationen dort austauschen, wo sie nützlich sind.

Eine Identitätsdomäne kann Kandidatenäquivalenzklassen mit expliziten, unterschiedlichen und ungelösten Beziehungen darstellen. Es darf nicht nur Einheiten zusammenführen, weil Namen übereinstimmen. Ein zeitlicher Bereich kann Teilreihenfolgen, Intervalle oder Differenzbeschränkungen verwenden. Eine Modalitätsdomäne könnte behaupteten, gemeldeten, beabsichtigten, erlaubten, erforderlichen, verbotenen, hypothetischen und kontrafaktischen Inhalt unterscheiden. Ein Mengenbereich kann exakte Werte, Intervalle, Einheiten und Dimensionskompatibilität verwenden. Eine Provenienzdomäne könnte Beweisstatus wie extrahierte, vorgeschlagene, angenommene, abgeleitete, verifizierte und vom Menschen bestätigte Nachweisstatus anordnen, ohne das Vertrauen zuzulassen, um ihren Ursprung zu löschen. Eine Abdeckungsdomäne kann nachverfolgen, ob eine Suche vollständig genug ist, um Abwesenheitsansprüche zu unterstützen.

Die Domains sollten von der zu überprüfenden Eigenschaft ausgewählt werden. Ein Terminologie-Litter braucht kein reiches deontisches Gitter. Eine zeitliche Regel braucht keine vollständige lexikalische Semantik, wenn Ereignisse und Daten bereits materialisiert sind. Diese aufgabengesteuerte Auswahl reduziert die Komplexität und spiegelt die bedarfsgesteuerte statische Analyse wider. In einer langen Dokumentlaufzeit erklären Schaltkreise oder Regeln die Beobachtungstypen und abstrakten Dimensionen, die sie benötigen. Der Compiler materialisiert nur diese Dimensionen und lehnt den Lauf ab, wenn eine kritische Domäne nicht angegeben werden kann.

Transferfunktionen sind der schwierigste Teil. In der gewöhnlichen Programmanalyse wird die Semantik einer Zuordnung oder arithmetischen Operation durch die Programmiersprache festgelegt. In der natürlichsprachlichen Analyse kann die Operation, die den semantischen Zustand aktualisiert, von einem LLM vorgeschlagen werden. Ein Satz könnte ein Ereignis einführen, einen früheren Zustand ändern, eine zeitliche Beziehung herstellen, eine Ausnahme erstellen oder den Glauben einer anderen Person melden. Wenn ein gelernter Übersetzer direkt zur Übertragungsfunktion wird, geht die klassische Klinglichkeit verloren.

Ein vertretbares Design trennt die Kandidatengenerierung von der zugelassenen Übertragung. Der LLM bietet einen endlichen Satz von typisierten semantischen Updates mit Quellspannen und Alternativen. Ein deterministischer Materialisierer bestätigt, dass die zitierten Beweise vorliegen, die Felder einem Schema entsprechen, Einheiten und Typen kompatibel sind, Referenzen auf zulässige Objekte hinweisen und kein Status stillschweigend aktualisiert

wird. Der Abstract-Interpreter wendet dann registrierte Übertragungsfunktionen auf die zugelassenen Datensätze an. Die resultierende Analyse ist relativ zum zugelassenen Kandidatensatz solide, nicht unbedingt relativ zu jeder Interpretation, die ein Mensch erkennen könnte. Diese Konditionalität muss ausdrücklich angegeben werden.

Es gibt mehrere Möglichkeiten, die Annahme der Kandidaten-Sets zu stärken. Mehrere unabhängige Eingabeaufforderungen oder Modelle können Alternativen vorschlagen. Grammatik- oder Ontologie-basierte Generatoren können die Abdeckung von kontrollierten Fragmenten sicherstellen. Paraphrase-Varianten können die Stabilität testen. Kontrastive Beispiele können fehlende Interpretationen aufdecken. Die menschliche Überprüfung kann sich auf ergebnisrelevante Mehrdeutigkeiten konzentrieren. Ein Compiler kann absichtlich innerhalb einer begrenzten Grammatik übergenerieren und spätere Einschränkungen unmögliche Kandidaten beseitigen lassen. Jede Methode handelt mit Kosten, um Vertrauen zu erhalten, dass das Set alle materiell plausiblen Messwerte enthält.

Auch für die Ausbreitung von Dokumentenregeln ist eine abstrakte Interpretation relevant. Viele Flint- und Compliance-Probleme ähneln Dataflow-Analysen. Fakten werden an Quellstandorten generiert, über Bereiche verbreitet, abschnittsübergreifend verbunden und durch spätere Anweisungen ungültig. Eine Definition kann nach ihrer Erklärung, aber nicht vorher verfügbar sein. Ein Objekt kann den Zustand über eine Erzählung hinweg verändern. Eine Anforderung kann innerhalb einer Gerichtsbarkeit oder Version gelten. Eine Behauptung kann später Beweise erhalten. Eine inkrementelle Fixpoint-Engine kann Konsequenzen berechnen und nur betroffene Regionen aktualisieren, wenn sich das Dokument ändert.

Diese Perspektive vermeidet, dass nach jeder Bearbeitung ein allgemeines LLM über ein ganzes Dokument erneut ausgeführt wird. Stabile strukturelle und semantische Artefakte können durch Content Digest zwischengespeichert werden. Ein geänderter Absatz entkräftet lokale Beobachtungen und die nachgelagerten abstrakten Zustände, die von ihnen abhängen. Der Analysegraph identifiziert, welche Schaltungen wiedergegeben werden müssen. Dies ist analog zu inkrementellen Zusammenstellungs- und statischen Analyse-Frameworks. Es macht auch Erklärungen präzise: Das System kann zeigen, welche Quellenänderung die abstrakte Invariante verändert hat und warum.

Erweiterung und Präzisionsverlust haben semantische Analoga. Eine lange Erzählung kann Hunderte von möglichen Objektidentitäten oder zeitlichen Beziehungen enthalten. Jede Kombination zu halten ist unpraktisch. Das System kann "eines dieser Objekte", "einige Zeit innerhalb dieses Intervalls" oder "eine dieser Gerichtsbarkeiten" zusammenfassen. Eine solche Erweiterung kann falsche Warnungen einführen. Verengen oder lokale Verfeinerung kann Präzision um einen vermuteten Verstoß zurückgewinnen. Das Designziel besteht nicht darin, Abstraktionen zu vermeiden, sondern ihren Präzisionsverlust sichtbar und reversibel zu machen.

Die etablierte NLP Literatur zur zertifizierten Robustheit nutzt abstrakte Interpretationsideen auf andere Weise. Jia und Kollegen nutzten die Intervall-gebundene Ausbreitung, um Modelle zu trainieren, deren Vorhersagen gegen jede Wortsubstitution in einer vordefinierten Familie zertifiziert sind und 75 Prozent der gegnerischen Genauigkeit auf IMDB und SNLI unter ihrem Bedrohungsmodell [Jia et al. 2019] melden. Spätere Arbeiten verwendeten randomisierte Glättung, robuste Einbettungen und spezialisierte Transformer-Domänen, um zertifizierte Grenzen [Ye et al. 2020; Wang et al. 2023; Huang et al. 2026] zu verbessern. Dies sind echte Garantien, aber die konkrete Domäne ist eine Reihe von gestörten Modelleingaben, keine

Reihe von semantischen Interpretationen. Die Transformationsfamilie wird äußerlich definiert und soll das Etikett erhalten.

Diese Unterscheidung ist wichtig. Die Bestätigung, dass die Vorhersage eines Klassifikators unter allen Substitutionen in einer Synonymliste invariant ist, beweist nicht, dass jede Substitution die Bedeutung des Satzes im Kontext bewahrt. Es beweist nicht, dass der Klassifizierer im ursprünglichen Satz korrekt ist. Es zertifiziert Robustheit im Vergleich zu einem formellen Störungsset. Diese Literatur ist immer noch hoch relevant, weil sie zeigt, wie abstrakte Domänen über kombinatorische Spracheingaben skalieren können und wie die Verifikationsgenauigkeit von der Darstellung von Korrelationen abhängt. Es liefert auch eine methodische Warnung: Der stärkste formale Anspruch bezieht sich immer auf die tatsächlich definierte Betonmenge.

Eine vielversprechende Forschungsrichtung ist die Verbindung der beiden Verwendungen. Die zertifizierte neuronale Analyse könnte das Verhalten des semantischen Vorschlagsmodells unter kontrollierten Paraphrasen binden, während die semantische abstrakte Interpretation die Konsequenzen alternativer geordeter Interpretationen verbinden könnte. Erstere fragt, ob sich die Extraktion eines Modells unter zulässigen Oberflächentransformationen ändert. Letztere fragt, ob sich eine Schlussfolgerung auf Dokumentebene über zulässige semantische Strukturen hinweg ändert. Zusammen konnten sie erkennen, ob Unsicherheit von Modellinstabilität oder echter semantischer Zweideutigkeit ausgeht.

LLMs kann auch dabei helfen, abstrakte Interpreten zu konstruieren, aber nur unter mechanischer Akzeptanz. SAIL demonstriert dieses Prinzip, indem es mit LLMs nach abstrakten Transformatoren sucht, syntaktische und semantische Einschränkungen durchsetzt und Kosten optimiert, die die Unertlichkeit messen. Die akzeptierten Transformatoren sind global solide, und das System kann präzise Transformatoren für nichtlineare Operatoren synthetisieren, die bisher nicht verfügbar waren [Gu et al. 2026]. Die breitere Lektion ist die Architektur: Generative Modelle können einen immensen Designraum erkunden, wenn ein kleiner formaler Validator die Zulassung kontrolliert.

Direktes Auffordern einer LLM, abstrakte Interpretationen durchzuführen, ist weniger zuverlässig. Experimente an Programmanalyse-Benchmarks zeigen, dass Modelle abstrakten Interpretationsstilen folgen können, aber logische Fehler auf komplexen Strukturen und halluzinären Zuständen [Mitchell et al. 2025] machen. Dieses Ergebnis sollte Vorschläge abschwächen, bei denen ein Modell eine Invariante und die Erzählung als Analyse akzeptiert wird. Das Modell ist wertvoll für die Vorschlag von Invarianten, Domänenpartitionen oder Verfeinerungsprädikaten. Die Fixpoint-Berechnung und die Soliditätsprüfung sollten symbolisch bleiben.

Für ausführbare natürliche Sprache ist abstrakte Interpretation daher weder eine Metapher noch eine Komplettlösung. Es ist eine Theorie für eine disziplinierte Annäherung, nachdem das System definiert hat, was als mögliche Interpretation zählt. Es unterstützt möglicherweise / muss Semantik, inkrementelle Ausbreitung, Domänenzusammensetzung, konservative Abwesenheitsüberlegung und gezielte Verfeinerung. Seine Garantie ist von der semantischen Abdeckung abhängig. Diese Einschränkung ist nicht fatal; es ist der genaue Punkt, an dem modellgestützte Interpretation bewertet und geregelt werden muss.

3.2 Symbolische Ausführung, Constraint-Lösung, Modellprüfung und semantische Verfeinerung

Die klassische symbolische Ausführung führt ein Programm mit symbolischen Eingaben anstelle von spezifischen Werten [King 1976] aus. Ein symbolischer Zustand enthält einen Steuerort, symbolische Ausdrücke für Programmvariablen und eine Pfadbedingung, die beschreibt, welche Eingänge diesen Zustand erreichen. In einer Filiale gibt sich die Hinrichtung. Ein Pfad fügt die Zweigbedingung hinzu, der andere fügt seine Negation hinzu. Ein Constraint-Löser bestimmt, ob die Bedingungen zufriedenstellend sind und kann konkrete Eingaben erzeugen, die einen praktikablen Weg realisieren. Die Methode ist mächtig, weil sie Zeugen hervorbringt: nicht nur eine Warnung, dass ein Fehler existieren kann, sondern eine Eingabe und ein Pfad, die ihn demonstrieren.

Die Schwierigkeit ist die Wegexplosion. Zweige multiplizieren, Schleifen und Rekursion schaffen nicht gebundene Pfade, komplexe Theorien fordern Solver heraus, und externe Bibliotheken sind schwer zu modellieren. Umfragen zur symbolischen Ausführung beschreiben einen großen Gestaltungsraum von Suchstrategien, Zustandszusammenführung, Kompositionszusammenfassungen, konkrolische Ausführung, Umgebungsmodelle und Löseroptimierungen [Baldoni et al. 2018]. Keine einzige Engine untersucht jedes Verhalten echter Software. Symbolische Ausführungsansprüche umfassen daher Grenzen, Modellierungsannahmen und Abdeckungsbeschränkungen.

Natürliche Sprache verbindet sich mit symbolischer Ausführung auf verschiedene Weise. Am einfachsten ist language-to-code: ein LLM generiert ein Programm, und ein herkömmlicher symbolischer Ausführer analysiert dieses Programm. Eine andere ist die Sprach-zu-Spezifikation: Das Modell generiert eine Eigenschaft oder Einschränkung, die mit vorhandenem Code überprüft wird. Ein drittes ist die Ausführung des semantischen Programms: Ein Dokument wird in eine Zustandsmaschine oder ein Regelprogramm zusammengestellt, dessen alternative Interpretationen zu symbolischen Entscheidungen werden. Ein viertes ist die LLM-unterstützte Exploration: Ein Modell priorisiert Pfade, schlägt Zusammenfassungen vor oder Gründe für Einschränkungen, die herkömmliche Löser nicht leicht ausdrücken können.

Die Terminologie muss präzise bleiben. Die Ausführung von SQL, Python oder einer logischen Form, die aus einer Frage generiert wird, ist formale Ausführung, aber es ist nicht unbedingt eine symbolische Ausführung im pfadsensitiven Sinne. Der Unterschied ist wichtig, weil die symbolische Ausführung Wegbedingungen und konkrete Zeugen bietet, während die gewöhnliche Ausführung ein Ergebnis für einen konkreten Zustand liefert. Beides ist nützlich. Eine Übersicht, die sie ohne Unterschied zusammenfasst, lässt Garantien stärker erscheinen als sie sind.

SymGPT bietet eines der klarsten Sprach-zu-Spezifikations-Beispiele. Das System untersucht 132 Natursprachenregeln von drei ERC-Standards, übersetzt sie mit einem LLM in eine eingeschränkte Grammatik, synthetisiert Einschränkungen, die Verstöße beschreiben, und verwendet symbolische Ausführung, um Ethereum-Verträge zu durchsuchen. Die Autoren melden 5.783 Verstöße in 4.000 Verträgen, darunter 1.375 mit expliziten Angriffspfaden für Diebstahl, und berichten über eine stärkere Leistung als sechs automatisierte Techniken und einen Sicherheitsexperten-Auditierungsdienst [Xia et al. 2026]. Die Stärke des Designs liegt in der schmalen Grammatik, der systematischen Zwangssynthese und den konkreten

Programmpfaden. Sein verbleibendes semantisches Risiko liegt darin, ob jede ERC Regel treu übersetzt wurde.

Dieses Muster verallgemeinert sich. Eine Richtlinie kann zu Vorbedingungen, verbotenen Übergängen und Ausnahmen zusammengestellt werden. Ein Verfahren kann in Staaten und Aktionen zusammengestellt werden. Eine narrative Kontinuitätsregel kann Objekte und Ereignisse modellieren und dann nach einem Pfad suchen, in dem ein Objekt verwendet wird, nachdem es ohne Abruf zurückgelassen wurde. Ein klinisches Protokoll kann Berechtigungs- und Interventionssequenzen modellieren. Ein wissenschaftlicher Workflow kann Datentransformationen und Provenienzanforderungen modellieren. In jedem Fall sollte der symbolische Testamentsvollstrecker auf einem typisierten Programm und nicht mit roher Prosa arbeiten.

Die symbolische Ausführung ist besonders nützlich für Gegenbeispiele. Eine abstrakte Analyse kann darauf hindeuten, dass eine Verletzung möglich ist, weil sie eine Korrelation verloren hat. Eine Pfadsuche kann versuchen, konkrete Identitäten, Zeiten, Bedingungen und Ausnahmen zu wählen, die die Verletzung erkennen. Wenn der Pfad zufriedenstellend ist, kann der Zeuge als quellenbasiertes Szenario dargestellt werden. Wenn es unzufrieden ist, erfährt das System, dass die abstrakte Warnung unter der aktuellen Semantik falsch war.

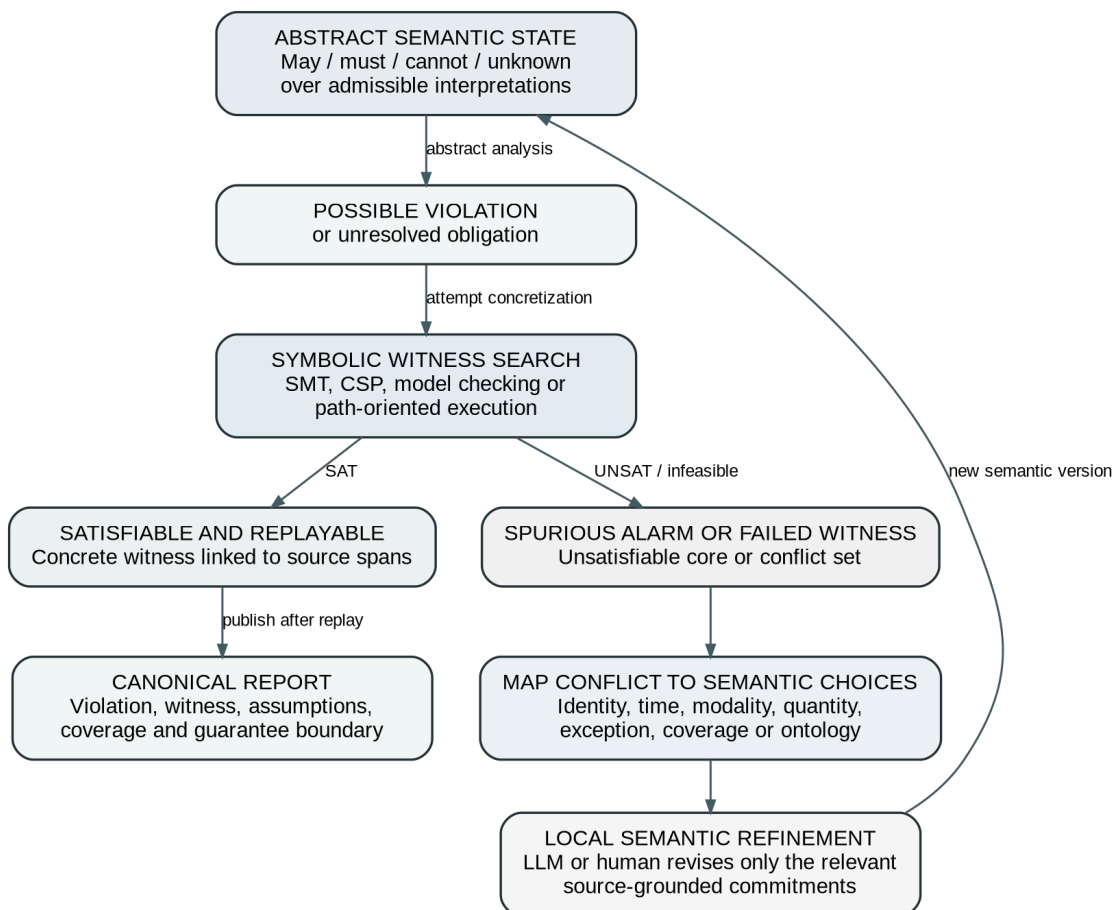


Abbildung 2. Gegenbeispielgeführte semantische Verfeinerung. Abstrakte Analyse identifiziert eine mögliche Verletzung; symbolische Zeugensuche bestätigt entweder einen wiederholbaren Fall oder enthüllt die semantischen Unterscheidungen, die verfeinert werden müssen.

Dies führt zu einer gegenbeispielgesteuerten semantischen Verfeinerung. Traditionelle CEGAR beginnt mit einer groben Programmabstraktion. Die Modellprüfung findet ein abstraktes Gegenbeispiel. Eine Durchführbarkeitsprüfung legt fest, ob das Gegenbeispiel einer konkreten

Ausführung entspricht. Wenn nicht, werden Prädikate hinzugefügt, um die Abstraktion zu verfeinern, und die Analyse wiederholt sich. In der Spracheinstellung enthält die Abstraktion semantische Auswahlmöglichkeiten. Ein falsches Gegenbeispiel kann entstehen, weil zwei Erwähnungen zusammengeführt wurden, eine Ausnahme ignoriert wurde, ein zeitliches Intervall erweitert wurde oder eine modale Aussage als Tatsache behandelt wurde.

Eine semantische Verfeinerungsschleife sollte die symbolische Erklärung des Scheiterns bewahren. Angenommen, eine Compliance-Warnung hängt von der Annahme ab, dass „der Vorstand“ in zwei Abschnitten dasselbe Gremium bezeichnet. Symbolische Wiedergabe zeigt, dass der Pfad undurchführbar wird, wenn die Entitäten unterschiedlich sind. Das System bildet das Identitätsprädikat den relevanten Erwähnungen ab und stellt eine fokussierte Frage: Sollen diese Referenzen die gleiche Tafel bezeichnen? Ein LLM kann den lokalen Kontext erneut untersuchen und Alternativen vorschlagen; ein Mensch kann gegebenenfalls entscheiden. Der Rest des Dokumentenmodells bleibt unverändert. Das ist zuverlässiger und effizienter, als ein Modell zu bitten, alles erneut zu lesen und es erneut zu versuchen.

Unzufriedene Kerne bieten einen verwandten Mechanismus. Wenn eine Reihe von Einschränkungen inkonsistent ist, kann ein Löser oft eine für den Konflikt verantwortliche Teilmenge zurückgeben. Durch die Zuordnung des Kerns auf Quellspannen wird die Mindestmenge von Anweisungen und Interpretationen identifiziert, die nicht nebeneinander existieren können. In einer Policy Review kann dies widersprüchliche Verpflichtungen aufdecken. Beim semantischen Parsen kann es zeigen, dass ein Quantifizierer, eine Identitätsverbindung und eine zeitliche Annahme gemeinsam Unmöglichkeit schaffen. Der Benutzer erhält eine Erklärung, die im formalen Konflikt basiert, anstatt einer allgemeinen Modellkritik.

Nicht jedes Problem wird am besten als Pfadausführung dargestellt. Die Modellprüfung analysiert Übergangssysteme mit zeitlichen Eigenschaften. Die lineare zeitliche Logik kann ausdrücken, dass etwas schließlich vor einem anderen Ereignis geschieht, immer hält oder auftritt. Rechenbaum und probabilistische Logiken drücken verzweigtes oder unsicheres Verhalten aus. Natürlich-Sprache-zu-Zeit-Logik-Systeme wie NL2TL und nl2spec zeigen, dass LLMs zeitliche Spezifikationen konstruieren kann, wenn der Wortschatz und die Domäne [Chen et al. 2023; Cosler et al. 2023] gesteuert werden. Der Executor ist dann ein Modellprüfer oder Planer, kein LLM.

Die Zwangslösung umfasst einen breiten Mittelweg. SAT steht für boolesche Entscheidungen. SMT fügt Theorien wie Arithmetik, Arrays, Strings und nicht interpretierte Funktionen hinzu. CSP und CP-SAT verarbeiten endliche Domänen, Planung und Optimierung. MaxSAT und Soft Constraints drücken Präferenzen aus. Viele natürliche Sprachaufgaben – Planung, Ressourcenzuweisung, Konsistenzprüfung, Familienbeziehungen, Rätsel und Richtlinienkombinationen – werden besser in Einschränkungen zusammengestellt als in Verfahrenscodes. SatLM zeigte, dass deklarative Spezifikationen sowie ein Solver programmgestützte Ansätze zu harten arithmetischen und logischen Aufgaben [Ye et al. 2023] übertreffen können. Das Ergebnis unterstützt eine zentrale architektonische Wahl: Der LLM sollte das Problem beschreiben, während der Löser die Suche durchführt.

Die formale Planung bietet Handlungssemantik und zielgerichtete Suche. Die jüngsten Stellen Stellen LLMs sind eher als Planungsformalisierer als als Planer. Hao und Kollegen übersetzen reale Planungsbeschränkungen in formale Verifikationstools, verwenden unzufriedene Kerne für die Reparatur und berichten von 93,9 Prozent Erfolg auf TravelPlannerr

in ihrer Umgebung, verglichen mit viel niedrigeren Direktmodell-Basislinien [Hao et al. 2025]. Eine Übersicht für 2025 kommt zu dem Schluss, dass die Hauptchance nicht darin besteht, Planer zu ersetzen, sondern mit LLMs Planungsmodelle [Tantakoun et al. 2025] zu konstruieren und zu verfeinern. Genau dafür ist die hier befürwortete Spaltung.

Dataflow- und Datalog-Engines bieten Fixpoint-Ausführung anstelle von Pfadsuche. Sie sind nützlich für monotone Konsequenzen und dokumentenweite Ausbreitung. Automata- und Muster-Engines behandeln geordnete Ereignissequenzen. Entscheidungstabellen behandeln endliche Richtlinien. Die Architektur sollte jedes Problem an den einfachsten Formalismus weiterleiten, der es ausdrückt. Symbolische Ausführung sollte nicht zu einem universellen Label für jede symbolische Komponente werden.

LLMs kann die traditionelle symbolische Ausführung auf sicherere und riskantere Weise unterstützen. Eine sicherere Rolle ist die Pfadpriorisierung. Ein Modell kann Code-Regionen identifizieren, die wahrscheinlich Schwachstellen oder Pfade enthalten, die wahrscheinlich ein Ziel erreichen, während KLEE oder ein anderer Motor die endgültige Machbarkeitsentscheidung beibehält. Jüngste Arbeiten wie KLEECOpilot berichten von wesentlichen Abdeckungsverbesserungen durch LLM-geführte Suche, obwohl der Preprint-Status und die Modellempfindlichkeit eine vorsichtige Interpretation [Chen et al. 2026] erfordern. Das Prinzip ist gesund: Gelehrte Heuristiken entscheiden, wo sie suchen, nicht, ob ein Weg gültig ist.

Eine riskantere Rolle ist die Ersetzung des Löses. AutoBug, das von seinen Autoren als großsprachige, modellgetriebene symbolische Ausführung präsentiert wird, zerlegt Code in pfadbasierte Argumentationsaufgaben und bittet ein Modell, direkt darüber zu argumentieren, ohne alles in SMT zu übersetzen. Der Ansatz ist leicht und sprachunabhängig, und berichtete Experimente verbessern LLM-basierte Programmanalyse [Li et al. 2025]. Die Autoren bezeichnen es als ungefähr. Es kann Konstrukte analysieren, die für herkömmliche Löser schwierig sind, aber es bietet keine klassische Solidität. Für den hochsicheren Einsatz wird ein solches Modell besser als ein Motor in einem Portfolio behandelt, dessen Ergebnisse Tests, Wiederholungen oder unabhängige Bestätigung erfordern.

LLMs kann auch path-satifying Tests, Zusammenfassungen oder Modelle für externe APIs generieren. Studien berichten über nützliche Berichterstattung über Programme, die herkömmliche Tools schwer handhaben müssen, aber die Leistung mit langen Pfaden, komplexen APIs, Rekursion und undurchführbarer Pfadklassifizierung abnimmt. Diese Methoden sind vielversprechend, da Softwareumgebungen Semantik enthalten, die teuer zu axiomatisieren ist. Ihr Ergebnisstatus sollte „Kandidat“ bleiben, bis die konkrete Ausführung den Weg bestätigt oder ein Löser den Zustand beweist.

Sicherheit ist ein großes Problem. Natürliche Sprachregeln und Quelldokumente sind nicht vertrauenswürdige Eingaben. Ein bösesartiges Dokument kann versuchen, eine Eingabe zu veranlassen, Referenzen zu fabrizieren oder einen Codegenerierenden Übersetzer zu nutzen. Formale Artefakte sollten Daten sein, nicht uneingeschränkt ausführbarer Code. Wenn benutzerdefinierte Algorithmen benötigt werden, sollten Hostweiterungen mit gesperrten Digests, eingegebenen Ein- und Ausgängen, Ressourcengrenzen und ohne Umgebungsautorität überprüft werden. Solver-Aufrufe benötigen Zeit- und Speicherbudgets. Ein Zeuge sollte die genaue Version der Theorie, des Compilers und der Engine enthalten, die sie produziert hat.

Das kombinierte Analysemuster ist nun klar. Eine grobe semantische und Dataflow-Analyse identifiziert mögliche Probleme über ein langes Dokument hinweg. Spezialisierte Formmotoren

testen nur die relevanten Scheiben. Symbolische Hinrichtung konstruiert Zeugen für pfadabhängige Ansprüche. Modellprüfung behandelt zeitliche Verhaltensweisen. Constraint-Solver behandeln kombinatorische Entscheidungen. Gescheiterte Zeugen und Löserkerne treiben die lokale semantische Verfeinerung voran. Jede akzeptierte Schlussfolgerung wird mit einem Status und einer Provenienz zurückgegeben, die das schwächste Glied in der Kette widerspiegelt.

Diese Architektur verspricht keine vollständige Überprüfung der uneingeschränkten Sprache. Es verspricht etwas operativeres: Jeder formelle Anspruch hat eine ausführbare Bedeutung; jedes Symbol hat eine Quelle oder eine erklärte Annahme; jedes Gegenbeispiel kann wiederholt werden; jeder Abwesenheitsanspruch ist an die Berichterstattung gebunden; und jede verbleibende Zweideutigkeit wird eher dargestellt als versteckt. Die abstrakte Interpretation bietet Skalierung und konservative Aggregation. Symbolische Ausführung liefert Präzision und Zeugen. Ihre Kombination ist eine der plausibelsten Grundlagen für ausführbare natürliche Sprache.

4. Was die LLM-Ära tatsächlich erreicht hat

4.1 Autoformalisierung, programmgestütztes Denken, Logik, zeitliche Spezifikationen und Planung

Die Ankunft großer Sprachmodelle veränderte die praktische Grenze der semantischen Zusammenstellung. Frühere Systeme erforderten aufgabenspezifische Grammatiken, überwachte logische Formen oder sorgfältig entwickelte Lexika. LLMs kann formale Artefakte aus offenen Beschreibungen mit wenig oder gar keiner aufgabenspezifischen Ausbildung vorschlagen. Dies erweitert die Abdeckung erheblich, schafft aber auch die Versuchung, mehr aus Benchmark-Gewinnen zu schließen, als die Mechanismen rechtfertigen. Die relevante Frage ist nicht einfach, ob eine Methode die Antwortgenauigkeit verbessert. Welche Fehler wurden auf eine symbolische Komponente übertragen, welche Fehler in der Formalisierung verbleiben, und welche unabhängigen Beweise vorliegen, dass das formale Artefakt die Quelle darstellt.

Programmgestützte Sprachmodelle oder PAL bieten einen sauberen Ausgangspunkt. Das LLM liest ein Problem, generiert ein Python-Programm und delegiert die Berechnung an den Interpreter [Gao et al. 2022]. Auf GSM8K verbesserte sich das gemeldete System gegenüber Chain-of-Thought-Produzent erheblich. Der Mechanismus ist überzeugend, weil Sprachmodelle oft besser darin sind, ein Problem zu zerlegen, als lange Rechenkenntnisse genau durchzuführen. Sobald die Zersetzung korrekt ist, liefert Python deterministische Operationen und Status.

Die Beschränkung von PAL ist ebenso lehrreich. Der Interpreter bescheinigt nur, dass er das generierte Programm ausgeführt hat. Es bescheinigt nicht, dass das Programm die natursprachliche Frage originalgetreu modelliert. Eine falsche Variable, eine ausgelassene Bedingung oder eine falsche Formel können fehlerfrei ausgeführt werden. Der Erfolg des Systems hängt daher von einer Verteilung ab, bei der Programme kurz sind, Beispiele die Struktur stark einschränken und Ausführungsfehler viele Fehler aufdecken. Für Dokumenten, Politikanalysen und wissenschaftliche Interpretation ist die Kartierung weniger eingeschränkt und der Dolmetscher liefert schwächere Rückmeldungen.

SatLM ersetzt den Verfahrenscode durch deklarative Einschränkungen [Ye et al. 2023]. Das Modell gibt Variablen und Beziehungen an, und eine Zufriedenheits-Engine sucht nach einer Lösung. Dies kann einfacher sein als die Erzeugung eines Algorithmus, da das Modell nicht über die Reihenfolge der Operationen entscheiden muss. Deklarative Darstellungen ermöglichen auch mehrere Modelle, Undurchführbarkeitsnachweise und Debugging auf Constraint-Ebene. Die gemeldeten Gewinne bei Hard-Reasoning-Aufgaben unterstützen die Idee, dass die Wahl der Solver der Problemstruktur folgen sollte. Ein Zeitplan ist natürlich ein Constraint-Problem; ein dynamischer Prozess kann eine Zustandsmaschine sein; ein arithmetisches Wortproblem kann entweder ein Programm oder Einschränkungen sein.

Logic-LM formalisiert sprachliche Logikprobleme und ruft symbolische Löser auf [Pan et al. 2023]. Es verwendet auch Lösefehler, um die Selbstverfeinerung zu veranlassen. Über mehrere Datensätze hinweg berichten die Autoren von großen durchschnittlichen Verbesserungen gegenüber Standard- und Chain-of-Thought-Prompting. Die Architektur zeigt zwei nützliche Prinzipien. Erstens entfernt die symbolische Engine deduktive Inkonsistenz, sobald eine gültige

Formalisierung existiert. Zweitens bieten formelle Fehlermeldungen ein besseres Reparatursignal als eine allgemeine Anforderung, die es zu überdenken gilt.

Syntaxreparatur ist jedoch keine semantische Reparatur. Eine Formel kann gut geformt, zufriedenstellend und falsch sein. Die Pipeline von Logic-LM kann fehlerhafte Prädikate oder Lösefehler korrigieren, aber sie kann nicht erkennen, dass "einige" "alle" wurden oder dass eine Ausnahme weggelassen wurde. Nachfolgende Arbeiten wie Logic-LM++ fügt mehrstufige Verfeinerung hinzu, und LTRAG ruft gedankengesteuerte Formalisierungsbeispiele ab und meldet Verbesserungen gegenüber Logic-LM und LINC auf FOLIO und AR-LSAT [Kirtania et al. 2024; Hu et al. 2025]. Diese Gewinne deuten darauf hin, dass die Formalisierung selbst von Beispielen und strukturierter Kritik profitiert. Sie beseitigen nicht die Notwendigkeit, die semantische Fidelität direkt zu bewerten.

LINC macht die Modularität explizit: Ein LLM dient als semantischer Parser, die Logik erster Ordnung stellt das Problem dar, und ein Theorem-Prover führt die Deduktion [Olausson et al. 2023] durch. Auf ProofWriter erzielte die gemeldete Kombination große Gewinne gegenüber Chain-of-Thought-Systemen, einschließlich Modelle, die wesentlich größer waren als der Parser. Die Fehleranalyse zeigte komplementäre Fehlermodi: Das symbolische System vermeidet viele Argumentationsfehler, während Formalisierungsfehler bestehen bleiben. Das sind genau die Beweise, die von einer föderierten Architektur erwartet werden. Das System ist am stärksten, wenn die natursprachliche Aufgabe bereits einer kompakten Theorie erster Ordnung entspricht.

FOLIO bietet einen nützlichen Benchmark, da es von Menschen verfasste Natursprachen-Desing-Probleme gepaart mit der verifizierten Logik [Han et al. 2024] erster Ordnung enthält. Es bewertet sowohl das Denken als auch die Übersetzung von NL-to-FOL. Der Datensatz bleibt auch für starke Modelle eine Herausforderung, was die Idee untergräbt, dass die Formalisierung erster Ordnung ein gelöster Vorverarbeitungsschritt ist. FOLIO veranschaulicht auch die Bedeutung der vom Menschen verfassten Komplexität. Synthetische Regeldatensätze können die Sprache ungewöhnlich regelmäßig machen und es Modellen ermöglichen, Vorlagen zu nutzen. Menschliche Beispiele führen lexikalische Variationen, Weltwissen und realistischere Umfangsentscheidungen ein.

FoVer verfolgt einen anderen Ansatz, indem es das natürliche Sprachdenken durch die Logik [Pei et al. 2025] erster Ordnung überprüft. Anstatt fließenden Argumentationstext als selbstbeurteilend zu behandeln, konstruiert er formale Verpflichtungen und überprüft sie. Dies ist ein vielversprechendes Muster für eine nachweisbeschränkte Erzeugung: Ein LLM kann ein Argument vorschlagen, aber ein Prüfer akzeptiert nur inferenzielle Schritte, die gültigen formalen Konsequenzen entsprechen. Das verbleibende Problem ist die Übereinstimmung zwischen jedem Satz und den formalen Prädikaten, die verwendet werden, um ihn zu überprüfen.

Die Autoformalisierungsforschung behandelt die Übersetzung zunehmend als eigenständige Disziplin und nicht als schnellen Trick. Umfragen, die im Jahr 2025 veröffentlicht wurden, katalogisieren die Übersetzung von Theoremprüfern, logischen Sprachen und formalen Spezifikationen und betonen die Notwendigkeit von Feedbackschleifen, Abrufen, Zwischendarstellungen und Benchmarks, die die Syntax von der Semantik [Weng et al. 2025] trennen. Direkte Arbeitsabläufe von NL bis FOL berichten über aussagekräftige Fortschritte, aber auch systematische Fehler in implizitem Wissen, Quantifiziererumfang und Ontologiedesign [Yang et al. 2024; Lalwani et al. 2025]. Das Hinzufügen von implizitem

Hintergrundwissen kann die Aufgabenleistung verbessern und gleichzeitig das Risiko erhöhen, dass das formale System eine Prämisse beweist, die nicht von der Quelle lizenziert wurde.

Die Behandlung von Hintergrundwissen ist eine entscheidende architektonische Wahl. In einem Puzzle-Benchmark können vernünftige Prämissen implizit erwartet werden. Bei der rechtlichen oder wissenschaftlichen Überprüfung kann die Einführung unaufgeforderter Räumlichkeiten das Ergebnis ungültig machen. Ein strenges System sollte Quellenprämissen, Autoritätsgruppen, Domänenaxiome und von Modellen vorgeschlagene Annahmen unterscheiden. Der Löser kann über alle von ihnen grundlegen, aber die endgültige Erklärung muss zeigen, welche Schlussfolgerungen von welcher Autorität abhängen. Wenn eine Annahme wesentlich und nicht verifiziert ist, sollte das Ergebnis bedingt sein.

Zeitlich-logische Übersetzung bringt unterschiedliche Herausforderungen mit sich. NL2TL verwendet angehobene Darstellungen, um die zeitliche Struktur über Domänen hinweg zu erlernen, und berichtet über eine starke Anpassung mit relativ wenig Zieldomänenendaten [Chen et al. 2023]. Die Hebestategie verbirgt konkrete atomare Sätze, während sie wiederverwendbare logische Muster lernt. Dies ist eine wertvolle Design-Einsicht: Trennen Sie die Struktur von "always before", "eventually after" oder "bis" von der domänenspezifischen Grundlage von Ereignissen.

nl2spec betont die Interaktion [Cosler et al. 2023]. Das System ordnet Teile der natürlichsprachlichen Anforderung Teilen der erzeugten zeitlichen Formel ab, so dass Benutzer lokale Fehlübersetzungen reparieren können. Diese Schnittstelle erkennt an, dass Mehrdeutigkeit nicht immer automatisch gelöst werden kann. Es schlägt auch vor, wie die LLM-basierte semantische Verfeinerung funktionieren sollte: Der Benutzer sollte die sprachliche Entscheidung sehen, nicht nur eine Reihe von zeitlichen Operatoren.

Die Erdung bleibt der fragile Punkt. Eine zeitliche Formel kann die richtige Form haben, während sich ihre atomaren Sätze auf die falschen Ereignisse oder Objekte beziehen. Die Arbeit an fundierten logischen Äquivalenzberichten, dass explizite Erdungsmechanismen die Korrektheit verbessern und die Unterscheidung zwischen Formel-Syntax und semantischer Bindung verstärken. Für eine ausführbare Sprachlaufzeit sollten zeitliche Atome auf typisierte Beobachtungen mit Spannweiten und Identitäten verweisen, nicht auf Freiformetiketten, die vom Übersetzer erfunden wurden.

Die automatisierte Planung bietet ein besonders ausgereiftes symbolisches Backend. Planungssprachen definieren Typen, Prädikate, Aktionen, Voraussetzungen, Effekte, Anfangszustände und Ziele. Planer suchen dann nach einer gültigen Aktionssequenz. LLMs sind schlechte Long-Horizont-Planer, wenn sie intern Zustand und Einschränkungen aufrechterhalten müssen, aber sie können effektiv sein, um ein Problem für einen Planer zu formalisieren. Eine Erhebung aus dem Jahr 2025 identifiziert Modellbau und Verfeinerung als zentrale Rolle für LLMs in diesem Bereich [Tantakoun et al. 2025].

Hao und Kollegen zeigen eine strenge Version des Musters zur realen Planung [Hao et al. 2025]. Das System formalisiert Einschränkungen, ruft Verifizierungstools auf und verwendet unzufriedene Kerne, um Probleme zu beheben. Die Autoren berichten von 93,9 Prozent Erfolg auf TravelPlannerrr in ihrem experimentellen Umfeld, weit über den direkten LLM Basislinien, die sie vergleichen. Das Ergebnis ist wichtig, weil es zeigt, dass formale Werkzeuge die Planungssicherheit verändern können, wenn das Problem darstellbar ist. Das bedeutet nicht, dass der LLM zu einem Soundplaner geworden ist. Es bedeutet, dass der Planer und der Prüfer

den Long-Horizon-Zustand tragen, während der LLM die formale Instanz konstruiert und repariert.

Die Planung deckt auch versteckte Einschränkungen auf. Natürliche Sprachanfragen enthalten vernünftige Erwartungen, Präferenzen und Machbarkeitsbedingungen, die möglicherweise nicht als formale Anforderungen angegeben werden. Systeme wie LRPlan versuchen die Zusammenarbeit zwischen Sprach- und Argumentationsmodellen für explizite und implizite Einschränkungen, während Benchmarks wie PlanningArena zeigen, dass selbst starke Modelle mit der Werkzeugauswahl, dem Kontextspeicher und der logischen Konsistenz [Karthikeyan et al. 2025; Zheng et al. 2025] zu kämpfen haben. Ein hochsicheres System muss entweder die impliziten Einschränkungen aus einer vertrauenswürdigen Quelle formalisieren oder als Annahmen kennzeichnen. "Commonsense" ist keine ausreichende Provenienzkategorie.

RuleArena und verwandte Benchmarks testen LLMs gegen realistische Regelsysteme und stellen fest, dass die Werkzeugvergrößerung hilft, aber nicht die Formalisierung trivial macht. SemEval 2026 Aufgaben auf Inhalt und formales Denken zeigen weiter, dass Modelle von semantischen Inhalten beeinflusst werden, auch wenn die logische Form die Antwort bestimmen sollte. Modulare neurosymbolische Einträge verwenden FOL und Z3, um die Inhaltsverzerrung zu reduzieren, aber formale Übersetzung und mehrsprachiges Rauschen bleiben schwierig [Grimaldi 2026; Kartáč et al. 2026; Wang et al. 2026]. Diese Ergebnisse verstärken die Notwendigkeit, die sprachliche Interpretation von formaler Inferenz zu entwirren.

Eine der folgenreichsten Beobachtungen in dieser Literatur ist, dass erfolgreiche Systeme zielspezifische Zwischensprachen verwenden. PAL verwendet Python. SatLM verwendet deklarative Einschränkungen. Logic-LM und LINC verwenden Logik. NL2TL verwendet eine zeitliche Logik. Planungssysteme verwenden PDDL-ähnliche Modelle. SymGPT verwendet eine Grammatik, die auf die ERC-Regeln zugeschnitten ist. Das Feld hat keine universelle formale Repräsentation für die natürliche Sprache entdeckt. Es hat festgestellt, dass LLMs Sprache in mehrere formale Ökosysteme leiten kann, wenn Beispiele, Schemata und Feedback verfügbar sind.

Dies unterstützt eine Portfolioarchitektur und nicht einen universellen „Sprachbytecode“. Ein semantischer Pivot kann Beweise, Entitäten, Ereignisse, Umfang und Modalität bewahren, während ein Formalismus-Selektor das Ausführungsziel wählt. Der Selektor selbst kann von einem LLM unterstützt werden, aber seine Entscheidung sollte gegen Fähigkeitsverträge validiert werden. Ein Formalismus ist nur dann vereinbar, wenn er die erforderlichen Unterscheidungen ausdrücken kann und die notwendigen geordneten Beobachtungen existieren. Ein Planer sollte nicht für eine rein taxonomische Abfrage ausgewählt werden; die Logik erster Ordnung sollte sich nicht stillschweigend unsicherer zeitlicher Dynamiken annähern; eine einfache relationale Engine sollte nicht behaupten, verächtliche rechtliche Ausnahmen zu entscheiden.

Die empirische Evidenz stützt daher eine engere, aber substanzielle Behauptung. LLMs kann die Konstruktion ausführbarer formaler Objekte dramatisch verbessern. Externe Symbolsysteme können die Korrektheit, Konsistenz und Interpretierbarkeit verbessern. Solver Feedback kann die Reparatur leiten. Die stärksten Ergebnisse erscheinen in begrenzten Domänen mit expliziten Sprachen, klaren Schemata und auswertbaren Antworten. Die ungelöste Herausforderung ist die semantische Zusammenstellung: sicherzustellen, dass das

formale Objekt die relevante Bedeutung der Quelle bewahrt und dass jede Prämisse geerdet und autorisiert ist.

4.2 LLM-gestützte Programmanalyse, symbolische Exploration, abstrakte Transformatoren und zertifizierte NLP

Die Interaktion zwischen LLMs und Programmanalyse verläuft in beide Richtungen. LLMs kann formalen Tools helfen, Spezifikationen, Codemuster und Umgebungen zu verarbeiten, die schwer zu modellieren sind. Formelle Tools können Code überprüfen, der von LLMs generiert wird, neuronale Netze verifizieren und gelernte Komponenten einschränken. Diese Literatur ist besonders wertvoll für ausführbare natürliche Sprache, weil sie unter klarerer Semantik die gleiche architektonische Frage testet: Wo kann ein probabilistisches Modell beitragen, ohne das letzte Orakel zu werden?

Die traditionelle statische Analyse hängt von handgestalteten Abstraktionen, Transferfunktionen, Zusammenfassungen und Umgebungsmodellen ab. Die symbolische Ausführung hängt von der Pfadsuche und den Lösungscodierungen ab. Diese Tools bieten prinzipientreue Garantien, haben aber Probleme mit dynamischen Sprachfunktionen, Bibliothekscode, komplexer String-Manipulation, externen Diensten und Skalierbarkeit. LLMs kann Codeabsicht erkennen, Invarianten vorschlagen, Funktionen zusammenfassen, wahrscheinliche Fehlerorte identifizieren und Tests generieren. Eine Übersicht der LLM-gestützten Programmanalyse im Jahr 2025 kategorisiert die Arbeit in statische, dynamische und hybride Methoden und betont sowohl die Chancen als auch die Risiken probabilistischer Outputs [Wang et al. 2025].

Eine architektonische Rolle ist die heuristische Führung. Ein symbolischer Ausführer kann seine Semantik beibehalten und einen LLM verwenden, um Pfade zu ordnen, anfällige Regionen zu identifizieren oder Schleifengrenzen auszuwählen. Wenn die Heuristik falsch ist, kann die Berichterstattung leiden, aber akzeptierte Wege bleiben löslich überprüft. Dies ist analog zur Verwendung eines neuronalen Modells für die Suchordnung in der Theorembeweise. Die Unterscheidung zwischen Führung und Akzeptanz ist entscheidend. Ein gelernter Score kann bestimmen, wo Ressourcen ausgegeben werden; es sollte nicht bestimmen, ob eine Pfadbedingung zufriedenstellend ist.

KLEECOpilot vertritt diese Richtung. Der Vordruck 2026 verwendet die LLM-Anleitung, um KLEE auf wahrscheinliche Schwachstellen hinzuweisen, und führt Heuristiken für die Schleifenflucht ein, die erhebliche Verbesserungen in der Basis-Block- und Linienabdeckung und Dutzende von eindeutigen Verstößen [Chen et al. 2026] melden. Als neuerer Preprint erfordern die Ergebnisse eine unabhängige Replikation und eine Sensibilität für modellfamilienangelegenheiten. Dennoch ist das Designmuster plausibel: LLMs tragen zur semantischen Priorisierung bei, die der klassischen Suche fehlt, während KLEE die Ausführungssemantik beibehält.

Eine weitere Rolle ist die Generierung von Pfadbefriedigungstests. Ein Modell kann Code lesen und Eingaben vorschlagen, die einen Zielpfad durchlaufen sollen. Die konkrete Ausführung bestätigt dann, ob der Weg erreicht wurde. Dieser Ansatz kann API-Konventionen oder Datenformate verarbeiten, die für SMT belastend sind. Studien berichten über nützliche Erfolgsraten, aber auch Misserfolge auf langen Pfaden, Rekursion, undurchführbaren Zweigen und komplexen Bibliotheken. Die sichere Interpretation ist, dass der LLM ein Testgenerator ist. Ein Betonlauf, eine Instrumentierungsspur oder ein Löser bleibt der Validator.

AutoBug verfolgt einen radikaleren Ansatz, indem es LLMs verwendet, um das Pfad-zerlegte Denken direkt [Li et al. 2025] durchzuführen. Anstatt alle Einschränkungen in eine Lösertheorie

zu übersetzen, werden Pfadbedingungen auf Codeebene zum Modell dargestellt. Dies kann sprachunabhängig sein und die Semantik analysieren, die schwer zu kodieren ist. Die Autoren berichten über Verbesserungen bei mehreren Programmanalyse-Benchmarks, insbesondere bei kleineren Modellen. Sie charakterisieren die Analyse auch als ungefähr. Das System bietet keine klassische Solidität, und seine Schlussfolgerungen können mit dem Modell variieren.

AutoBug ist wertvoll, weil es zeigt, wo symbolische Methoden operativ scheitern. Viele reale Programme rufen undurchsichtige Bibliotheken auf, manipulieren komplexe Objekte oder hängen von informellen Verträgen ab. Ein Sprachmodell kann wahrscheinlich auf Verhalten von Namen, Kommentaren und gelernten Codemustern schließen. Aber diese Flexibilität ist die gleiche Quelle der Unsicherheit. In einem versicherungsorientierten Portfolio sollte eine ungefähre LLM-Analyse Kandidaten, Zusammenfassungen oder priorisierte Prüfungen generieren. Die Ergebnisse sollten durch Tests, Verträge, Laufzeitüberwachung oder einen formellen Motor bestätigt werden, wenn ein Sicherheitsanspruch erhoben wird.

LLMs kann Invarianten für abstrakte Interpretationen vorschlagen. Eine Kandidateninvariante kann den Nachweis beschleunigen, ob ein Prüfer überprüft, ob er zunächst hält, durch Übergänge erhalten bleibt und die Eigenschaft impliziert. Dies ist eine mächtige Arbeitsteilung: Invariante Entdeckung ist kreativ und kombinatorisch, während invariante Überprüfung mechanisch ist. Ähnliche Muster erscheinen in der Theorem-Nachweis- und Programmsynthese. Das Sprachmodell sucht; der Prüfer akzeptiert.

SAIL erweitert dieses Prinzip auf die Synthese von abstrakten Transformatoren [Gu et al. 2026]. Der Bau eines Transformators, der sowohl klanglich als auch präzise für einen nichtlinearen Operator ist, ist schwierig. SAIL fordert LLMs auf, Kandidaten zu generieren, erlegt jedoch harte syntaktische und semantische Einschränkungen auf und verwendet mathematisch geerdete Kosten für die Unerträglichkeit. Das gemeldete System synthetisiert weltweit Soundtransformatoren über mehrere Domänen hinweg, passt sich manuellen Designs an und findet präzise Transformatoren, die nicht zuvor in der Literatur enthalten sind. Die Bedeutung geht über die Verifikation des neuronalen Netzwerks hinaus. Es ist ein Beweis dafür, dass LLMs die formale Methodenkonstruktion automatisieren kann, wenn jedes akzeptierte Artefakt unabhängigen Beweisverpflichtungen unterliegt.

Der Kontrast dazu, einen LLM zu bitten, „als abstrakter Interpretierer zu fungieren“, ist scharf. Mitchell und Kollegen veranlassten Modelle, durch abstrakte Interpretation der SV-COMP-Programme [Mitchell et al. 2025] zu argumentieren. Modelle könnten in einigen Fällen Kompositions- oder Fixpoint-Stilen folgen, machten aber logische Fehler und halluzinierten auf komplexen Strukturen. Das Ergebnis steht im Einklang mit einem allgemeinen Muster: Modelle können formale Prozesse imitieren und nützliche Kandidaten generieren, aber die Zuverlässigkeit sinkt, wenn der Prozess einen anhaltend genauen Zustand erfordert. Ein echter abstrakter Interpretierer sollte die Gitteroperationen ausführen; das Modell kann Domänen, Prädikate oder Erklärungen vorschlagen.

Die formale Verifikation neuronaler NLP-Modelle ist ein weiterer wichtiger Zweig. Zertifizierte Robustheitsarbeit fragt, ob die Vorhersage eines Modells über alle Eingaben in einem definierten Störungssatz unverändert bleibt. Jia und Kollegen verwendeten Intervall-gebundene Ausbreitung für Wortsubstitutionen und berichteten von 75 Prozent der kontradiktorischen Genauigkeit auf IMDB und SNLI unter den konstruierten Substitutionsfamilien [Jia et al. 2019]. SAFER verwendete randomisierte Glättung und konnte für Black-Box-Modelle gelten, einschließlich BERT [Ye et al. 2020]. Robustheitsbewusste Einbettungen verschärften die

Zertifizierungsgrenzen und verbesserten die zertifizierte robuste Genauigkeit von IMDB von 76,73 bis 84,78 Prozent in der gemeldeten Einstellung [Wang et al. 2023].

Die Verifizierung von Transformatoren ist schwierig, da die Selbstaufmerksamkeit Produkte, Softmax-Operationen und Korrelationen über Positionen hinweg enthält. Spezialisierte Domains verbessern die Grenzen. Parametrisierte abstrakte Interpretation für die Transformatorverifizierung führte parametrisierte affine Domänen für innere Produkte ein und berichtete über engere Grenzen und Überprüfung von Instanzen, die frühere Methoden nicht nachweisen konnten [Huang et al. 2026]. Diese Arbeitslinie zeigt, dass die formale Analyse auf nicht-triviale NLP-Architekturen skalieren kann, wenn die Eigenschaft und die Eingabemenge genau definiert sind.

Der Umfang der Garantie darf nicht verwischt werden. Ein zertifizierter Klassifizierer kann robust für jede Substitution sein, die aus einem bestimmten Set gezogen wird, und immer noch falsch liegen. Eine Substitutionsliste kann kontextuell unangemessene „Synonyme“ enthalten oder sinnvolle Paraphrasen auslassen. Die Zertifizierung besagt, dass das Modell invariante über die formelle Störungsfamilie ist. Sie begründet keine semantische Gleichwertigkeit in der menschlichen Sprache. Diese Unterscheidung sollte in jedem ausführbaren Sprachsystem, das zertifizierte Extraktionsmodelle verwendet, explizit sein.

Eine nützliche Integration würde ein Robustheitszertifikat an eine semantische Beobachtung anhängen. Angenommen, ein Ereignisextraktor weist darauf hin, dass ein Satz eine Verpflichtung angibt. Ein zertifiziertes Modell könnte zeigen, dass diese Klassifizierung unter einer kontrollierten Familie von lexikalischen Substitutionen stabil ist. Die Beobachtung ist noch Modell vorgeschlagen, aber es hat eine zusätzliche Stabilitätseigenschaft. Nachgelagerte symbolische Analyse kann diese Beweise aufzeichnen, ohne die Beobachtung auf Fakten zu aktualisieren. Ein solcher geschichteter Status ist informativer als ein Vertrauenswert.

Formale Methoden verifizieren auch den von LLMs generierten Code. Testgenerierung, statische Analyse, Typprüfung und Beweisassistenten können viele Fehler erkennen. Programmsynthesesysteme verwenden zunehmend Compiler- und Testfeedback für iterative Reparaturen. Das gleiche Muster sollte generierte semantische Schaltkreise und Formalisierungscode regeln. Ein Codierungsmittel kann eine Regelimplementierung erstauern, aber die Veröffentlichung sollte Zusammenstellung, Mutationstests, natürlichsprachliche Benchmark-Fälle, Wiederholungen und unabhängige Prüfer erfordern. Ein LLM sollte nicht erlaubt sein, verstecktes ausführbares Verhalten innerhalb eines deklarativen Theorieartefakts zu erzeugen.

Die Literatur zur Protokollmodellierung aus der Natursprache bietet ein ernüchterndes Ergebnis. Die auf dem ICSE 2025 vorgestellten Arbeiten untersuchten die Erzeugung von Tamarin- oder ProVerif-Modellen aus der Protokolldokumentation. Die Verwendung von Naive LLM war unwirksam; die erfolgreiche Pipeline erforderte eine semantische Zersetzung, eine begrenzte menschliche Disambiguation und Klangmitteltransformationen. Es generierte korrekte Moderat-Skalen-Modelle für 10 von 18 realen Protokollen [Mao et al. 2025]. Das Ergebnis ist aussagekräftig, da Protokolle Rollen, Nachrichten, kryptografische Annahmen, Zustand und zeitliche Ordnung enthalten. Es zeigt sich auch, dass auch starke strukturierte Dokumentation nicht automatisch in formale Sicherheitsmodelle zusammengeführt wird.

Die Fehlerfälle sind lehrreich. Die Dokumentation lässt Annahmen aus, aus denen Experten schließen. Der gleiche Begriff kann einen Nachrichtentyp, einen Zustand oder eine Instanz bezeichnen. Sicherheitseigenschaften können informell oder abschnittsweise verteilt

angegeben werden. Ein formales Protokollmodell muss die feindlichen Fähigkeiten explizit machen. Dies sind direkte Analoga der Dokumentenvernichtung. Die Lösung war keine größere Aufforderung, sondern eine Architektur, die semantisches Parsing, Disambiguation und verifizierte Transformation trennt.

Externe Überprüfungen des breiteren Feldes nähern sich zu einer ähnlichen Schlussfolgerung. Die neurosymbolische NLP-Überprüfung 2026 begünstigt eine stärkere Untersuchung föderativer Architekturen, warnt jedoch davor, dass die aktuellen Vergleiche nicht kontrolliert werden und dass kein föderatives System im Grundstab etabliert wurde [Chatzikyriakidis and Lappin 2026]. Die Planungs-Formalizer-Umfrage behandelt LLMs als Konstrukteure und Raffinerien formaler Modelle, nicht als Ersatz für Planer [Tantakoun et al. 2025]. Die Programmanalyse-Umfrage betont hybride Systeme und die Notwendigkeit, Halluzination und Reproduzierbarkeit [Wang et al. 2025] zu verwalten. Dies sind unabhängige Beweislinien für die gleiche Arbeitsteilung.

Das Feld hat daher mehr als ein oberflächliches „LLM plus Solver“-Muster erreicht. Es hat solvergesteuerte Reparaturen, fachkundige Formalisierungen, formale Planung, Wegführung, invarianten Vorschlag, Transformatorsynthese und zertifizierte Robustheit gezeigt. Es hat auch eine stabile Grenze freigelegt. Wenn der LLM die Endabnahme steuert, werden Garantien ungefähr. Wenn der LLM Artefakte vorschlägt, die ein separates formales System überprüft, können starke Garantien für die geprüfte Eigenschaft beibehalten werden. Die unbeantwortete Frage ist, wie man die Natural-To-Artifact-Grenze vergleichbar diszipliniert.

Die später in diesem Buch vorgeschlagene Architektur wendet den Programmanalyseunterricht direkt an. Die semantischen Transformationen des Kandidaten werden wie synthetisierter Code behandelt. Sie haben Verträge, Quellkarten, Ressourcengrenzen und Tests getippt. Abstrakte Domains werden während eines Laufs registriert und nicht improvisiert. Verfeinerungsprädikate werden von LLMs vorgeschlagen, aber durch konkrete Unterscheidungen validiert. Solver-Portfolios enthüllen Beweisobjekte oder Zeugen. Der Sprachrenderer verbraucht kanonische Ergebnisse, keine rohen Spuren. In diesem Design ist die Flexibilität des LLM ein Vorteil, gerade weil er von symbolischen Akzeptanztoren umgeben ist.

5. Warum ausführbare Sprachsysteme scheitern

5.1 Grounding, Identität, scheinbare Formalisierungen, Ontologien und Weltwissen

Der dominierende Fehler in ausführbaren Sprachsystemen ist kein falscher Abzug von akzeptierten Räumlichkeiten. Es ist die Aufnahme der falschen Räumlichkeiten, Gegenstände oder Beziehungen. Erdung ist der Prozess, durch den Symbole operative Referenz erhalten: Ein Prädikat ist mit einer Phrase verbunden, eine Entität mit einer oder mehreren Erwähnungen, ein Ereignis zu Beweisen, ein Zeitausdruck in ein Intervall, ein Datenbankfeld zu einem Konzept oder eine Aktion zu einer Umgebungsfähigkeit. Ein formelles Programm ohne zuverlässige Erdung ist im geringsten nützlichen Sinne genau. Es führt konsequent aus, während es die falsche Welt beschreibt.

Die Erdung beginnt mit dem Standort der Quelle. Jede semantische Beobachtung sollte mit genauen Quellbeweisen verknüpft oder explizit als externe Annahme gekennzeichnet werden. Ein Zitat muss gegen die unveränderliche Quellenrevision neu validiert werden. Eine Dokumentzusammenfassung kann die Quelle nicht ersetzen, da sie die Qualifikationen auslassen kann. Die Abrufergebnisse können keinen Anspruch auf Abwesenheit stützen, es sei denn, der Abrufprozess ist für die betreffende Domäne abgeschlossen. Diese Quellendisziplin verhindert einen gemeinsamen Ausfall in LLM-Systemen: Ein Modell erzeugt plausiblen unterstützenden Text, der nicht im Dokument auftritt, und die nachgeschaltete strukturierte Ausgabe gibt der Erfindung einen Erscheinungsbild der Autorität.

Die lexikalische Erdung verbindet Sprache mit Ontologie. Der Ausdruck "Zulassung" kann eine Handlung, einen Staat, ein unterzeichnetes Artefakt, eine Erlaubnis oder eine institutionelle Entscheidung bezeichnen. Ein Domänenschema muss die durch eine Regel erforderliche Interpretation auswählen und dokumentieren. LLMs sind nützlich, weil sie auf Kontext schließen und Mappings vorschlagen können, aber ein Mapping sollte anhand von Beispielen und Alternativen getestet werden. Eine Zeichenfolgenübereinstimmung zwischen einem Satz und einem Typnamen ist keine semantische Kompatibilität. Die nllAgent-Dokumentation warnt ausdrücklich davor, zwei Konzepte zu kartieren, nur weil ihre Etiketten übereinstimmen; sie erfordert einen Adapter oder eine semantische Sonde, wenn Bedeutungen unterschiedlich sind.

Identitätsersteigung ist schwieriger und konsequenter. Die Dokumentenanalyse muss oft wissen, ob zwei Erwähnungen dieselbe Person, dieselbe Organisation, ein Objekt, einen Datensatz, eine Anforderung oder ein Ereignis bezeichnen. Gleiche Namen implizieren keine Identität, und verschiedene Namen implizieren keine Unterschiede. Titel ändern sich, Aliase treten auf, Pronomen sind mehrdeutig, Organisationen haben Unterteilungen, und fiktive oder zitierte Erwähnungen gehören möglicherweise nicht zur Quellwelt. Eine sichere Identitätsschicht trennt Erwähnungen von Entitätshypothesen und Identitätskandidaten. Es zeichnet den Produzenten, die Beweise, den Umfang, die Welt und die Alternativen für jeden Link auf.

Vorzeitige Identitätszusammenführung erzeugt kaskadierende Fehler. In einer Erzählung können zwei Zeichen mit demselben Nachnamen zusammengeführt werden, was zu unmöglichen Kontinuitätsverletzungen führt. In einem Verordnungsdokument kann sich „die Behörde“ auf verschiedene Stellen in verschiedenen Rechtsordnungen beziehen. In der Softwaredokumentation kann „client“ einen Benutzer, eine Anwendungskomponente oder einen

HTTP-Endpunkt bezeichnen. Nach dem Zusammenführen behandeln spätere Regeln alle Eigenschaften als zu einem Objekt gehörend. Der Fehler wird schwierig zu diagnostizieren, da der Löser eine konsistente Kennung sieht. Ein konservatives System sollte eine ungelöste Identität ermöglichen und unbekannt zurückkehren, wenn eine Regel einen Link erfordert, der nicht etabliert wurde.

Zeitliche Erdung hat ähnliche Schichten. Termine können explizit oder relativ sein. „Innerhalb von dreißig Tagen“ erfordert ein Ankerereignis und eine Kalenderrichtlinie. "Vor der Veröffentlichung" kann sich auf eine geplante oder tatsächliche Veröffentlichung beziehen. "Nach der Genehmigung" kann unmittelbar nach oder zu einem späteren Zeitpunkt bedeuten. Die Gerichtsbarkeiten definieren Geschäftstage unterschiedlich. Ein zeitlicher Löser kann Intervalle verwalten und genau ordnen, sobald diese Entscheidungen getroffen sind, aber er kann nicht auf den beabsichtigten Anker aus einem beliebigen Etikett schließen. Zeitliche Beobachtungen sollten daher den Textausdruck, den normalisierten Wert oder die Einschränkung, den Anker, den Kalender, die Zeitzone, das Vertrauen und die Alternativen umfassen.

Normative Erdung wird häufig falsch behandelt. "Muss", "sollen", "sollte", "kann", "kann" und "wird erwartet", dass es domänenspezifische Kraft hat. Die rechtlichen Gestaltungskonventionen unterscheiden sich von der gewöhnlichen Sprache. Eine Ausnahme kann eine Regel besiegen, ihren Umfang einschränken oder eine separate Berechtigung erstellen. Widersprüchliche Behörden können Prioritäten haben. Eine einfache boolesche Darstellung der Compliance verliert diese Unterscheidungen. Deontische Logik, Regelprioritäten, Entscheidungstabellen oder verächtliche Systeme können sie modellieren, aber die Quell-zu-Regel-Kartierung bleibt ein sprachliches und institutionelles Urteil.

Spornige Formalisierungen entstehen, wenn das falsche Programm das erwartete Ergebnis hervorbringt. Ein schwach überwacht semantisches Parsen machte dieses Problem deutlich, ist aber in der LLM-Auswertung durchdringend. Eine generierte Formel kann einen Benchmark lösen, da Beispiele regelmäßige Vorlagen enthalten. Eine Text-zu-SQL-Abfrage kann die korrekten aktuellen Zeilen trotz einer falschen Verknüpfung zurückgeben. Ein Planungsmodell kann einen Plan finden, weil eine ausgelassene Einschränkung in der getesteten Instanz irrelevant ist. Eine Regelschaltung kann zehn Beispiele passieren, während sie das falsche Konzept kodiert. Die Genauigkeit der letzten Antwort allein kann diese Fälle nicht unterscheiden.

Die passende Abwehr ist die semantische Mutation. Der Bewerter verändert einen Aspekt der Quelle oder der Umgebung, während er andere bewahrt, und prüft dann, ob sich die Formalisierung in der erwarteten Weise ändert. Wenn „alle“ zu „einigen“ werden, sollte sich der Quantifizierer ändern. Wenn eine Ausnahme entfernt wird, sollte der zuvor ausgenommene Weg verboten werden. Wenn zwei Entitäten in dieselbe Zeichenfolge umbenannt werden, sollte die Identität nicht automatisch zusammengeführt werden. Wenn sich die Datenbankinstanz ändert, sollte eine versehentlich korrekte SQL-Abfrage fehlschlagen. Mutationstests bewerten die Bedeutung des Programms und nicht nur sein Verhalten auf einer Vorrichtung.

Gegenbeispiel-Generierung kann konkurrierende Formalisierungen vergleichen. Bei zwei logischen oder ausführbaren Programmen kann ein Solver nach einem Modell suchen, in dem sich ihre Ausgänge unterscheiden. Das daraus resultierende Szenario wird in natürliche Sprache übersetzt und einem Rezensenten präsentiert. Dies macht einen undurchsichtigen Formelvergleich zu einer konkreten Frage: „Unter der Interpretation A kann ein Vorgesetzter,

der keine Ausbildung abgeschlossen hat, zustimmen; unter der Interpretation B dürfen sie es nicht. Was ist beabsichtigt?“Die Methode ist besonders nützlich, wenn mehrere LLMs plausible, aber nicht gleichwertige Formen produzieren.

Ontology-Design erzeugt einen weiteren Fehlermodus. Ein universelles Schema ist für die Wiederverwendung attraktiv, kann aber Domänenunstimmigkeiten verbergen und Konzepte in unangemessene Kategorien zwingen. Ein extrem schmales Schema hingegen erzeugt viele inkompatible lokale Sprachen. Bevorzugt ist ein geschichtetes Design. Die obere Ebene definiert Quelle, Beweise, Identitätshypothesen, Welten, Zeit, Modalität, Status, Abdeckung und Lücken. Domain-Pakete definieren inhaltliche Typen und Beziehungen. Mapping-Pakete deklarieren Korrespondenzen, Verluste und Annahmen zwischen Domains.

Die Ontologie-Kompatibilität sollte operativ und nicht rhetorisch sein. Eine Schaltung erklärt die Typen und garantiert, dass sie verbraucht. Ein Hersteller erklärt, welche Typen es materialisieren kann, unter welchen Status- und Deckungsmodi. Die Veröffentlichung prüft, ob ein kompatibler Hersteller existiert. Laufzeitprüfungen, ob das konkrete Dokument die erforderlichen Beobachtungen ergab. Wenn die Schaltung von Menschen bestätigte Identitäten benötigt, aber nur modellträchtige Identitätskandidaten existieren, kann sie nicht bei der beanspruchten Garantie laufen. Wenn eine zeitliche Regel einen Gerichtsstandskalender benötigt, der fehlt, ist das Ergebnis eine operative Lücke, kein vermutetes Datum.

Weltwissen ist besonders gefährlich. LLMs enthalten breites Wissen, aber keine stabile Provenienz, Datum des Inkrafttretens oder Gerichtsbarkeit. Die Behandlung des Modellgedächtnisses als formale Faktenbasis schafft unsichtbare Abhängigkeiten und veraltete Schlussfolgerungen. Ein symbolisches System sollte explizite Wissenspakete mit Quellkennungen, Verdauungen, effektiven Intervallen, Gerichtsbarkeiten, Konflikttrichtlinien und Garantieobergrenzen laden. Die Packung kann kuratiert, abgerufen oder modellunterstützt sein, aber ihre akzeptierten Ansprüche sind externe Artefakte. Ein legales Paket kann ablaufen. Ein wissenschaftliches Paket kann ersetzt werden. Ein fiktives Welt-Pack kann bewusst der aktuellen Realität widersprechen.

Konflikte zwischen Quellen sollten nicht durch Mehrheitsabstimmungen gelöscht werden, es sei denn, die Aufgabe definiert eine solche Politik. Das System kann parakonsistente Argumentation, Quellpriorität, zeitliche Versionierung oder separate Welten benötigen. Ein Anspruch kann von einer Quelle unterstützt und von einer anderen widerlegt werden. Das kanonische Ergebnis sollte sowohl bewahren als auch angeben, welche Politik sie gegebenenfalls löst. Dies ist für wissenschaftliche Erhebungen und regulatorische Analysen von wesentlicher Bedeutung, bei denen Meinungsverschiedenheiten eher Informationen als Lärm sind.

Hintergrundannahmen müssen sichtbar sein. Logik-Benchmarks gehen oft davon aus, dass Namen unterschiedliche Individuen bezeichnen oder dass Kategorien unzusammenhängend sind. Echte Dokumente geben selten alle solche Axiome an. Ein LLM kann sie einfügen, weil sie das Puzzle lösbar machen. In einem Forschungs- oder Compliance-System sollten diese Annahmen in einer separaten Autoritätsebene erscheinen. Die endgültige Antwort kann dann besagen, dass eine Schlussfolgerung nur dann folgt, wenn die Annahme gilt. Dies verhindert, dass sich implizites Weltwissen als Beleg darstellt.

Die Abdeckung ist die Grundlage der Abwesenheit. Um zu dem Schluss zu kommen, dass ein Dokument keine erforderliche Genehmigung enthält, muss das System wissen, was durchsucht wurde, welche Abschnitte und Kanäle enthalten waren, wie das Konzept erkannt

wurde und ob die Suchmethode für den Anspruch vollständig war. Ein lexikalischer Scan kann für eine genaue Phrase vollständig sein, aber nicht für Paraphrasen. Der Abruf kann ein hoher Rückruf sein, aber keine geschlossene Welt. Ein von Menschen kuratiertes Register kann für eine Freigabe vollständig sein. Abdeckung sollte ein erstklassiges Objekt sein, das durch negative Regeln konsumiert wird.

Die nllAgent Architektur enthält eine wichtige Version dieses Prinzips. LongTextJS zeichnet Abdeckung und Lücken neben Beobachtungen auf. Die Schaltungskompatibilität prüft, ob das tatsächliche Programm den erforderlichen Status und die Abdeckung erfüllt. Fehlende Kenntnisse werden nicht als Compliance gemeldet. Dies ist stärker als das übliche Muster, ein LLM zu bitten, true, false oder nicht gefunden zurückzugeben, da nicht gefunden wird, in Bezug auf eine dokumentierte Suchdomäne [AssistOS-AI 2026] interpretiert wird.

Die verbleibende Schwierigkeit besteht darin, modellproduzierte Beobachtungen zu validieren. Die Schema-Validierung ist notwendig, aber schwach. Evidenzverankerung verhindert erfundene Zitate, aber keine Fehlinterpretation. Kalibrierungsergebnisse helfen, Kandidaten zu bewerten, etablieren aber keine Bedeutung. Der stärkste praktische Ansatz kombiniert unabhängige Signale: kontrolliertsprachige Parser für traktible Fragmente; Modellensembles oder verschiedene Aufforderungen; Ontologiebeschränkungen; Kreuzsentenzkonsistenz; semantische Mutationen; negative Beispiele; menschliche Überprüfung für hochwirksame Unterscheidungen; und nachgeschaltete Gegenbeispiele, die Konsequenzen aufdecken.

Die Erdung sollte auch abfragegesteuert sein. Ein System muss nicht jede Entität oder jedes Ereignis in einem 300-seitigen Dokument auflösen. Es sollte eine semantische Nachfrage aus der auszuführenden Regel abgeleitet werden, die relevanten Zielfernrohre abrufen und nur die notwendigen Typen materialisieren. Die Abfrage-Erstverarbeitung darf jedoch die Relevanz nicht mit Vollständigkeit verwechseln. Wenn eine Regel fragt, ob eine Ausnahme irgendwo im Autoritätstext vorhanden ist, muss der Abruf ein Deckungsargument liefern oder die Antwort bleibt unbekannt. Nachfragegesteuerte Ausführung reduziert die Arbeit; sie rechtfertigt keine vorzeitige Schließung.

Die grundlegende Schlussfolgerung ist schwerwiegend, aber konstruktiv. Es gibt kein rein symbolisches Heilmittel für ein Symbol, das dem falschen Referenzanten beigefügt ist. Die Erdung ist kein vorläufiges NLP-Detail; es ist Teil der vertrauenswürdigen Berechnung. Das System muss die Kette von der Quelle zum Symbol aussetzen, Alternativen beibehalten, die Abdeckung verfolgen und die inkompatible Ausführung ablehnen. Sobald diese Kette diszipliniert ist, können ausgereifte formale Werkzeuge eine erhebliche Zuverlässigkeit hinzufügen. Ohne sie bescheinigen die Werkzeuge lediglich die Folgen einer plausiblen Fiktion.

5.2 Pragmatik, Nichtmonotonität, Skala, Sicherheit und Bewertungsfehler

Selbst eine fundierte Darstellung kann scheitern, weil die natürliche Sprache mehr tut als monotone Fakten. Sprecher implizieren, voraussetzen, revidieren, zitieren, scherzen, verhandeln und verlassen sich auf den institutionellen Kontext. Dokumente enthalten Ausnahmen, Prioritäten, Definitionen, die sich später ändern, und Aussagen, deren Kraft vom Genre abhängt. Ausführbare natürliche Sprache muss entscheiden, welche dieser Phänomene sie unterstützen will, und muss unbekannt außerhalb dieses Bereichs zurückkehren.

Pragmatik ist die erste Grenze. Ein Satz kann mehr als seine wörtlichen Wahrheitsbedingungen kommunizieren. "Der Bericht ist fast vollständig" kann bedeuten, dass er nicht vollständig ist. "Einige Tests haben bestanden" pragmatisch darauf hindeuten, dass es nicht alle getan haben, obwohl die klassische Logik dies nicht mit sich bringt. Ironie kehrt das scheinbare Gefühl um. Indirekte Anfragen verwenden Anweisungen als Aktionen. Ein Formalisierer, der pragmatische Schlussfolgerungen als Fakten behandelt, kann übertreffen; einer, der sie ignoriert, kann die praktische Absicht des Autors verfehlen. Die entsprechende Behandlung hängt von der Anwendung ab. Eine Compliance-Engine sollte in der Regel explizite Autorität privilegieren und pragmatische Interpretation als vorgeschlagen kennzeichnen. Ein Gesprächspartner benötigt möglicherweise reichhaltigere pragmatische Modelle, sollte sie aber nicht als harte Beweise darstellen.

Die Vorsorge schafft eine damit verbundene Herausforderung. "Der Ausschuss hat die Überprüfung des Vorschlags eingestellt", heißt es davon, dass er ihn überprüft hat. Definitive Beschreibungen setzen häufig Existenz oder Einzigartigkeit voraus. Dokumente können solche Konstruktionen ausnutzen, ohne den Hintergrund explizit geltend zu machen. Ein formales System muss entscheiden, ob Voraussetzungen als Annahmen, verächtliche Schlussfolgerungen oder Anforderungen zur Klärung eintreten. Die automatische Durchsetzung kann unsicher sein; sie zu ignorieren kann den Diskurs inkohärent machen. Das Statussystem sollte ihren Ursprung bewahren.

Natürliche Sprachregeln sind oft nicht monoton. Neue Informationen können eine Schlussfolgerung zurückziehen. "Mitarbeiter können auf das System zugreifen" kann durch "es sei denn, ihre Zertifizierung ist abgelaufen" besiegt. Standardlogik, Answer-Set-Programmierung, veräußerbare Logiken und priorisierte Regeln bieten formale Mechanismen für Ausnahmen. Die klassische Logik erster Ordnung kann Ausnahmen explizit kodieren, aber große Regelsätze werden schwierig zu verwalten und inkonsistente Informationen können die Inferenz trivialisieren, wenn gewöhnliche Explosion gilt.

Eine praktische Laufzeit sollte die Mehrfachregelsemantik unterstützen. Monoton Datalog ist angemessen, wenn sich nur Fakten ansammeln. Angezögerbare Regeln sind für Standardwerte und Ausnahmen geeignet. Parakonsistente Argumentation ist angemessen, wenn Quellen in Konflikt geraten und das System vermeiden muss, alles abzuleiten. Entscheidungstabellen sind angemessen, wenn Richtlinienkombinationen endlich und explizit sind. Die gewählte Semantik muss mit dem Regelpaket versioniert werden; sie kann von der LLM während der Ausführung nicht ad hoc abgeleitet werden.

Normative Systeme schaffen Autorität und Priorität. Eine spätere Regel könnte eine frühere überschreiben. Eine übergeordnete Verordnung kann einem lokalen Verfahren zuvorkommen. Gerichtsbarkeit und Datumersatz. Berechtigungen und Verpflichtungen können in Konflikt geraten. Deontische Paradoxien zeigen, dass naive logische Kodierungen kontraintuitive

Ergebnisse liefern können. Viele praktische Fälle werden besser durch explizite Regelprioritäten, Ausnahmen und Verletzungsbedingungen behandelt als durch ein universelles deontisches Kalkül. Ausführbare natürliche Sprache sollte die operative Klarheit über die theoretische Größe begünstigen.

Scale erzeugt mehrere interagierende Explosionen. Lange Dokumente enthalten viele Spannweiten und Kandidatenbeobachtungen. Mehrdeutige Erwähnungen schaffen kombinatorische Identitätszuweisungen. Zeitliche Beziehungen schaffen Teilreihenalternativen. Regeln schaffen abgeleitete Konsequenzen. Symbolische Ausführung schafft Pfade. Die Löser sind mit großen Zwangssystemen konfrontiert. LLM Kontextfenster und Kosten setzen separate Grenzen. Eine naive Architektur, die jeden Satz in jede mögliche Logik formalisiert, ist undurchführbar.

Die nachfragegesteuerte Materialisierung ist die erste Antwort. Schaltungen erklären, was sie konsumieren. Die Laufzeit indiziert Struktur und stabile lexikalische Informationen, dann erzeugt eine reichhaltigere Semantik nur für relevante Bereiche. Die inkrementelle Berechnung verwendet unveränderte Artefakte. Die abstrakte Interpretation fasst Alternativen zusammen. Staatliche Zusammenführung und begrenzte symbolische Explorationskontrollpfade. Solver-Portfolios leiten einfache Koffer zu billigen Motoren. Harte Budgets liefern explizit unvollständige Ergebnisse statt versteckte Abschneidung.

Die zweite Antwort ist die semantische Modularität. Ein Dokument kann durch Ansichten analysiert werden: Abschnitte, Zeitfenster, Jurisdiktionen, narrative Szenen, experimentelle Phasen oder Autoritätsebenen. Identitäts- und Regelfolgen können lokal sein. Cross-View-Beziehungen werden nur bei Bedarf eingeführt. Das reduziert die versehentliche globale Verschmelzung und unterstützt Parallelität. Das System sollte immer noch aufzeichnen, wenn eine Schlussfolgerung von der Annahme abhängt, dass eine Ansicht vollständig ist.

Die dritte Antwort ist die inszenierte Präzision. Ein kostengünstiger struktureller oder lexikalischer Durchgang eliminiert viele Fälle. Ein modellgestützter Extraktionspass produziert Kandidaten für die übrigen. Abstract-Analyse identifiziert mögliche Verstöße. Symbolische Verfeinerung behandelt nur ergebnisrelevante Unklarheiten. Die menschliche Überprüfung ist für hochwirksame ungelöste Entscheidungen reserviert. Dies ähnelt statisch-analysistischen Pipelines, die billige Analysen verwenden, bevor teures Theorem nachgewiesen wird.

Sicherheitsbedrohungen entstehen in jeder Phase. Prompte Injektion in Quelldokumenten kann den LLM anweisen, Schemas zu ignorieren oder Ergebnisse zu erstellen. Der Übersetzer sollte Quellinhalte als Daten unter einer rollenspezifischen Eingabeaufforderung erhalten, und seine Ausgabe sollte unabhängig validiert werden. Die Quelle darf niemals die Befugnis erhalten, die Laufzeit-, Regelpaket- oder Publikationsrichtlinie zu ändern. Jeder Text, der behauptet, eine Systemanweisung zu sein, ist immer noch Dokumentinhalt.

Generierter Code ist ein noch größeres Risiko. Ein DSL, das beliebige JavaScript-, Python-, Import- oder Netzwerkzugriffe ermöglicht, verwandelt semantisches Lernen in die Ausführung von Code. Deklarative Schaltkreise sollten sich auf einfache Daten normalisieren und nur registrierte Operatoren aufrufen. Benutzerdefinierte Algorithmen sollten als überprüfte Erweiterungen mit typisierten Verträgen, Effekten, Ressourcenlimits und gesperrten Digests installiert werden. Sandboxing bleibt notwendig, aber die Einschränkung der Fähigkeiten ist stärker als die Abhängigkeit nur auf eine Prozessgrenze.

Solver-Angriffe umfassen Denial-of-Service durch pathologische Einschränkungen, fehlerhafte Beweisobjekte und die Nutzung nativer Bibliotheken. Budgets, Theoriebeschränkungen, Proof-Checking und Versions-Pinning sind erforderlich. Ein Modell kann absichtlich oder versehentlich eine Formel konstruieren, die technisch gültig, aber rechenexplosiv ist. Die Auswahl des Formalismus sollte die Kosten schätzen und Aufgaben außerhalb der Politik ablehnen.

Datenvergiftung betrifft gelernte semantische Produzenten und Benchmark-Suiten. Erzeugt ein Codierungsmittel sowohl eine Schaltung als auch die Beispiele, die sie validieren, können die Artefakte das gleiche Missverständnis aufweisen. Unabhängige Testgenerierung, semantische Mutationen, manuell kuratierte Ankerfälle und separate Überprüfungsrollen reduzieren dieses Risiko. Die nllAgent Lernarchitektur verhindert korrekt, dass ein Codierungsagent einen eigenen Kandidaten veröffentlicht. Die manuelle Veröffentlichung ist eine Governance-Kontrolle, obwohl ihre Wirksamkeit von der Qualität der Benchmarks und unabhängigen Prüfungen abhängt.

Natürliche Sprachrealisierung kann nicht unterstützte Ansprüche wieder einführen. Ein Modell kann „unter einer Interpretation möglich“ in „das Dokument verstößt gegen die Regel“ umwandeln. Es kann Annahmen auslassen oder eine unvollständige Suche in Abwesenheit verwandeln. Die Realisierung sollte daher durch ein kanonisches Ergebnisschema eingeschränkt und auf Anspruchsebene überprüft werden. Hochwirksame Ausgänge können kontrollierte Vorlagen verwenden. Erläuternde Kommentare sollten eindeutig von den verifizierten Befunden getrennt werden.

Die Auswertung ist vielleicht der systemischste Ausfall. Viele Benchmarks messen nur die endgültige Antwortgenauigkeit. Dies verwirrt die Extraktion, Formalisierung, Argumentation und Darstellung. Ein Modell kann aus dem falschen Grund die richtige Antwort finden. Ein formales System kann bestraft werden, um unbekannt zurückzukehren, wenn der Benchmark eine unausgesprochene Konvention ausnimmt. Synthetische Datensätze können die Erkennung von Vorlagen belohnen. LLM-as-a-judge-Metriken können Verzerrungen mit den zu bewertenden Systemen teilen.

Ein strenger Maßstab sollte mehrere Schichten enthalten. Die Quellebene enthält exakten Text und Struktur. Die semantische Schicht enthält eine oder mehrere zulässige Interpretationen mit Beweisspannweiten. Die formale Ebene umfasst Zielprogramme oder Erweiterungsäquivalenztests. Die Ausführungsschicht umfasst erwartete Modelle, Beweise, Zeugen oder unzufriedene Kerne. Die Ergebnisschicht enthält kanonische Status und Abdeckung. Die Realisierungsschicht umfasst erlaubte Ansprüche und verbotene Übertreibungen. Fehler können dann eher zugeschrieben als in einer Punktzahl zusammengebrochen werden.

Minimale Paare sind unerlässlich. Das Ändern eines Wortes oder einer Beziehung sollte einen vorhersehbaren formalen Unterschied hervorrufen. Paraphrasenfamilien testen Invarianz. Distraktorschnitte Testabruf. Identitätskollisionen testen Erdung. Fehlende Beweise testen unbekanntes Verhalten. Widersprüchliche Quellen testen Parakonsistenz oder Autoritätspolitik. Testkalender und Intervalle für temporäre Grenzfälle. Gegnerischer prompter Text testet Sicherheitsisolierung. Semantische Mutationen testen, ob eine Schaltung tatsächlich ihre Regel implementiert.

Die Bewertung sollte auch die Enthaltung messen. Ein vertrauenswürdiges System muss wissen, wann es an Ontologie, Produzent, Abdeckung oder Budget fehlt. Unbekannt ist kein

Misserfolg, wenn die Frage nicht unterstützt wird. Metriken sollten berechnete Enthaltung von vermeidbarer Unvollständigkeit unterscheiden. Kurven in Deckungsqualität können zeigen, wie sich Genauigkeit und falsche Sicherheit ändern, da das System eine modellträglichere Semantik zulässt.

Unabhängige Umsetzungsfragen. Ein Schaltungs-Compiler und sein "Schatten" -Bewerter sollten nicht jede kritische Funktion teilen, wenn differentielle Tests behauptet werden. Mutationswerte sollten aus ausführbaren Mutationen stammen, nicht aus Prosabeschreibungen. Live-Modellprofile sollten getrennt von deterministischen Vorrichtungen bewertet werden. Planung und Realisierung erfordern aufgabenspezifische Benchmarks, anstatt aus der Auditleistung abgeleitet zu werden. Dies sind genau einige der schwerwiegenden Probleme, die im nllAgent-Repository [AssistOS-AI 2026] anerkannt sind.

Externe Überprüfungen sollten mit ähnlicher Sorgfalt interpretiert werden. Die Umfrage von Frontiers 2026 ist begutachtet und nützlich, aber ihre Behauptung, dass föderative Systeme besser funktionieren, wird explizit durch die Verwirrung der Aufgabe qualifiziert. Planungs- und Programmanalyse-Untersuchungen zeichnen Trends ab, können aber nicht feststellen, dass eine vorgeschlagene Architektur an uneingeschränkten Dokumenten arbeitet. Es wurde keine unabhängige Überprüfung von nllAgent gefunden. Ein verantwortungsbewusstes Buch sollte daher Rezensionen verwenden, um Konsens und Meinungsverschiedenheiten zu identifizieren, und dann seine eigenen Urteile angeben, in denen Beweise fehlen.

Das zentrale Bewertungsprinzip ist die kausale Zuordnung. Wenn sich ein System verbessert, sollten die Forscher feststellen, ob der Gewinn aus einem besseren semantischen Parsing, einer besseren Korrektheit des Löfers, des Abrufs, der Probenahme der Testzeit oder der Benchmark-Leckage resultierte. Wenn es fehlschlägt, sollten sie die Ebene identifizieren. Das ist nicht nur wissenschaftliche Hygiene. Layered-Fehlerdaten ermöglichen es der Architektur, dies zu verbessern, ohne das gesamte System zu ersetzen oder Regressionen in aggregierten Scores zu verbergen.

Ausführbare natürliche Sprache wird eine begrenzte Fähigkeit bleiben. Pragmatik, Open-World-Wissen, sozialer Kontext und sich entwickelnde Institutionen sorgen dafür, dass kein endlicher symbolischer Kernel alle Bedeutung erfasst. Das Ziel ist nicht, diese Grenze zu entfernen, sondern sie explizit und nützlich zu machen. Ein System kann starke Garantien für strukturelle, relationale, zeitliche, quantitative und regelgesteuerte Fragmente bieten, während man an anderer Stelle bedingte Ergebnisse zurückgibt. Das gefährlichste System ist nicht eines, das Grenzen einräumt. Es ist eine, die unmodellerte Sprache in eine selbstbewusste symbolische Ausgabe umwandelt und das Ergebnis als verifiziert bezeichnet.

6. Architekturfamilien für ausführbare natürliche Sprache

6.1 Konkurrierende Designs: universelle Repräsentationen, Formalismus-Portfolios, kontrollierte Kompilierung und query-first-Systeme

Ausführbare Spracharchitekturen können danach organisiert werden, wo sie semantisches Engagement hinlegen. Einige versuchen, jeden Input in eine universelle Darstellung zu übersetzen. Andere behalten mehrere Kandidatenformulare und wählen einen Formalismus pro Aufgabe aus. Einige beschränken die Eingabesprache, bis die Zusammenstellung deterministisch ist. Andere bewahren die offene Sprache, formalisieren aber nur die Beobachtungen, die von einer Abfrage verlangt werden. Jedes Design löst ein anderes Problem, und ein ausgereiftes System wird sie wahrscheinlich kombinieren, anstatt ausschließlich eins auszuwählen.

Die universelle Zwischendarstellung ist die intuitivste Architektur. Text wird in einem reichen semantischen Graphen oder einer logischen Sprache kompiliert, und jeder nachgeschaltete Vorgang weist dieses gemeinsame Objekt ab oder transformiert es. Die Vorteile sind konzeptionelle Einheit, Wiederverwendung und die Möglichkeit globaler Konsistenzprüfungen. Eine einzelne Darstellung kann mehrere Anwendungen unterstützen und eine stabile Schnittstelle zwischen NLP und Reasoning erstellen.

Seine Schwäche ist semantische Überreichweite. Um jeder Aufgabe zu dienen, muss die Repräsentation Entitäten, Ereignisse, Diskurse, Zeit, Modalität, Quantität, Kausalität, Normen, Unsicherheit, Provenienz und domänenspezifische Konzepte abdecken. Entweder wird die Sprache extrem komplex oder sie vereinfacht Unterschiede, die eine Aufgabe benötigt. Eine universelle Darstellung schafft auch einen einzigen Punkt des Scheiterns: Ein falscher globaler Parse kontaminiert jede Analyse. Der Druck, frühzeitig eine Identität, einen Umfang oder eine Ontologie zu wählen, kann Mehrdeutigkeit verbergen.

Ein semantischer Drehpunkt ist eine bescheidenere Variante. Es behauptet nicht, eine vollständige logische Form zu sein. Es standardisiert Quellenspannen, Erwähnungen, Entitätshypothesen, Beobachtungen, Welten, zeittragende Werte, Modalität, Provenienz, Alternativen, Abdeckung und Lücken. Spezialisierte Compiler unterlegen ausgewählte Teile in formale Ziele. Dies bewahrt die übliche Infrastruktur, ohne so zu tun, als würde eine Ausführungssemantik für jede Bedeutung passen. LongTextJS, wie in nllAgent dokumentiert, ist dieser Idee nahe: Es ist eine adressierbare Quellwelt und ein Beobachtungsprogramm und nicht eine universelle theorem-nachweisende Sprache [AssistOS-AI 2026].

Die Multi-Candidate-Architektur behandelt die Formalisierung als Suche. Ein LLM erzeugt mehrere logische Formen, Programme oder semantische Graphen. Typprüfungen und Ontologiebeschränkungen beseitigen ungültige Kandidaten. Ausführung, Beispiele und Gegenbeispiele ordnen oder unterscheiden die Überlebenden. Das System kann ein Muss zurückgeben, wenn alle Kandidaten zustimmen und eine Schlussfolgerung, wenn nur einige dies tun. Diese Architektur ist angemessen, wenn Mehrdeutigkeit echt ist oder wenn die Formalisierungszuverlässigkeit begrenzt ist.

Die Kosten sind kombinatorisch. Kandidatenformulare multiplizieren über Sätze und Identitätsentscheidungen. Ein naives Kreuzprodukt wird undurchführbar. Die Lösung besteht darin, die gemeinsame Struktur und lokale Alternativen kompakt darzustellen, abstrakte Domänen zu verwenden, um sie zusammenzufassen und nur ergebnisrelevante Entscheidungen zu verfeinern. Vertrauen kann die Erforschung leiten, aber Kandidaten mit geringer Wahrscheinlichkeit sollten nicht aus einer fundierten Analyse entfernt werden, es sei denn, die Richtlinie akzeptiert ausdrücklich probabilistische Unvollständigkeit.

Die kontrollierte Sprachzusammenstellung verfolgt den gegenteiligen Ansatz. Der Input wird in eine eingeschränkte natürliche Sprache umgeschrieben, deren Grammatik und Semantik deterministisch oder stark eingeschränkt sind. Benutzer können direkt in der kontrollierten Sprache verfassen, oder ein LLM kann eine kontrollierte Paraphrase zur Bestätigung vorschlagen. Nach der Annahme ist die Zusammenstellung zuverlässig. Diese Architektur ist besonders stark für Anforderungen, Verträge, Verfahren und Regelerstellung.

Seine Beschränkung ist Abdeckung und Adoption. Gewöhnliche Dokumente enthalten Konstruktionen außerhalb des kontrollierten Fragments. Umschreiben kann Bedeutung verändern. Benutzer können eine Paraphrase genehmigen, ohne einen subtilen Unterschied zu bemerken. Die Architektur funktioniert am besten, wenn die kontrollierte Form neben der Quelle angezeigt wird, Unterschiede lokalisiert werden und formale Gegenbeispiele wichtige Entscheidungen konkret treffen. Die kontrollierte Sprache sollte ein expliziter semantischer Checkpoint sein, keine unsichtbare Normalisierung.

Query-first-Architekturen vermeiden eine vollständige Formalisierung. Eine Schaltung oder Frage erklärt, welche Beobachtungen und Beziehungen sie braucht. Die Laufzeit bewahrt die Quelle strukturell, ruft relevante Regionen ab und materialisiert nur geforderte semantische Typen. Relationale Abfragen, Tabellen und Entscheidungsregeln funktionieren über die resultierende endliche Welt. Dieses Design skaliert besser auf lange Dokumente und richtet die Berechnung mit dem Zweck aus.

Eine eng verwandte Vision von Datenbanksystemen 2026 unterscheidet interpretierte semantische Bediener von zusammengestellten. Im interpretierten Design wird innerhalb der Datenverarbeitungsschleife für jedes Zeilen-, Datensatz- oder Kandidatenpaar ein LLM aufgerufen. Im kompilierten Entwurf wird der LLM während einer separaten Bauphase aufgerufen, um den semantischen Betrieb in einen deterministischen lokalen Code zu übersetzen, der ohne Modellaufruf wiederholt ausgeführt werden kann. Vorläufige Experimente berichteten über eine erhebliche Verringerung der Latenz- und Modellnutzung, wobei ein Großteil der Ausgabequalität [Dong and Wang 2026] beibehalten wurde. Der Vorschlag ist wichtig, weil er unabhängig voneinander die gleiche architektonische Intuition erreicht, die in diesem Buch entwickelt wurde: Das Sprachmodell ist am nützlichsten als Compiler oder Plankonstruktor, während die wiederholte Laufzeit stabil und inspizierbar sein sollte. Seine Begrenzung ist ebenso lehrreich. Generierter Code kann den modelldefinierten Operator effizient annähern, ohne einen semantischen Beweis dafür zu liefern, dass er das beabsichtigte Konzept des Benutzers erfasst. Die Zusammenstellung beseitigt die pro-Punkte-Stochastik; sie löst nicht von selbst Erdung oder Spezifikationstreue.

Das Risiko ist eine falsche Schließung. Die Abrufen kann relevante Beweise übersehen, und die Bedarfsanalyse kann ein erforderliches Konzept unterlassen. Query-first-Systeme müssen die Abdeckung und die Produzenten verfolgen. Wenn eine negative Schlussfolgerung erfordert, dass das gesamte Dokument nach einer Ausnahme durchsucht wird, sind einige abgerufene

Passagen unzureichend. Die Architektur sollte ein grenzendes Abfrageergebnis von einer Closed-World-Garantie unterscheiden. Die Kompatibilitäts- und Deckungsverträge von nllAgent sind ein wichtiger Mechanismus in dieser Familie.

Eine Single-Formalism-Architektur senkt alle Aufgaben in eine Logik, oft Logik erster Ordnung, Datalog oder SMT. Es vereinfacht die Umsetzung und Beweispflichten. Für eine enge Domain kann dies die beste Wahl sein. Logic-LM und LINC gewinnen einen Großteil ihres Wertes aus einer stabilen Zielsprache. Ein System zur Planung kann vernünftigerweise eine Einschränkungssprache verwenden. Ein Protokollprüfer kann ein Prozesskalkül verwenden.

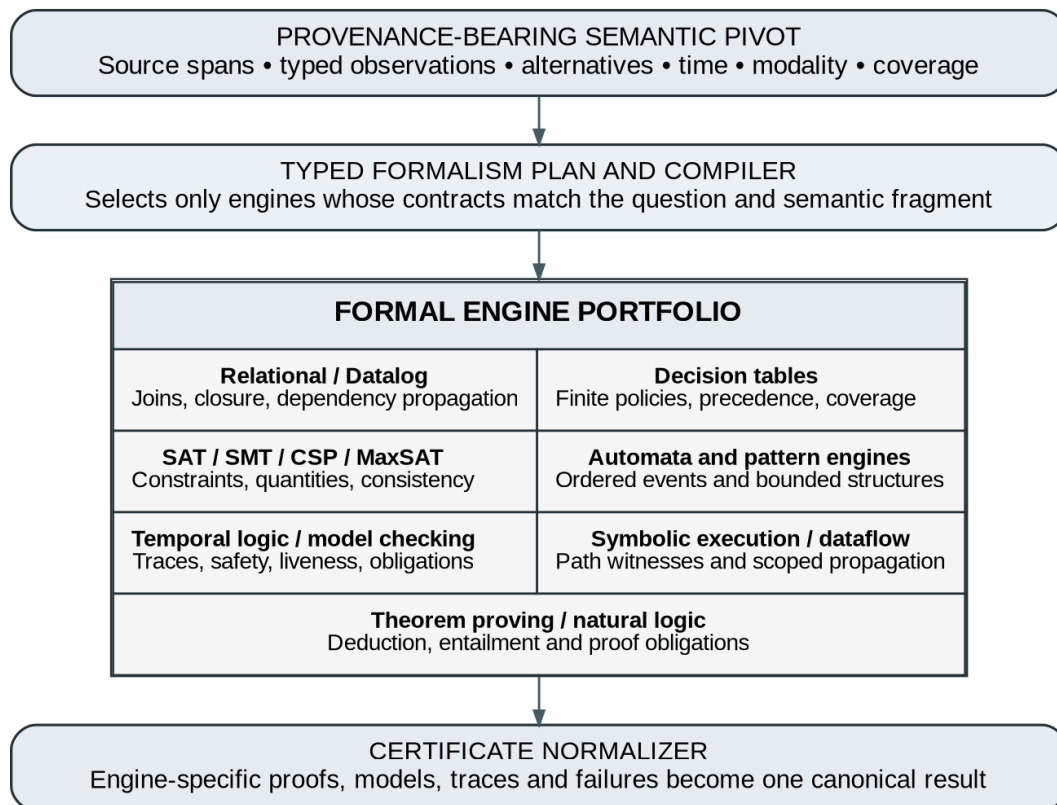


Abbildung 3. Ein provenienzhaltiger semantischer Drehpunkt kann zu einem Portfolio spezialisierter Formmotoren zusammengestellt und in ein kanonisches Ergebnis normalisiert werden.

Für breite natursprachliche Aufgaben ist ein Solver-Portfolio plausibler. Die relationale Auswertung behandelt Dokumentabfragen. Datalog berechnet Monotonverschluss. Entscheidungstabellen drücken endliche Richtlinien aus. SAT und SMT behandeln Einschränkungen und Rechenbedingungen. Die Modellprüfung behandelt das zeitliche Verhalten. Symbolische Hinrichtung konstruiert Pfadzeugen. Natürliche Logik behandelt bestimmte mit sich bringende Beziehungen in der Nähe der Oberflächensprache. Automata erkennen geordnete Muster. Der semantische Drehpunkt liefert geordnete Atome, und ein Formalismusplan wählt einen oder mehrere Motoren aus.

Das Portfolio schafft neue Verpflichtungen. Der Router muss die ausdrucksstarken und betrieblichen Fähigkeiten jeder Engine kennen. Ein Compiler muss die Bedeutung über die Senkungen hinweg beibehalten. Die Ergebnisse verschiedener Motoren benötigen ein gemeinsames Status- und Provenienzmodell. Wenn zwei Motoren überlappende Prädikate begründen, muss die Konsistenz überprüft oder explizit offen gelassen werden.

Ressourcenrichtlinien müssen verhindern, dass ein Modell einen unnötig teuren Solver auswählt.

Ein nützlicher Formalismusvertrag besagt, dass die Eingabetypen, unterstützten Betreiber, die Semantik, die erforderliche Abdeckung, die Garantiekategorie, das erwartete Zertifikat, das Kostenmodell und die bekannten Einschränkungen. Ein zeitlich-logischer Motor kann geordnete Ereignisse und ein endliches Übergangssystem erfordern. Eine SMT-Engine kann ganzzahlige arithmetische, aber nicht uneingeschränkte nichtlineare reale Funktionen unterstützen. Eine Entscheidungstabelle kann endliche aufgezählte Bedingungen erfordern. Ein natürlich-logischer Motor kann Monotonität, aber nicht willkürliche Quersatz-Korreferenz unterstützen. Der Router kann dann einen vorgeschlagenen Plan ablehnen, dessen Anforderungen die Engine übersteigen.

Die Proof-Carrying-Generation ist eine weitere Architekturfamilie. Das primäre Artefakt ist nicht die natürliche Sprachantwort, sondern ein kanonisches Ergebnis plus ein Beweis, ein Zeugnis, eine Hinrichtungsspur oder eine überprüfbare Ableitung. Das Sprachmodell erzeugt eine Erklärung aus diesem Artefakt. Jeder erläuternde Anspruch ist mit einem Ergebniselement verbunden. Ein unabhängiger Checker stellt sicher, dass die Erklärung den formalen Inhalt nicht übersteigt.

Dieses Design ist wertvoll, da die Erzeugung und Verifizierung unterschiedliche Stärken haben. Ein LLM kann eine klare Erklärung eines Gegenbeispiels oder einer unzufriedenen Einschränkung schreiben. Ein symbolisches Ergebnis kann sicherstellen, dass das Szenario unter dem akzeptierten Modell real ist. Die Erklärung kann immer noch irreführend sein, wenn sie Annahmen oder Unsicherheiten verbirgt, so dass das kanonische Ergebnis diese Dimensionen enthalten muss und der Renderer-Vertrag sie beim Material erfordern muss.

Abstrakt-semantische Architekturen platzieren eine abstrakte Interpretationsebene vor der speziellen Ausführung. Bewerberbeobachtungen können und müssen Domänen. Billige Propagation bestimmt, welche Eigenschaften definitiv zufrieden, definitiv unmöglich oder potenziell verletzt sind. Nur mögliche Verstöße werden in teure symbolische Probleme gesenkt. Dies reduziert die Löselast und liefert prinzipielle unbekannte Zustände. Es schafft auch eine natürliche inkrementelle Architektur für Dokumentbearbeitungen.

Gegenbeispielgesteuerte Architekturen schließen den Kreislauf. Ein gescheiterter Beweis oder falscher Zeuge ist nicht nur ein Fehler; er identifiziert, welche semantische Unterscheidung zählt. Das System ordnet die Fakten des Solvers auf Quellenspannweiten ab, fordert den LLM oder Benutzer auf, die relevante Interpretation zu verfeinern, und spielt die Analyse wieder ab. Diese Architektur ist besonders vielversprechend, weil sie formales Feedback in ein gezieltes Sprachverständnis und nicht in die allgemeine Selbstreflexion verwandelt.

Lernarchitekturen regeln, wie neue Regeln, Ontologien und Schaltkreise in die Produktion gelangen. Ein Extrem ermöglicht es einem LLM-Agenten, bei jedem Durchlauf ausführbaren Code zu generieren. Das maximiert die Flexibilität und minimiert die Sicherheit. Das entgegengesetzte Extrem erlaubt nur handschriftliche, formal verifizierte Regeln. Eine praktische Mitte verwendet Codierungsmittel in der Inszenierung von Arbeitsbereichen, generiert Benchmarks und Mutationen, beschränkt Artefakte auf deklarative Formen und erfordert eine explizite Veröffentlichung. Die Produktionslaufzeit liest unveränderliche Releases. Dies ist das Governance-Modell, das in nllAgent implementiert ist und allgemeiner werden sollte.

Der folgende Vergleich fasst den Kernproxe zusammen, ohne zu behaupten, dass ein Design jede Aufgabe dominiert.

Tabelle 1. Hauptarchitekturfamilien und ihre wesentlichen Kompromisse.

Kategorie: Architektur	Wesentliche Stärke und Hauptrisiko
Universelle semantische Darstellung	Wiederverwendung und globale Konsistenz; Risiken überkomplexität und vorzeitiges semantisches Engagement.
Semantischer Drehpunkt plus spezialisierte Absenkungen	Bewahrt gängige Beweise und unterstützt mehrere Motoren; erfordert sorgfältige Compiler- und Cross-Engine-Verträge.
Mehrfache Kandidatenformalisierungen	Stellt Mehrdeutigkeit dar und ermöglicht möglicherweise Schlussfolgerungen; Risiken kombinatorisches Wachstum.
Controlled-language-Compilation	Starke Semantik und benutzerinspektierbare Formen; opfert die Abdeckung und kann beim Umschreiben Bedeutungsänderungen einführen.
Query-first Materialisierung	Skalierungen auf lange Dokumente und aufgabenspezifische Anforderungen; kann das Abrufen mit Vollständigkeit verwechseln, es sei denn, die Abdeckung ist explizit.
Einzelnes formales Ziel	Einfache Vertrauensbasis und Bewertung in einem begrenzten Bereich; schlechte Passform für heterogene Sprachphänomene.
Solver-Portfolio	Kombiniert die Berechnungsmethode an die Problemstruktur; fügt Routing-, Interoperabilitäts- und Zertifikat-Normalisierungsverpflichtungen hinzu.
Proof-Carrying-Generierung	schränkt natürlichsprachliche Antworten auf verifizierte Inhalte ein; erfordert eine robuste Überprüfung der Realisierung auf Anspruchsebene.
Abstrakte Analyse plus symbolische Verfeinerung	Gleicht Skalierung und Präzision aus; die Solidität bleibt von der Interpretationsabdeckung abhängig.
Geregierte gelernte Schaltkreise	Ermöglicht die Evolution ohne Laufzeit-Selbstmodifikation; hängt von starken Benchmarks, Mutationstests und Veröffentlichungsüberprüfung ab.

Die glaubwürdigste allgemeine Architektur ist ein Hybrid aus den mittleren Optionen. Es verwendet einen provenienzhaltigen semantischen Drehpunkt, bewahrt Alternativen, wo sie wichtig sind, führt abfragegesteuerte Materialisierung durch, führt Wege zu einem Formalismus-Portfolio und akzeptiert nur Proof- oder Witness-gestützte Ergebnisse. Die kontrollierte Sprache kann als interaktiver Kontrollpunkt für hochwirksame Formalisierungen dienen. Die abstrakte Interpretation steuert die Skalierung, während die gegenbeispielgesteuerte Verfeinerung die Präzision verbessert. Das Lernen bleibt außerhalb der Produktion.

Diese Schlussfolgerung ist nicht nur theoretisch. Sie spiegelt sich in der empirischen Literatur wider. PAL, SatLM, Logic-LM, LINC, temporal-logic-Übersetzer, Planungsformalisierer, SymGPT und zertifizierte Verifizierungssysteme sind alle erfolgreich, indem sie das formale Ziel eingrenzen. Kein Ergebnis zeigt, dass eine universelle Logik oder ein End-to-End-Modell das Portfolio ersetzen kann. Die Architektur sollte daher Heterogenität umfassen und gleichzeitig die vertrauenswürdigen Schnittstellen zwischen Komponenten minimieren.

6.2 Eine Referenzpipeline von Text zu Symbolen, Ausführung und eingeschränkten natürlichen Sprachantworten

Eine Referenz-Ausführbare Sprachpipeline sollte als Compiler und Verifikations-Toolchain und nicht als Agentenaufforderung verstanden werden. Jede Stufe materialisiert ein unveränderliches oder inhaltsbezogenes Artefakt. Ein Lauf kann wiederholt werden, ohne das Modell zu bitten, seine vorherige Argumentation zu rekonstruieren. Fehler werden Stufen zugeordnet. Lernen kann einen Produzenten oder eine Theorie verbessern, ohne die Bedeutung alter Ergebnisse zu ändern.

Das erste Artefakt ist das Quellpaket. Es enthält den kanonischen Text oder die Daten, Revisionsverdauung, Sprache, Struktur, Kanäle und stabile Spannweiten. Absätze, Überschriften, Tabellen, Zitate und Codeblöcke behalten Ordnung und Identität bei. Externe Datensätze haben ähnlich stabile Schlüssel. Das Quellpaket ist lexikalische Autorität: Es stellt fest, was vorhanden war, nicht, was es bedeutet.

Das zweite Artefakt ist der Aufgabenvertrag. Es gibt die Frage, Regelfreigabe, Quellumfang, gewünschte Garantie, erlaubte Annahmen, Welt und Gerichtsbarkeit, Budgets und Output-Politik. Die Aufgabe bestimmt die semantische Nachfrage. Eine Anfrage zur Suche nach undefinierten Akronymen erfordert unterschiedliche Beobachtungen von einer Anfrage, um die zeitliche Einhaltung zu überprüfen. Der Vertrag legt auch fest, ob Abwesenheit abgeschlossen werden kann und welche Formen der Unsicherheit eine menschliche Überprüfung erfordern.

Die dritte Stufe konstruiert einen Evidenzgraphen. Deterministische Produzenten fügen strukturelle Beobachtungen hinzu. Kontrollierte Parser fügen Beobachtungen für die Fragmente hinzu, die sie erkennen. LLM-gestützte Hersteller fügen schemagebundene Kandidatenereignisse, Beziehungen, Identitäten, Modalitäten oder Mengen hinzu. Jede Beobachtung zeichnet Anker, Hersteller-, Modell- oder Algorithmusversion, Status, Alternativen und Abdeckung auf. Das Modell schreibt nicht direkt in den kanonischen Graphen; ein Materializer überprüft Spannen, Typen und referenzielle Integrität.

Identität und Weltmanagement finden in derselben Schicht statt, werden aber nicht stillschweigend aufgelöst. Erwähnungen sind lokale Beweise. Entity Hypothesen sammeln Erwähnungen unter einer Welt und einem Umfang. Identitätskandidaten geben gleiche, andere oder ungelöste Beziehungen mit Unterstützung an. Gegenseitig exklusive Interpretationen gehören zu getrennten Welten oder bewachten Alternativen. Die Grafik kann daher Meinungsverschiedenheiten darstellen, ohne die globale Konsistenz zu verfälschen.

Die vierte Stufe berechnet einen abstrakten semantischen Zustand. Domänenspezifische Abstraktionen fassen mögliche Identitäten, zeitliche Ordnungen, normative Zustände, Größen und Evidenzstufen zusammen. Müssen Fakten in allen zugelassenen Welten enthalten sein; können Fakten in mindestens einer enthalten sein. Unbekannt zeichnet eher eine Lücke als eine probabilistische Vermutung auf. Der abstrakte Zustand unterstützt die billige Regelausbreitung und identifiziert, welche Fragen mehr Präzision erfordern.

Die fünfte Stufe erstellt einen Formalismusplan. Der Planer ist keine Freiform LLM Entscheidung. Sie vergleicht die Aufgabe mit registrierten Formalismusverträgen. Eine einfache Dokumentabfrage kann relational bleiben. Eine Richtlinie kann in Entscheidungstabellen und -beschränkungen zusammengestellt werden. Ein Verfahren kann in ein Übergangssystem kompilieren. Eine pfadabhängige Verletzung kann sich auf eine symbolische Ausführung berufen. Ein deduktiver Anspruch kann sich auf einen Theorem-Prüfer berufen. Die Plannamen

erforderten Eingaben, Absenkungen, Motorversionen, Ressourcengrenzen und erwartete Zertifikate.

Der LLM kann einen Plan vorschlagen, da er Problemstruktur und Domänensprache erkennt. Ein deterministischer Checker bestätigt, dass der Plan unterstützte Formalismen verwendet, dass jede erforderliche Beobachtung einen kompatiblen Produzenten hat und dass keine semantische Unterscheidung, die für die Aufgabe wesentlich ist, verloren geht. Beispielsweise kann eine Regel, die Ausnahmen und Prioritäten beinhaltet, ohne explizite Kodierung nicht in eine Monoton-Engine kompiliert werden. Eine Regel, die Identität beinhaltet, kann nicht vorgehen, wenn die Identitätsdomäne ungelöst ist und das Urteil davon abhängt.

Die sechste Stufe führt eine typisierte Zusammenstellung auf. Dieser Compiler sollte klein, deterministisch und stark getestet sein. Es konvertiert semantische Objekte in Solver-Variablen, Beziehungen, Übergänge oder Prädikate. Quellkarten verbinden jedes formelle Element mit Beobachtungen und Spannen. Compiler sollten Beweisverpflichtungen generieren, wenn eine Senkung verlustreich ist oder von Annahmen abhängt. Verschiedene Compiler können durch semantische Mutationen und Unterscheidungsmodelle verglichen werden.

Die siebte Stufe ist die formelle Hinrichtung. Relationale und Datalog-Engines berechnen Verschlüsse. Entscheidungstabellen identifizieren übereinstimmende und widersprüchliche Zeilen. SAT/SMT/CSP-Solver finden Modelle oder beweisen Undurchführbarkeit unter ihren unterstützten Theorien. Modellprüfer bewerten zeitliche Eigenschaften. Symbolische Vollstrecker erforschen die Wegbedingungen und geben konkrete Zeugen zurück. Beweisassistenten oder Theoremprüfer produzieren Ableitungen. Die Motorausgabe ist noch kein benutzerorientiertes Urteil; es handelt sich um einen zertifikathaltigen Ausführungsdatensatz.

Die achte Stufe ist die Wiederholungsprüfung. Ein separater Prüfer überprüft den Zeugen oder Beweis gegen das quellengebundene formale Problem und die veröffentlichte Theorie. Für eine Dokumentbefundung bestätigt sie, dass die zitierten Beobachtungen existieren, Spannen mit der Quellrevision übereinstimmen, Status die Regel erfüllen und die Ausführungsspur den Anspruch unterstützt. Für ein negatives Ergebnis bestätigt es, dass Abdeckungs- und Vollständigkeitsannahmen ausreichend sind. Replay ist wertvoll, auch wenn der zugrunde liegende Solver vertrauenswürdig ist, da er die Verbindung zwischen Kandidatenausgabe und freigegebener Richtlinie validiert.

Wenn die abstrakte Analyse eine Verletzung vorhersagte, aber die symbolische Ausführung sie nicht realisieren kann, beginnt die semantische Verfeinerung. Das System extrahiert den unzufriedenen Kern, widersprüchliche Prädikate oder Beweisfehler. Quellkarten identifizieren die relevanten Spannen und semantischen Entscheidungen. Der LLM erhält eine begrenzte Aufgabe: Überdenken Sie diese Identitätsverbindung, den zeitlichen Umfang, das Ausnahmezuordnung oder die Prädikatausrichtung. Es schlägt überarbeitete Kandidaten vor; der Materialisierer und der Compiler bestätigen sie; Ausführung wiederholt sich. Die menschliche Überprüfung wird beantragt, wenn die Unterscheidung eine hohe Wirkung hat oder ungelöst bleibt.

Diese Schleife sollte monoton in Bezug auf Beweise sein, nicht nur Vertrauen. Eine Verfeinerung kann eine Entität teilen, ein Zeitintervall eingrenzen, eine übersehene Ausnahme hinzufügen oder eine nicht unterstützte Prämisse markieren. Es sollte keine vorgeschlagene Beobachtung auf die Tatsache hochstufen, da das neue Ergebnis bequem ist. Jede Änderung wird als neue semantische Revision aufgezeichnet. Frühere Wege bleiben reproduzierbar.

Das neunte Artefakt ist ein kanonisches Ergebnis. Es enthält die Regel oder Abfrage Identität, akzeptierten Status, Annahmen, muss und kann Schlussfolgerungen, abgelehnte Kandidaten, Unbekannte, Deckung, Quellbeweise, Beweis- oder Zeugenverweise und Ausführungsgrenzen. Es kann auch einen prägnanten maschinenprüfbaren Erklärungsgraphen enthalten. Dieses Artefakt ist maßgeblich für jede Prosa-Rendering.

Die zehnte Stufe verwirklicht die natürliche Sprache. Der Renderer erhält nur das kanonische Ergebnis und ausgewählte Quellausschnitte. Es wird die ursprüngliche Argumentation nicht wiederholt. Jeder generierte Satz ist mit einem oder mehreren Ergebnisansprüchen verknüpft. Ein Checker vergleicht den Satz mit den erlaubten Ansprüchen, stellt sicher, dass Modalität und Unsicherheit erhalten bleiben, und lehnt nicht unterstützte Ergänzungen ab. Die Ausgabe kann Ergebnisse, Annahmen, Interpretationen und Empfehlungen unterscheiden.

Zum Beispiel kann die richtige Antwort lauten: „Unter der Interpretation, dass beide Verweise auf die Kammer dasselbe Gremium bezeichnen, wird die Regel verletzt, weil die Genehmigung der aufgezeichneten Sicherheitsüberprüfung vorausgeht. Das Dokument legt nicht fest, ob die Referenzen dieselbe Stelle bezeichnen; daher ist das Gesamtergebnis überprüft, das erforderlich ist.“ Ein fließender, aber unsicherer Renderer könnte den Zustand auslassen und einen Verstoß ankündigen. Das kanonische Ergebnis verhindert diese Vereinfachung.

Die Pipeline unterstützt auch die Erzeugung, nicht nur das Verrutschen. Ein Nutzer liefert eine Idee oder ein Ziel. Planungsschaltungen konstruieren eine kontrollierte Spezifikation mit Regel-zu-Plan-Zeugen. Ein LLM realisiert einen Entwurf. Audit-Schaltungen analysieren den Entwurf mit dem gleichen Beweis- und Verifizierungspfad. Die Überarbeitung wird fortgesetzt, bis die kanonische Prüfung die Freigabepolitik erfüllt oder das Budget ausgeschöpft ist. Das Generationsmodell wird nie zum letzten Orakel, ob sein eigener Text die Regeln erfüllt.

Governance umgibt die Toolchain. Ontologien, Produzenten, Schaltungen, Compiler, Formalismusverträge und Renderer haben Versionen und Verdauungen. Lernmittel arbeiten in Einweg-Staging-Bereichen. Die Veröffentlichung erfordert Benchmarks, semantische Mutationen, Kompatibilitätsprüfungen und eine ausdrückliche Genehmigung. Die Produktion liest unveränderliche Releases. Live-Modellprofile werden separat bewertet, da deterministische Vorrichtungen keine Anbieterqualität herstellen. Aktuelle Wissenspakete haben effektive Daten und Gerichtsbarkeiten.

Die Architektur sollte eine Hierarchie von Garantien aufdecken. Strukturelle Garantien betreffen die genaue Spannweite, die Reihenfolge und die Schemagültigkeit. Grundsicherungsgarantien betreffen die Ausrichtung von Beweisen und den Identitätsstatus. Die Hinrichtung garantiert die Konsequenzen eines formellen Problems. End-to-End-Garantien kombinieren sie bedingt. Der Benutzer sollte in der Lage sein, die schwächste Ebene zu sehen. Ein Ergebnis, das aus modellvorgeschlagenen Ereignissen und einem mechanisch verifizierten Übergang abgeleitet ist, sollte nicht einfach als "verifiziert" bezeichnet werden; es sollte angeben, dass die Ausführung über vorgeschlagene Beobachtungen überprüft wird.

Ein kleiner vertrauenswürdiger Kernel ist ein wichtiges Designziel. Der Kernel umfasst die Quell-Kanonikalisierung, Schema- und Typprüfung, formale Compiler, registrierte Ausführer oder Proof-Checker, Replay-Überprüfungen und die Validierung des kanonischen Ergebnisses. LLM Aufforderungen, Kandidatengeneratoren, Retrieval-Heuristiken und Erklärungsmodelle bleiben draußen. Vertrauenswürdige Komponenten können immer noch Fehler enthalten, aber ihr Verhalten ist stabil, testbar und überprüfbar. Dies ist eine wesentliche Verbesserung

gegenüber dem Vertrauen in ein sich entwickelndes Gründungsmodell als monolithischer Reasoner.

Die vorgeschlagene Pipeline ist nicht die einzige mögliche Umsetzung, sondern sie erfasst die in der Literatur offenbarten Verpflichtungen. Es erklärt, warum ein Solver notwendig, aber unzureichend ist. Es gibt abstrakte Interpretation und symbolische Ausführung verschiedene Rollen. Es unterstützt mehrere formale Sprachen, ohne eine gemeinsame Beweisschicht aufzugeben. Es ermöglicht LLM Kreativität bei der Entdeckung, Reparatur und Erklärung, während die symbolische Autorität über die Erdung und akzeptierte Ergebnisse erhalten bleibt. Am wichtigsten ist, dass es Misserfolg in ein benanntes Artefakt verwandelt - einen nicht unterstützten Typ, eine ungelöste Identität, eine unvollständige Abdeckung, einen Zusammenstellungsverlust, eine unzufriedene Formalisierung oder einen nicht lizenzierten Satz - und nicht eine vage Halluzination.

7. LongTextJS, CircuitJS und nllAgent: Eine kritische Fallstudie

Das öffentliche NaturalLanguageLinterAgent-Repository ist einer der wenigen Versuche, die Grenze zwischen Sprachdolmetschung und Regelausführung in einer ausführbaren Implementierung explizit zu machen. Das Projekt ist kein allgemeiner Natursprachler, und seine Dokumentation präsentiert es nicht als einen. Es ist eine Node.js 22+ ESM Bibliothek und Befehlszeilen-Laufzeit mit zwei steuerungsgesteuerten Produktmodi über die gleiche LongTextJS-CircuitJS-Architektur. Der Audit-Modus wendet eine versionierte Theorie auf ein bestehendes Markdown-Dokument an und sendet ein kanonisches Auditprodukt aus. Der Spezifikationsmodus wendet Planungstheorien auf eine hochrangige Idee an und gibt einen verifizierten Erzeugungsplan aus; ein optionales Sprachmodell kann diesen Plan als Prosa realisieren, aber die unveränderten Auditschaltungen bleiben das Akzeptanzorakel [AssistOS-AI 2026].

Das Repository ist als Fallstudie nützlich, da es die zentrale These dieses Buches in Softwareform verkörpert: Modelle können semantische Beobachtungen oder Prosa vorschlagen, aber die Produktionsautorität wird unveränderlichen Veröffentlichungen, expliziten Kompatibilitätsprüfungen, eingeschränkter Schaltungsdurchführung, Prüfern und kanonischen Artefakten zugewiesen. Das Projekt unterhält auch ein öffentliches seriöses Verzeichnis. Diese Praxis ist wichtig, weil eine große Dokumentationsoberfläche sonst architektonische Absichten wie umgesetzte Garantien aussehen lassen kann. Die folgende Diskussion unterscheidet das, was operativ etabliert ist, von dem, was eine Forschungshypothese bleibt.

Keine unabhängige Peer-Review-Bewertung oder technische Überprüfung von nllAgent, LongTextJS oder CircuitJS durch Durchsuchungen der Projektnamen, des Repositorynamens, der Dokumentation URL und Kombinationen mit "Review", "Evaluation", "Benchmark" und "Replikation" ab dem 2. August 2026. Die Suchergebnisse führten eher zum Projekt selbst als zu einer externen Bewertung. Das Urteil in diesem Kapitel ist daher die unabhängige Architekturüberprüfung des Autors, basierend auf dem öffentlichen Repository, Dokumentation, ausführbaren Beispielen, deklarierten Freigabeverfahren und aktuellen seriösen Problemen. Die Abwesenheit einer externen Überprüfung ist weder Lob noch Kritik; es bedeutet, dass Projektansprüche bis zur unabhängigen Reproduktion eng bleiben sollten.

7.1 Was die öffentliche Architektur implementiert und warum ihre Grenzen wichtig sind

Das System definiert drei Produktionsaktivitäten, die bewusst getrennt werden. Das Produktionsaudit liest eine unveränderliche veröffentlichte Version, stellt ein Dokument in LongTextJS zusammen, führt die Validierungsschaltungen der Version aus, überprüft die Kandidatenergebnisse und behält ein kanonisches CNL/Audit-1-Ergebnis mit einem gerenderten Bericht fort. Die Produktionsspezifikation stellt eine Idee in LongTextJS-Beobachtungen zusammen, wendet Planungsschaltungen an und erzeugt ein kanonisches CNL/Plan-1-Artefakt; optionale Realisierung kann dann Markdown erzeugen und an dieselbe Prüfmaschine senden. Das Lernen findet in einem Einweg-Staging-Arbeitsbereich statt. Ein Codierungsagent kann Kandidaten-Artefakte und Benchmarks generieren, aber hostseitige Steuerelemente lassen nur Whitelist-Dateien zu, und ein Betreuer muss eine Version explizit veröffentlichen und aktivieren. Dies macht das Lernen analog zur verwalteten Softwareentwicklung und nicht zur Selbstmodifizierung der Laufzeit.

Die wichtigste Arbeitseinheit ist nicht der Abschlussbericht, sondern das Run-Verzeichnis. Ein Audit bewahrt die kopierte Eingabe, das Quellprogramm, die ausgewählten Grundlagen-, Kompatibilitäts- und Abdeckungsergebnisse, die Schaltungsverfolgung, die Ergebnisse der Prüfergebnisse, die Ergebnisse, den kanonischen maschinenlesbaren Bericht und die vom Menschen lesbare Ansicht. Planungsläufe erhalten die entsprechenden Planartefakte. Ein Lauf, der aufhört, weil Wissen fehlt, zeichnet immer noch den Fehler auf und kann ein wiederverwendbares Problem verursachen. Die Designregel, dass fehlendes Wissen nie als Compliance gemeldet wird, ist von grundlegender Bedeutung. Es verhindert ein häufiges Versagen von Dokumentenagenten: "Ich habe keine Beweise gefunden" in "das Dokument erfüllt die Regel".

LongTextJS sollte nur im eingeschränkten Sinne als quellenseitiges Programm verstanden werden. Es weist willkürlichen Sätzen kein zwingendes Verhalten zu. Es erstellt eine feste Quellrevision in eine endliche, abfragebare Welt von Strukturen und Beobachtungen. Die wertvollen Eigenschaften sind Quellidentität, stabile Spannweiten, versionierte Beobachtungstypen, Herstelleridentität, Abdeckung, explizite Lücken und Widerspielbarkeit. Ein dokumentenseitiges Artefakt kann ausgeführt werden, da registrierte Verbraucher diese Beziehungen unter deklarierter Semantik abfragen und transformieren können; es liegt näher an einer semantischen Zwischendarstellung oder einem Datenbank-Loading-Programm als an uneingeschränktem Anwendungscode.

Diese Unterscheidung ist mehr als Terminologie. Wenn ein Modell einen Satz übersetzt, sollte seine Ausgabe nicht autoritativ werden, nur weil es in einer Programmiersprache serialisiert wird. Der Quelltext bleibt Beweismittel; die übersetzte Beobachtung zeichnet eine Interpretation dieser Beweise auf, die von einem benannten Produzenten unter einem Schema und einem Modellprofil erzeugt wurde. Die Schaltung bleibt eine eigene Theorie. Ein Ergebnis wird erst veröffentlicht, nachdem ein Prüfer die von der Veröffentlichung erwarteten Bedingungen überprüft hat. Diese Schichten verhindern, dass ein Parser stillschweigend Richter wird und verhindern, dass eine Erklärung zur einzigen Aufzeichnung einer Schlussfolgerung wird.

Das aktuelle Fundamentpaket veranschaulicht den beabsichtigten semantischen Umfang. Gewöhnliche Audit-, Plan- und Benchmark-Befehle wählen einen begrenzten Foundation-core

@1.1.0 aus, sofern nicht ausdrücklich deaktiviert. Das Paket erkennt dokumentierte kontrolliert-englische Formen für Zustand, Typ, genaue zeitliche Vorrang, ausgedrückt durch "vorher", genaue arithmetische, Quantität und wörtliche Emotionsbehauptungen. Fünf replay-verifizierte Schaltkreise überprüfen nur die veröffentlichten logischen, zeitlichen, arithmetischen, elementaren physischen und Emotions- / Typinvarianten. Das ausgewählte Paket und der Digest sind bestehen. Die sich ändernden politischen, geografischen, sozialen, wirtschaftlichen und rechtlichen Fakten werden absichtlich von der wahren Wahrheit der zeitlosen Stiftung ausgeschlossen. Das ist eine vertretbare Weltmodellgrenze. Ein System, das aktuelle Fakten ohne effektive Daten, Gerichtsbarkeiten, Quellen und Konfliktpolitik festcodiert, kann keine reproduzierbare semantische Ausführung bieten.

Das System unterstützt die deterministische und modellgestützte Materialisierung. Die Übersetzungsauswahl kann einen konfigurierten AchillesAgentLib-Sprachmodell-Agent bevorzugen, den aktuellen eingeschränkten Codierungs-Agent-Adapter verwenden oder ohne Modell für deterministische Agenten ausführen. Jeder Modellaufruf erhält eine rollenspezifische Fähigkeit anstelle des gesamten Agentenarbeitsbereichs. Wenn eine kritische Beobachtung nicht innerhalb des erforderlichen Schemas und Ressourcenbudgets materialisiert werden kann, ist das Ergebnis inkompatibel, unvollständig oder budgetvervollständigt. Das Modell kontrolliert nicht die Quellenkonstruktion, die Publikationspolitik oder die Bedeutung eines Prüfers. Genau dies ist die richtige Richtung für ein föderatives neurosymbolisches System: Das Modell ist am Rande mächtig, während symbolische Artefakte und die Akzeptanz der Host-Politik kontrolliert werden.

Die Deckung wird als ausführbare Verpflichtung und nicht als informelle Vertrauenswürdigkeit behandelt. Die Schaltungskompilierung arbeitet von ausgangserreichbaren Knoten rückwärts, um zu bestimmen, welche externen Beobachtungen ein Ergebnis beeinflussen können. Die Veröffentlichung zeichnet Verträge auf, die diese Anforderungen mit den verfügbaren Herstellern verknüpfen. Die Laufzeitkompatibilität fragt dann, ob das konkrete Dokument tatsächlich Beobachtungen mit dem erforderlichen Typ, Status, Kardinalität, Struktur, Kanal und Abdeckung ergab. Die erklärte Fähigkeit eines Produzenten wird nicht mit Beweisen verwechselt, dass das aktuelle Dokument erfolgreich verarbeitet wurde. Wenn eine kritische Nachfrage unerfüllt ist, stoppt der Lauf an einer benannten Grenze.

Dieses bedarfsgerechte Design eignet sich gut für lange Dokumente. Es vermeidet es, ein Modell zu bitten, eine Allzweckinterpretation jedes Satzes zu erstellen, bevor irgendeine Frage bekannt ist. Eine Schaltung kann die semantischen Dimensionen anfordern, die sie benötigt: genaue Begriffe, Zustandsbehauptungen, Größen, zeitliche Beziehungen oder Domänenbeobachtungen. Der Compiler kann nur ausgangsrelevante Beobachtungen materialisieren und das, was abgedeckt war und nicht abgedeckt war, bestehen. Der wissenschaftliche Vorteil sind nicht nur geringere Kosten. Es begrenzt unnötige semantische Verpflichtungen und macht eine unvollständige Formalisierung sichtbar.

CircuitJS ist die theoretisch-seitige Darstellung. Agentenbearbeitbare Schaltungsmodule sind auf einen deklarativen Ausdruck beschränkt, auf einfache Daten normiert und durch denselben Compiler geleitet, der für Nicht-Code-Schaltungsbeschreibungen verwendet wird. Sie können nur registrierte Operatoren und Prüfer aufrufen; Sie können willkürliches ausführbares Verhalten nicht in einer gelernten Regel verbergen. Eine Host-Anwendung kann eine überprüfte Laufzeiterweiterung mit echten Algorithmen installieren, aber Ein- und Kontext sind eingefrorene

Klardatenkopien, Ergebnisse müssen einfache Daten sein, die Cache-Identität enthält den Erweiterungsdigest und Veröffentlichungen, die von der Erweiterungssperre abhängen, die digest. Die Standard-Befehlszeilenschnittstelle akzeptiert keine beliebigen ausführbaren Erweiterungspfade.

Diese Grenze behebt eines der schwerwiegendsten praktischen Risiken in „agent-lernten“ Regelsystemen. Wenn ein Codierungsagent in jeder Schaltung beliebige JavaScript erzeugen kann, wird das Erlernen eines Linters zur Remote-Code-Generierung und das semantische Verhalten einer Freigabe wird schwer zu inspizieren. Durch die Einschränkung der Erstellung gewöhnlicher Schaltungen auf registrierte Vorgänge wird ein endlicher Betriebswortschatz erstellt. Spezialisierte Algorithmen bleiben möglich, aber sie treten durch einen separaten Engineering-, Überprüfungs- und Versionierungspfad ein. Die Architektur unterscheidet daher die Evolution von Theorien von der Evolution der vertrauenswürdigen Laufzeit.

Die Verifizierergrenze ist ebenso wichtig. Die Ausführung von Schaltkreisen kann Kandidatenergebnisse erstellen, aber ein Prüfer steuert, ob diese Kandidaten in das kanonische Audit oder Plan eintreten. In den begrenzten Beispielen kann der Prüfer Quellanker, Beobachtungen, Freigabe-Metadaten und die anzuwendende Richtlinie erneut lesen. Es reicht nicht aus, dass ein Ausführungsknoten ein Objekt aussendet, dessen Felder wie eine Warnung aussehen. Die Prüferin legt die vom Produkt erwartete spezifische Eigenschaft fest. Dies beweist nicht automatisch die semantische Interpretation richtig, sondern verhindert die versehentliche oder bösartige Schaltungsausgabe von der Maskierung als akzeptierte Erkenntnis.

Der Spezifikationsmodus zeigt, wie die gleiche Architektur die kontrollierte Erzeugung unterstützen kann, ohne dem Autorenmodell die Autorität zu geben. Planungskreisläufe bauen einen kanonischen Plan auf und erfordern Regel-zu-Plan-Zeugen für angewandte Regeln. Die optionale Realisierung verwandelt den Plan in Prosa. Die Kandidatenprosa durchläuft dann den Audit-Modus, und das Realisierungsmodell beurteilt seine eigene Compliance nicht. Dies ist stärker als die nur auffordernde, eingeschränkte Erzeugung, da das maßgebliche Objekt der Plan und die Audit-Spur ist, nicht die selbstberichtete Adhärenz des Modells.

Auch die experimentelle Abfrage-Erst-Teilmenge ist von Bedeutung. Canonical LongTextJS wird als endliche schreibgeschützte Abfragewelt entlarvt. Eingegebene Abfragen und evidenzbewusste Tabellenwerte werden in die gewöhnliche CircuitJS-Ausführung gesenkt. Der Compiler überprüft unterstützte Beziehungen, Ausdrücke, Anti-Joins, Ordering- und Hit-Richtlinien; Veröffentlichung zeichnet Abfrageverträge und Quellkarten auf; Benchmarks vergleichen direkte Referenzbewertung mit einer verringerten Ausführung. Die aktuelle produktionsredaktionelle Veröffentlichung bleibt bewusst einfacher, während Aggregate, geordnete Muster, native Indizes und fortgeschrittene Argumente über direkte Operatoren oder Grafiken abgewickelt werden. Diese Zurückhaltung ist der Beanspruchung einer ausgereiften universellen Abfragesprache vorzuziehen, bevor die Semantik und Optimierung stabil sind.

Das Publikationsmodell des Repositorys stärkt die Reproduzierbarkeit. Named Agents besitzen separates Autoritätsmaterial, Schemata, Extraktionsprofile, Schaltungen, Benchmark-Suiten, Kandidaten, Releases, Runs, Probleme und Feedback. Ein Codierungsbeamter kann Kandidaten vorschlagen, kann sie aber nicht aktiv machen. Die Produktion liest eine unveränderliche Veröffentlichung. Diese Struktur ermöglicht es, zu reproduzieren, welche Theorie, Grundlage, welches Produzentenprofil, welche

Erweiterungsdigest und welcher Verifier ein Ergebnis erzeugt hat. Es ermöglicht auch mehreren Codierungsagenten oder Modellfamilien, konkurrierende Theorien aufzubauen, ohne stillschweigend eine aktive Ontologie oder Freisetzung zu teilen.

Der stärkste implementierte Anspruch ist daher eher architektonisch als semantisch: nllAgent hat einen disziplinierten Weg von unveränderlichen Quellen zu versionierten Beobachtungen, von Beobachtungsanforderungen bis zur Kompatibilität, von eingeschränkten Schaltkreisen bis zur prüfergesteuerten Veröffentlichung und von einem kanonischen Produkt zu einem menschenlesbaren Bericht. Es hat explizite Stop-Staaten. Es muss nicht so tun, als wäre jedes Dokument onlogisch vereinbar. Sie behandelt Modellergebnisse als rollengebundene Vorschläge. Dies sind substanzielle Errungenschaften im Vergleich zu herkömmlichen LLM-Agent-Pipelines, bei denen die Eingabeaufforderung, Interpretation, Argumentation und Erklärung oft nur in einem transienten Kontext existieren.

Die Architektur stellt jedoch nicht fest, dass die willkürliche natürliche Sprache korrekt begründet ist. Stabile Spannen belegen, dass Beweise an einem Ort existieren; Sie beweisen nicht, dass ein Akteur, ein Ereignis, eine Ausnahme oder eine Modalität korrekt interpretiert wurde. Eine typisierte Beobachtung kann sich immer noch auf die falsche Entität beziehen. Ein vollständiger Lauf kann nur für den begrenzten Beobachtungsvertrag vollständig sein, nicht für jede relevante Bedeutung, auf die ein Mensch schließen könnte. Die eigene Dokumentation und das Ausgaberegister des Projekts sind in diesem Punkt entsprechend vorsichtig. Der Wert der Implementierung ist, dass diese Einschränkungen Orte haben, die dargestellt werden müssen, anstatt in einer endgültigen Antwort versteckt zu sein.

7.2 Unabhängige Bewertung: Stärken, aktuelle ernste Probleme und die erforderliche Evolution

Meine Gesamtbewertung ist positiv über die Vertrauensgrenzen und vorsichtig in Bezug auf die semantischen Behauptungen. nllAgent ist mehr als ein prompter Wrapper, mehr als eine Vektor-Abruf-Pipeline und mehr als eine Sammlung von JSON-Schemas. Seine quellenseitigen und theoretisch-seitigen Artefakte, unveränderliche Veröffentlichung, eingeschränkte Authoring-Grenze, Wiederholungsprüfung, Abdeckungsprüfungen und kanonische Produkte bilden ein glaubwürdiges Forschungssubstrat. Die Architektur ist besonders gut auf das föderative Muster abgestimmt, das in den jüngsten neurosymbolischen Bewertungen identifiziert wurde: Die neuronale Komponente schlägt Sprache vor oder realisiert sie, während die symbolische Komponente ihre eigene Identität, Semantik und Akzeptanzkriterien [Chatzikyriakidis and Lappin 2026] behält.

Die vorliegenden Erkenntnisse des Projekts stützen jedoch die begrenzte Machbarkeit, nicht die allgemeine ausführbare natürliche Sprache. Die Standard-Grundlage erkennt ein kleines kontrolliertes Fragment. Die redaktionelle Demonstration verwendet enge Operatoren und Prüfer. Die Query-First-Kompilierung ist experimentell. Die kontrollierte Realisierung ist begrenzt. Keine unabhängige Bewertung hat Formalisierungstreue, Identitätslösung, Übertragung über Domänen, Fehlerkalibrierung oder Überlegenheit gegenüber einfacheren Basislinien festgestellt. Die richtige Publikationsstrategie besteht daher darin, genau anzugeben, welche Eigenschaften umgesetzt werden, und jeden stärkeren Anspruch an einen Benchmark- und Garantierahmen anzuhängen.

Das aktuelle seriöse Register identifiziert sechs technische und semantische Einschränkungen. Die erste ist die inkrementelle Inkrementalität des Schedulers. Die Laufzeit hat nützliche Abhängigkeits- und Caching-Mechanismen, aber sie beansprucht noch keine vollständig allgemeine inkrementelle Fixpoint-Engine für beliebige Schaltkreise, rekursive Beziehungen, alternative Welten und Nicht-Monoton-Updates. Dies ist wichtig für lange Dokumente, da eine kleine Bearbeitung nur die semantischen Artefakte und Schaltungszustände, die davon abhängen, ungültig machen sollte. Eine naive vollständige Wiedergabe ist korrekt, aber teuer; ein unklanglicher inkrementeller Scheduler ist schnell, aber gefährlich. Die Lösung erfordert explizite Monotonizitäts- und Effektkontrakte, Abhängigkeitsprovenienz, Epochentrennung für Rückzüge und Referenzvergleiche zwischen inkrementeller und vollständiger Ausführung.

Das zweite Problem ist, dass die Interpretationsunterstützung noch nicht gleichmäßig durch jeden Ausführungspfad fließt. Ein Dokument kann konkurrierende Lesungen, Welten, Kontexte oder Anwesenheitsbedingungen enthalten, aber ein Operator, ein Cache-Schlüssel, eine Aggregation oder ein Verifizierer kann sie versehentlich kollidieren. Dies ist das zentrale Hindernis, um LongTextJS als abstrakten semantischen Zustand zu behandeln. Jeder Wert, der von einer Interpretation abhängt, muss seinen Kontext tragen. Joins müssen kompatible Kontexte kombinieren. Die Aggregation muss sagen, ob sie innerhalb einer Welt oder über Alternativen hinweg reicht. Die Cache-Identität muss die Kontextformel enthalten. Operatoren, die Alternativen nicht erhalten können, sollten für unvereinbar erklärt werden, anstatt stillschweigend eine zu wählen.

Dieses Problem ist keine geringfügige Feature-Lücke. Das wissenschaftliche Kernversprechen einer ausführbaren natürlichen Sprache besteht oft darin, dass das System

sagen kann, was unter allen zulässigen Lesungen wahr ist, was unter mindestens einem möglich ist und welches Ergebnis von einer umstrittenen Auslegung abhängt. Wenn Alternativen in einer internen Ausführungsrouten verloren gehen, wäre eine polierte May/Must-Schnittstelle irreführend. Das Projekt sollte daher die Interpretationskonservierung zu einer Ganz-Laufzeit-Invariante machen und sie durch erzeugte endliche Welten, metamorphe Eigenschaften und differentielle Wiedergabe testen.

Das dritte Problem ist die konservative Kompatibilität für undurchsichtiges Verhaltensverhalten. Registrierte Erweiterungen und Operatoren sind notwendig, da einige nützliche Algorithmen deklarativ zum Ausdruck bringen. Ein Compiler kann jedoch keine präzisen semantischen Anforderungen, Effekte, Monotonizität, Ressourcennutzung oder Zeugenverpflichtungen von undurchsichtigem JavaScript aufschließen. Konservative Ablehnung ist sicherer als optimistische Akzeptanz, aber übermäßige Opazität reduziert die Composability und macht die nachfrageorientierte Materialisierung weniger effektiv. Jedes prozedurale Primitivum benötigt einen Registry-eigenen Vertrag, der Eingabe- und Ausgabetypen, akzeptierte epistemische Status, Deckungsanforderungen, mögliche Effekte, Determinismus, Monotonität, Cache-Verhalten, Kostenklasse und verifizierbarer Zeuge erklärt. Der Vertrag sollte maßgeblicher sein als die Kommentare des Erweiterungsautors.

Das vierte Problem ist die minimale Abfrageplanung und zeitliche Indexierung. Eine endliche Abfragewelt kann ohne ausgeklügelte Optimierung korrekt ausgeführt werden, aber die Verwendung im Dokumentenmaßstab erfordert eine Verknüpfungsplanung, Selektivitätsschätzungen, Quell- und Entitätsindizes, zeitliche Intervallindizes und inkrementelle Wartung. Diese Optimierungen sind nicht nur Performance-Details. Ein Planer, der Abfragen umschreibt, muss die Evidenzmultiplizität, die Kontextformeln, die Ordnung, die Anti-Join-Semantik und die Abdeckung bewahren. Native Indizes müssen eher Einwegbeschleuniger als Wahrheitsquellen bleiben. Differentialtests sollten optimierte und Referenzauswertung über generierte Welten vergleichen, insbesondere für Abwesenheitsabfragen, zeitliche Überschneidungen, alternative Interpretationen und doppelte Beweise.

Das fünfte Thema ist bewusst eine enge kontrollierte Erzeugung. Die genaue oder überprüfte Realisierung von begrenzten Erkenntnissen und Plänen ist wertvoll, aber sie ist noch nicht auf eine verifizierte Langformprosa hinaus. Die Langform-Generation führt Aggregation, Pronomen, Ellipse, rhetorische Betonung, Diskursbeziehungen und Implikationen ein, die durch Satzordnung geschaffen wurden. Eine Realisierung kann nur lizenzierte atomare Aussagen enthalten und immer noch eine nicht unterstützte kausale oder normative Erzählung implizieren. Der nächste Schritt sollte eher eine nachweisführende Realisierung als ein breiterer, uneingeschränkter Schriftsteller sein. Ein kanonisches Ergebnis sollte Claim-Identifizierer, Entitäts- und Mengenschlitze, Modalität, Unsicherheit und zulässige Diskursbeziehungen liefern. Der generierte Text sollte neu analysiert und mit diesen Behauptungen verglichen werden. Kritische Fehlanpassungen sollten den Text ablehnen; Stilmaterial außerhalb des geprüften Fragments sollte auf einer niedrigeren Sicherungsebene gekennzeichnet werden.

Das sechste Thema ist die szenariospezifische Mutationsunterstützung. Die geordnete Benchmark-Genauigkeit kann Regeln verbergen, die inert, redundant oder falsch ausgelegt sind. Eine Schaltung kann alle kuratierten Beispiele passieren, selbst wenn eine Ausnahme ignoriert wird, weil kein Fall sie aktiviert. Eine seriöse Bewertung sollte typisierte semantische Mutanten erzeugen, die Schwellenwerte, Vergleiche, Quantifizierer, Negation, Modalität,

Identitätsverbindungen, zeitliche Reihenfolge, Evidenzstatus, Deckungsprädikate, Ausnahmebereich, Regelpriorität und Prüfrouten ändern. Eine Testsuite „tötet“ eine Mutante, wenn sich das erwartete kanonische Ergebnis in der erforderlichen Weise ändert. Überlebende Mutanten zeigen schwache Tests oder semantisch redundante Klauseln. Mutation sollte Quellprogramme, Schaltungstheorien, Compiler-Absenkungen und Realisierungsbeschränkungen abdecken, nicht nur Strings in Beispielen.

Diese sechs Ausgaben sind informativer als ein allgemeines "Prototyp" -Etikett, da sie erkennen, wo die aktuelle Architektur entweder Maßstab oder Bedeutung verlieren kann. Sie suggerieren auch einen kohärenten Evolutionspfad. Der erste Meilenstein sollte eine begrenzte formale Semantik für die Welt des semantischen Ladens und der Abfrage sein. Es muss nicht uneingeschränktes Englisch formalisieren. Es sollte unveränderliche Quellwelten, Beobachtungsidentität, epistemischen Status, Abdeckung, Alternativen, Transaktionen, Abfrageergebnisse, Veröffentlichung und Widerruf definieren. Wichtige Invarianten können über endliche Instanzen modelliert und in der Implementierung eigenschaftsmäßig getestet werden.

Der zweite Meilenstein sollte die Interpretationsausführung sein. Präsenzbedingungen oder faktorisierte Kontexte sollten alternative Beobachtungen begleiten. Der Scheduler, die Abfrage-Engine, die primitive Registrierung, der Cache, der Verifier und die kanonischen Produkte sollten diese Kontexte entweder beibehalten oder Inkompatibilität deklarieren. Möge, darf, kann und unbekannt sollte aus expliziten Mengen oder Abstraktionen zulässiger Welten berechnet werden, nicht aus Modellvertrauen. Damit würde die konkrete Brücke von der bestehenden Architektur zur abstrakten Interpretation entstehen.

Der dritte Meilenstein sollte ein Formalismusregister und nicht ein auf unbestimmte Zeit erweitern CircuitJS. CircuitJS wird am besten als Orchestrierung, Evidenzvertrag und Publikationssprache verwendet. Spezialisierte Motoren sollten eigene Spezialsemantik besitzen: relationale und Datalog-Bewertung für Schließung und Verknüpfungen; Entscheidungstabellen für politische Vorrang; SAT, SMT, CSP oder MaxSAT für Einschränkungen; Automaten für bestellte Muster; Zeitliche Lösungs- und Modellprüfer für Traces; symbolische Ausführung für Pfadzeugen; Dataflow-Frameworks für Jeder Adapter sollte ein eingegebenes Eingabefragment, Übersetzungsannahmen, Ausgabezertifikat, Wiedergabeprüfung und Fehlermodi deklarieren.

Der vierte Meilenstein sollte die gegenbeispielgesteuerte semantische Verfeinerung sein. Eine grobe abstrakte Analyse kann einen möglichen Verstoß erkennen. Ein Löser oder symbolischer Testamentsvollstrecker sucht dann ein zusammenhängendes Zeugnis. Wenn der Zeuge zufriedenstellend und wiederaufspielbar ist, meldet das System es mit Quellenspannweiten. Wenn es falsch ist, sollte ein unzufriedener Kern oder Konfliktsatz angeben, welche Identität, zeitliche, modale, Quantität, Ausnahme oder Deckungsunterscheidung verloren gegangen ist. Der Verfeinerungsantrag sollte nur auf die jeweilige Quelle und den jeweiligen Hersteller abzielen. Das reparierte semantische Artefakt wird zu einer neuen Version und das Gegenbeispiel wird zum Regressionsfall. Diese Schleife wäre eine wirklich neuartige Synthese von LLM-basiertem semantischem Vorschlag, abstrakter Interpretation und symbolischer Ausführung.

Der fünfte Meilenstein sollte eine stärkere Erdungsschicht sein. Typsicherheit und Quellanker sind notwendig, aber nicht ausreichend. Die Laufzeit erfordert Kandidatenidentitätssätze, explizite gleiche / verschiedene Einschränkungen, zeitliche Gültigkeit, Einheits- und Gerichtsstandsprüfungen, Quellenautorität und Konfliktrichtlinien.

Modellgenerierte Top-One-Links sollten Objekte nicht destruktiv zusammenführen. Kritische Entitäten sollten autoritative Kennungen oder menschliche Bestätigung verwenden. Externes Wissen sollte durch schreibgeschützte, inhaltsbezogene Pakete mit Quellen, effektiven Intervallen, Gerichtsbarkeit, Ablauf, Konfliktlösung und Garantieobergrenzen eingehen. Der Web-Retrieval kann Pack-Updates vorschlagen, aber es sollte nicht zu impliziten zeitlosen Wahrheit werden.

Der sechste Meilenstein sollte eine unabhängige Bewertung sein. Deterministische Vorrichtungen stellen Software-Reproduzierbarkeit her, aber nicht die Qualität der semantischen Live-Übersetzung. Ein Opt-in-Bewertungsprofil sollte Modell, Anbieter, Aufforderungen, Tool-Versionen, Temperatur und Erfassungsrichtlinien anheften; Wiederholungsfälle, um die Abweichung zu messen; Kosten und Latenz aufzeichnen; und stillen Fallback verbieten. Ein separat implementierter Referenzbewerter sollte Abfrage- und Schaltungsergebnisse vergleichen, soweit dies möglich ist. Die menschliche Annotation sollte sich auf Source-to-Semantic Fidelity und ergebnisrelevante Mehrdeutigkeiten konzentrieren. Baselines sollten direktes LLM-Urteil, LLM plus Abruf, LLM-zu-Eins-Formalismus, kontrolliertsprachige Zusammenstellung und deterministische Mustersysteme umfassen. Das Projekt soll zeigen, wann seine zusätzliche Architektur einen messbaren Wert erzeugt.

Eine sinnvolle Bewertung kann zusammengefasst werden, ohne die Ansprüche aufzublasen. Quellenerhaltung, unveränderliche Veröffentlichungen, Spurenpersistenz, Trennung von Dokumentenbeobachtungen von wiederverwendbarer Theorie, explizite Abdeckung, Erstellung eingeschränkter Schaltung, verifikatorgesteuerte Veröffentlichung und geregeltes Lernen sind starke Designentscheidungen. Allgemeines semantisches Verständnis, Interpretation-vollständige Ausführung, Planung von Produktionsabfragen, Proof-Carrying-Long-Form-Generierung, umfassende Mutation und unabhängige empirische Validierung bleiben offen. Die Glaubwürdigkeit der Architektur kommt zum Teil davon, diese Grenze zuzugeben.

Die wichtigste wissenschaftliche Möglichkeit besteht darin, keine weitere DSL-Syntax hinzuzufügen. Es soll eine bedingte semantische Solidität für ein begrenztes, aber nicht triviales Fragment nachweisen. Ein veröffentlichbares Ergebnis könnte eine Dokumentsprache angeben, die Entitäten, Ereignisse, zeitliche Beziehungen, Größen, Modalitäten, explizite Ausnahmen und alternative Referenzen enthält; die zugegebene Interpretationsmenge definieren; beweisen oder mechanisch testen, dass jedes veröffentlichte May/Must-Ergebnis in Bezug auf diese Menge korrekt ist; zeigen, dass symbolische Verfeinerung eine messbare Klasse von falschen Warnungen eliminiert; und überprüfen, ob generierte Erklärungen das kanonische Ergebnis bewahren. Ein solches Ergebnis wäre schmäler als "ausführbares Englisch", aber wesentlich wichtiger.

Die wichtigste praktische Gelegenheit ist eine neue Klasse von Dokumenten- und Agentensystemen, die bereit sind, aufzuhören. Ein normaler LLM-Assistent ist optimiert, um weiterhin eine Antwort zu erstellen. Eine symbolisch begründete Laufzeit kann inkompatible Ontologie, ungelöste Identität, unzureichende Abdeckung, nicht unterstützten Formalismus, Budgeterschöpfung, falsches Gegenbeispiel oder nicht lizenzierte Realisierung zurückgeben. Diese Ergebnisse mögen in einer Demonstration weniger beeindruckend erscheinen, aber sie sind Voraussetzung für einen zuverlässigen Einsatz in Wissenschaft, Regulierung, Ingenieurwesen und institutioneller Entscheidungsfindung.

Mein unabhängiges Urteil ist daher, dass nllAgent viele der richtigen architektonischen Grenzen gewählt hat und dem Druck widerstehen sollte, seine Ansprüche schneller zu erweitern, als es seine semantischen Beweise erweitert. Es ist bereits ein glaubwürdiger Prototyp einer verwalteten, quellengemerdeten, theoretisch getrennten Sprachanalyse. Es ist noch kein Beweis dafür, dass uneingeschränkte natürliche Sprache mit formalen Garantien ausgeführt werden kann. Sein bester Weg ist es, eine experimentelle Plattform für den engeren und vertretbareren Anspruch zu werden, der in diesem Buch entwickelt wurde: LLMs entdecken formale Kandidaten, ein symbolischer Kernel erdungs- und konsequenzgesteuert, und die Ausgabe natürlicher Sprache bleibt einem widerspielbaren kanonischen Ergebnis untergeordnet.

8. Aufbau und Validierung einer neurosymbolischen Laufzeit

8.1 Implementierungsstrategie: Ein minimales, aber wissenschaftlich ehrliches ausführbares System für Sprache

Die in diesem Buch vorgeschlagene Architektur ist nur dann ehrgeizig, wenn sie vage beschrieben wird. Sobald seine Verpflichtungen getrennt sind, ist das erste umsetzbare System vergleichsweise bescheiden. Sie braucht keine willkürliche Sprache zu verstehen, eine universelle Ontologie zu konstruieren oder jede Schlussfolgerung zu beweisen. Es muss eine begrenzte Dokumentenklasse akzeptieren, genaue Beweise bewahren, eine kleine Familie von typisierten Beobachtungen materialisieren, eine eingeschränkte Familie von Regeln in ein oder zwei formale Motoren zusammenstellen, Zeugen oder explizite Unbekannte zurückgeben und verhindern, dass die endgültige natürlichsprachige Schicht das symbolische Ergebnis ändert. Der wissenschaftliche Wert ergibt sich daraus, dass jede Einschränkung sichtbar und messbar wird.

Die erste Designentscheidung ist der Zielanspruch. Ein Forschungsprototyp sollte nicht mit „dem System versteht und ausführt natürliche Sprache“ beginnen. Es sollte mit einem Anspruch beginnen, wie z. B.: gegebene Dokumente aus einem deklarierten Genre, eine versionierte Ontologie, eine endliche Familie von Beobachtungsproduzenten und eine Reihe von veröffentlichten Regeln, die Laufzeit bestimmt, ob jede Regel erfüllt, verletzt, möglicherweise verletzt, unterstützt oder außerhalb der semantischen Berichterstattung über die Veröffentlichung ist. Das Ergebnis ist widerspielbar und mit Quellspannen verknüpft. Diese Behauptung ist eng genug, um zu testen und breit genug, um Compliance-Überprüfung, Anforderungsanalyse, wissenschaftlich-dokumentenartiges Faulenzen, Protokollinspektion und kontrollierte Planung zu unterstützen.

Die Quellschicht muss bewusst langweilig sein. Jedes Dokument wird als unveränderliche Revision mit einem stabilen Digest gespeichert. Strukturelle Einheiten – Überschriften, Absätze, Sätze, Listenelemente, Tabellenzellen, Beschriftungen, Fußnoten und eingebettete Objekte – erhalten persistente Bezeichner. Normalisierung wird aufgezeichnet, anstatt stillschweigend angewendet. Zeichenversätze sollten sich auf eine kanonische Quelldarstellung beziehen, während Mappings Positionen in der Originaldatei beibehalten. Eine Aussage in einer Tabelle und ein visuell identischer Satz in einem Absatz können unterschiedliche strukturelle Rollen tragen; diese Unterscheidung muss die Extraktion überstehen. Wenn OCR unvermeidlich ist, sind die OCR-Ausgabe und das Seitenbild separate Beweisobjekte, und die Unsicherheit in der Transkription wird nicht in gewöhnlichen Text zusammengebrochen.

Diese Quellendisziplin ist kein administrativer Gemeinkosten. Das macht symbolische Schlussfolgerungen anfechtbar. Ein Ergebnis, das nur eine generierte Zusammenfassung zitiert, kann nicht unabhängig überprüft werden. Ein Ergebnis, das mit exakten Text-, Tabellenzellen- und Transformationsaufzeichnungen verknüpft ist, kann überprüft werden, auch wenn sich jedes beteiligte Sprachmodell geändert hat. Das Quellpaket sollte daher als Teil des vertrauenswürdigen Audit-Datensatzes betrachtet werden, obwohl die Extraktoren, die es erstellen, fehlbar bleiben und formatspezifische Tests erfordern.

Die zweite Ebene ist eher ein Evidenzdiagramm als eine einzelne Parse. Der Graph enthält Erwähnungen, Entitäten, Ereignisse, Zustände, Größen, zeitliche Ausdrücke, Modalitäten, Beziehungen und Quelllinks. Entscheidend ist auch, dass es Alternativen und Status darstellt. Ein von Modell vorgeschlagener Entitätslink wird nicht als bedingungslose Gleichheit gespeichert. Es ist eine Kandidatenidentitätsbeziehung mit einem Produzenten, Score, unterstützende Spannen, widersprüchliche Beweise und ein epistemischer Status wie vorgeschlagen, bestätigt, akzeptiert, abgelehnt oder ungelöst. Eine zeitliche Beziehung, die von „bald danach“ abgeleitet wurde, sollte nicht auf eine willkürliche genaue Dauer normalisiert werden; sie sollte die qualitative Beziehung und die deklarierte Domäneninterpretation beibehalten.

Diese Schicht sollte endlich und aufgabengebunden bleiben. Das System konstruiert kein komplettes Weltmodell. Es materialisiert die von den veröffentlichten Schaltungen und der aktuellen Abfrage geforderten Unterschiede. Die Query-first-Materialisierung, die bereits in nllAgent untersucht wurde, ist der korrekte Standardwert. Eine Schaltung erklärt, dass sie die Identität des Akteurs, die Beziehungen zwischen Dokumentenreihenfolge, deontische Modalität und effektive Daten benötigt. Der Compiler leitet diese Anforderungen vor teurer Interpretation ab. Die Produzenten generieren dann nur die Beobachtungen, die erforderlich sind, um die Kompatibilität zu entscheiden und die Regel auszuführen. Dies reduziert die Kosten und, was noch wichtiger ist, begrenzt die semantische Oberfläche, auf der Fehler auftreten können.

Die dritte Ebene ist ein symbolischer Erdungskern. "Erden" wird oft lose verwendet, um zu bedeuten, dass eine Antwort eine Passage zitiert. Hier hat es eine strengere Bedeutung: Jedes Symbol, das zur formalen Ausführung zugelassen ist, hat einen typisierten Referenz, Provenienz, Umfang und Status, der für den Zielformalismus ausreicht. Ein Prädikat wie `approved(x, y, t)` kann nicht nur eingeführt werden, weil ein LLM diese Token produziert hat. Der Kernel überprüft, ob `x` und `y` sich auf zulässige Entitätstypen beziehen, dass es sich um einen unterstützten Zeitwert oder Intervall handelt, dass der Beobachtungshersteller für dieses Prädikat autorisiert ist, dass die Quellspanne in der deklarierten Revision besteht und dass der Status die Garantieranforderung der Regel erfüllt.

Erdungsverträge sollten Beweise von Ontologie unterscheiden. Es gibt Hinweise darauf, dass eine Spannweite eine Beobachtung unterstützt. Ontologie sagt, welche Art von Dingen die Beobachtung bezeichnet und welche Beziehungen zulässig sind. Domain-Axiome sagen, was aus diesen Beziehungen folgt. Das Wissen der Welt liefert Ansprüche aus der Außenwelt. Diese Schichten dürfen nicht zusammengeführt werden. Ein Firmenregister kann feststellen, dass zwei Namen eine juristische Person während eines angegebenen Zeitraums bezeichnen. Eine Domain-Ontologie kann erklären, dass juristische Personen Vereinbarungen unterzeichnen können. Eine Politiktheorie kann erklären, dass eine Unterschrift durch einen bevollmächtigten Vertreter als Genehmigung gilt. Ein Modell kann alle drei Verbindungen vorschlagen, aber jede Verbindung gehört einer anderen Autorität und sollte separat akzeptiert oder abgelehnt werden.

Identität verdient ihren eigenen Service, weil sie die häufigste versteckte Abhängigkeit in der Langdokumenten-Vermutung ist. Der Dienst sollte Erwähnungscluster, explizite Aliase, Identifikatoren, Organisationsrollen, zeitliche Validität und ausgewählte Welten verwalten. Es sollte mehrere ungelöste Kandidaten erlauben, anstatt eine frühe Vereinigung zu erzwingen. In einer ersten Veröffentlichung kann Identität absichtlich konservativ sein: explizite Kennungen und exakt deklarierte Aliase werden akzeptiert; rollenbasierte und kontextbezogene Links bleiben vorgeschlagen; nicht unterstützte dokumentarische Identität stoppt Regeln, die dies

erfordern. Diese konservative Politik wird die scheinbare Abdeckung verringern, aber die Bedeutung der akzeptierten Ergebnisse erheblich verbessern.

Die Zeit braucht eine ähnlich explizite Domain. Eine praktische Erstdarstellung kann exakte Augenblicke, geschlossene oder offene Intervalle, qualitative Ordnung und unbekannte Grenzen kombinieren. Die Laufzeit muss die Auftragsreihenfolge von der Ereignisreihenfolge und der Veröffentlichungszeit von der Wirkzeit unterscheiden. Zeitliche Phrasen wie „vor der Bereitstellung“, „innerhalb von dreißig Tagen“, „während die Autorisierung gültig bleibt“ und „nach jeder Materialänderung“ induzieren unterschiedliche Strukturen. Der Compiler sollte eine zeitliche Regel ablehnen, wenn der Evidenzgraph nur Beobachtungen der Dokumentreihenfolge aufweist, wenn eine Ereigniszeitsemantik erforderlich ist. Stille Substitution zwischen temporalen Domänen ist eine klassische Quelle für plausible, aber ungültige Schlussfolgerungen.

Modalität und normative Kraft erfordern eher ein kleines Gitter als ein Boolesches. Zumindest sollte das System die Behauptung, den Bericht, den Glauben, die Möglichkeit, die Fähigkeit, die Erlaubnis, die Verpflichtung, das Verbot, die Empfehlung, die Absicht, den Zustand und die kontrafaktischen unterscheiden. Sie sollte auch den Inhaber und Begünstigten einer Norm erfassen, soweit dies relevant ist. „Der Anbieter kann Daten offenlegen“ und „der Anbieter muss Daten offenlegen“ teilen Vokabular, haben aber entgegengesetzte Konsequenzen für die Einhaltung. "Das Dokument besagt, dass der Anbieter dem nachgekommen ist" ist ein Beweis für einen Bericht, nicht unbedingt ein Beweis für das zugrunde liegende Ereignis. Eine formale Laufzeit wird gerade nützlich, indem sie diese Zusammenbrüche ablehnt.

Sobald geerdete Beobachtungen existieren, berechnet die vierte Schicht einen abstrakten semantischen Zustand. Eine minimale Domäne kann für jedes abfragerrelevante Angebot darstellen, ob es in allen zulässigen Interpretationen wahr ist, in einigen wahr, in allen falsch, widersprochen oder ungelöst ist, weil die Abdeckung fehlt. Die Implementierung muss nicht mit einer ausgeklügelten Gitterbibliothek beginnen. Es kann mit expliziten endlichen Alternativen beginnen und eine mechanisch abgeleitete May/Must Zusammenfassung. Wenn die Skala wächst, können abstrakte Domänen Identitätskandidaten, numerische Bereiche, zeitliche Intervalle, modale Status und Provenienzsätze zusammenfassen, ohne jede Welt aufzuzählen.

Der abstrakte Zustand erfüllt zwei Aufgaben. Erstens bietet es eine konservative dokumentenweite Verbreitung. Wenn jede akzeptierte Interpretation eine Definition mit einem Begriff verbindet, ist diese Beziehung eine Tatsache. Wenn nur eine von drei plausiblen Identitätszuweisungen einen Interessenkonflikt schafft, ist der Konflikt eher ein möglicher als ein verifizierter Verstoß. Zweitens entscheidet sie, wo eine teurere Ausführung gerechtfertigt ist. Ein Muss-sicheres Ergebnis kann vorzeitig enden. Ein May-Vollstör-Ergebnis löst eine Zeugensuche aus. Ein Unbekannter, der durch fehlende Produzentenabdeckung verursacht wird, löst ein Problem mit der semantischen Kompatibilität und nicht einen Löseranruf aus.

Die fünfte Schicht ist ein Formalismusplan. Der Plan ist keine freiformende Prosa, die von einem Agenten verfasst wurde. Es ist eine typisierte Zusammenstellungsentscheidung, die angibt, welche Frage nach welcher Engine gestellt wird, welche Beobachtungen Inputs sind, was garantiert, dass die Engine zurückkehrt und wie ihr Ergebnis auf das kanonische Ergebnisvokabular abschneidet. Die Wahl sollte eher von semantischer Form als von Mode getrieben werden.

Die relationale Bewertung ist die erste Engine, die implementiert wird, da viele nützliche Dokumentenprüfungen auf Joins, Ordervergleiche, Gruppierungen und festgelegte

Unterschiede reduziert werden. Regeln wie "jeder definierte technische Begriff wird konsistent verwendet", "jeder akzeptierte Anspruch hat mindestens eine Quelle", "keine Anforderungskennung wird dupliziert" oder "alle aufgeführten Risiken haben einen Minderungseigentümer" erfordern keinen Theorem-Prüfer. Ein relationaler Kern ist transparent, effizient und einfach zu testen. Datalog erweitert diesen Kern, wenn eine vorübergehende Schließung oder rekursive Abhängigkeiten erforderlich sind. Beispiele sind Vererbung, Teil-Ganz-Propagation, Abhängigkeitsketten und Erreichbarkeit durch Zitat oder Anforderungsdiagramme.

Entscheidungstabellen sind die zweite nützliche Formalismus für eine grenzenlose Politik. Eine Entscheidungstabelle macht Zustandskombinationen und Ergebnisse explizit, zeigt überlappende oder fehlende Zeilen auf und unterstützt die Abdeckungsanalyse. Es ist besonders geeignet für Berechtigungs-, Routing-, Klassifizierungs- und Richtlinienregeln, deren Komplexität eher aus Kombinationen als aus Rekursion stammt. Eine Schaltung kann eine natürlichsprachliche Richtlinie in einer Kandidatentabelle zusammenstellen, aber die Veröffentlichung sollte Beispiele auf Zeilenebene, Überschneidungserkennung, Vollständigkeitsprüfungen und die menschliche Genehmigung der semantischen Partition erfordern.

SAT-, SMT-, CSP- und MaxSAT-Engines werden nützlich, wenn die Aufgabe interagierende Einschränkungen, Recheneinstellungen, Zeitpläne, Zuweisungen oder Optimierungen enthält. Der Compiler muss unterstützte Theorien auswählen und vermeiden, nicht unterstützte Semantik in nicht interpretierten Strings zu verstecken. Ein SMT-Ergebnis kann sich unter codierten Einschränkungen als Undurchführbarkeit erweisen, ein zufriedenstellendes Modell erstellen oder einen unzufriedenen Kern freilegen. Der Kern sollte auf Quellenverpflichtungen und Annahmen zurückgeführt werden. Dieses Mapping ist informativer, als einen LLM zu bitten, einen generischen Solver-Fehler zu erklären; es identifiziert die kleinsten formalen Verpflichtungen, die nicht gemeinsam gehalten werden können.

Zeitliche Logik- und Modellprüfungen sind für Prozeduren, Workflows, Protokolle und Lebenszyklusregeln geeignet. Die Evidenzgrafik liefert Ereignisse und Zustandsvorschläge; der Compiler konstruiert ein Übergangssystem und eine zeitliche Eigenschaft; der Prüfer gibt Zufriedenheit, eine Gegenbeispielspur oder ein nicht schlüssiges Ergebnis unter Grenzen zurück. Der schwierige Teil ist nicht der Zeitliche Checker. Es entscheidet darüber, ob die Quelle ein komplettes Übergangssystem definiert, ob ausgelassene Übergänge unmöglich oder nur undokumentiert sind und ob Fairness- oder Umweltannahmen gerechtfertigt sind. Das kanonische Ergebnis muss daher die Modellgrenze freilegen.

Symbolische Ausführung ist angemessen, wenn sich eine Regel auf praktikable Pfade durch ausführbaren Code, einen zustandsgemäßen Vorgang oder ein programmähnliches semantisches Artefakt bezieht. Seine Ausgabe sollte ein konkreter Zeuge sein: Aufträge, Zweigbedingungen, Zustandsübergänge und Quellverbindungen. In reinen Dokumenteneinstellungen kann die symbolische Ausführung alternative Bindungen oder Verfahrenszweige untersuchen, aber sie sollte nicht nur deshalb aufgerufen werden, weil das System Symbole enthält. Der Begriff sollte seine pfadsensitive Bedeutung behalten. SymGPT ist lehrreich, da die natürlichsprachliche Regel in eine Verletzungsbedingung zusammengestellt und dann auf tatsächliche Vertragswege getestet wird, anstatt gegen eine stärker generierte Prosa [Xia et al. 2026].

Natürliche Logik, endliche Automaten, Chart-Parsing, Semiring-Rechen und spezialisierte Theorem-Provers gehören in das Portfolio, wenn ihre strukturellen Annahmen passen. Natürliche Logik kann Monotonität und bestimmte lexikalische Nachfolgen in der Nähe von Oberflächensprache effizient verarbeiten. Automata eignen sich hervorragend für bestellte lokale Muster. Chart- und Semiring-Methoden können mehrere Parses mit gemeinsamer Berechnung beibehalten. Beweisassistenten sind geeignet, wenn eine kleine Anzahl von hochwertigen Ansprüchen manuelle Formalisierung und von Kernel überprüfte Proofs begründet. Die Architektur sollte das Hinzufügen einer Engine zu einer typisierten Erweiterung machen, nicht zu einer Einladung, beliebigen ausführbaren Code in Schaltkreise einzubetten.

Die sechste Schicht ist gegenbeispielgeführte semantische Verfeinerung. Ein möglicher Befund wird zu einem Machbarkeitsproblem zusammengestellt. Wenn der Löser einen Zeugen zurückgibt, wird der Laufzeiter wieder auf die Beweisgrafik zurückgespielt und die genauen Interpretationsoptionen gemeldet. Wenn der Pfad nicht infesierbar ist, gibt der Motor einen unzufriedenen Kern, eine fehlgeschlagene Vorbedingung oder eine Konfliktmenge zurück. Das System verfolgt, das durch die Zusammenstellung auf die Quellenbeobachtungen und Kandidatenkartierungen zurückgreift, die es produziert haben. Erst dann wird ein LLM um Hilfe gebeten.

Die Reparaturaufforderung sollte lokal und getippt werden. Sie kann sich fragen, ob zwei Erwähnungen falsch vereinheitlicht wurden, ob eine Bedingung als notwendig oder ausreichend beabsichtigt war, ob eine Ausnahme gilt oder ob eine zeitliche Beziehung den falschen Endpunkt hat. Es sollte die relevanten Quellenspannen, Ontologieverträge, aktuelle Alternativen und den symbolischen Konflikt darstellen. Das LLM schlägt ein oder mehrere überarbeitete semantische Artefakte vor. Diese Vorschläge durchlaufen die gleichen Erdungs- und Zusammenstellungsprüfungen wie die Originale. Ein Mensch kann eingefügt werden, wenn die Mehrdeutigkeit normativ oder wissenschaftlich material ist.

Dieser Prozess unterscheidet sich grundlegend von der generischen Selbstreflexion. Dem Modell wird nicht gesagt, dass seine Antwort falsch war und eingeladen wurde, eine weitere Denkkette zu improvisieren. Es wird ein mechanisch identifizierter Konflikt in einem persistenten semantischen Artefakt gegeben. Die Reparatur kann regressionsgeprüft werden. Wenn die Reparatur eine Ontologie oder einen Produzenten ändert, erfordert die Veröffentlichung neue Benchmark-Fälle und eine semantische Differenzierung. Das Gegenbeispiel wird zum permanenten Bestandteil des Bewertungskorpus.

Die siebte Schicht ist das kanonische Ergebnis. Es sollte ein stabiles, löserunabhängiges Produkt sein, das Urteil, Beweise, Annahmen, Interpretationen, Ausführung und Einschränkungen trennt. Für eine Audit-Regel zeichnet es die Regelversion, den anwendbaren Umfang, den Kompatibilitätsstatus, die Abdeckung, akzeptierte Beobachtungen, abgelehnte Beobachtungen, das Ausführungsergebnis, die Zeugen- oder Beweisreferenz, das Verifier-Ergebnis und ungelöste Probleme auf. Für einen Plan zeichnet es Ziele, Einschränkungen, Schritte, Abhängigkeiten, Ressourcen, Alternativen und die genauen Überprüfungen auf, die der Plan erfüllt. Das kanonische Produkt ist die Autorität für jede nachfolgende natürlichsprachliche Antwort.

Der Rückweg zur Sprache muss als Zusammenstellung behandelt werden, nicht als freie Generation. Ein Renderer erhält einen endlichen Satz von lizenzierten Ansprüchen. Jeder Anspruch hat Modalität, Polarität, Quantität, Entitätsreferenzen, Unsicherheit und Herkunft. Der Renderer kann Diskursordnung und erklärende Formulierungen wählen, aber er kann keine

neuen Entitäten einführen, kann sich verstärken, um fehlende Beweise in Abwesenheit umzuwandeln, eine materielle Annahme zu unterdrücken oder eine kausale Erklärung zu erfinden. Anspruchskennungen auf Sätzezebene sollten für die Prüfung verfügbar bleiben. Ein zweiter Checker sollte den gerenderten Text mit dem kanonischen Ergebnis vergleichen, wobei deterministische Tests für Namen, Größen, Daten, Negation und Modalität verwendet werden und gegebenenfalls ein separates Modell, das eher auf textuelles Einbringen als auf offene Bewertung beschränkt ist.

Die nachweisbeschränkte Realisierung ist besonders wichtig, da die Benutzer das System anhand seiner Prosa beurteilen. Eine perfekte symbolische Spur kann durch eine elegante, aber ungenaue Zusammenfassung untergraben werden. Die Architektur sollte daher Wortschatz optional, aber Treue verpflichtend machen. Im Hochsicherheitsmodus kann das System eine einfache, sich wiederholende Sprache erzeugen, da jeder Satz einer vom Ergebnistyp lizenzierten Vorlage folgt. Im erklärenden Modus kann ein LLM die Lesbarkeit verbessern, jedoch werden nicht unterstützte Sätze abgelehnt und regeneriert. Das kanonische Artefakt, nicht die Prosa, bleibt die Entscheidungsgeschichte.

Sicherheit und Governance müssen von Anfang an integriert werden. Natürliche Sprachdokumente sind nicht vertrauenswürdige Eingaben und können eine sofortige Injektion, kontradiktorische Beispiele, irreführende Formatierung oder Codefragmente enthalten. Semantische Hersteller sollten Quelldaten über typisierte Schnittstellen erhalten, anstatt eine privilegierte Systemaufforderung, die von Dokumentinhalten überschrieben werden kann. Generierte formale Artefakte müssen durch restriktive Grammatiken analysiert werden. Die Solver sollten unter Ressourcengrenzen laufen. Laufzeiterweiterungen sollten überprüft, versioniert und digest-locked werden. Lernarbeitsplätze sollten nicht direkt in der Produktion veröffentlicht werden. Diese Grenzen sind bereits teilweise in nllAgent vorhanden und sollten gestärkt und nicht umgangen werden, um denn [AssistOS-AI 2026].

Ein realistisches Umsetzungsprogramm kann in fünf Veröffentlichungen fortgesetzt werden. Die erste Version sollte unveränderliche Markdown- oder DOCX-Aufnahmen, Quellspannen, strukturelle Beobachtungen, eine relationale Abfrage-Engine, eingeschränkte kontrollierte englische Beobachtungen, Abdeckungszustände, wiederverwendbare Schaltkreise, Wiederholungsprüfung und kanonische Ergebnisse unterstützen. Es sollte begrenzte Aufgaben wie Provenienzzvollständigkeit, Definitionskonsistenz, Anforderungsdeckung, geordnete Abschnittsregeln und einfache Modalunterschiede lösen. Es sollte keine allgemeine Identität, zeitliche Inferenz oder Stromwelt-Abrufung beansprucht werden.

Die zweite Version sollte die eingegebene Identität und Zeit hinzufügen. Identität beginnt mit expliziten Bezeichnern, Aliasen und dokumentenerklärten Rollen; die zeitliche Semantik beginnt mit genauen Daten, Intervallen, Dokumentenreihenfolge und einem kleinen Satz von Ereignisreihenfolgemustern. Benchmarks sollten mehrdeutige Aliase, wechselnde Rollen, überlappende Intervalle und bewusst fehlende Links enthalten. Das System sollte aufhören, anstatt zu erraten, wann eine Schaltung eine stärkere Identität erfordert, als die Freigabe bietet.

Die dritte Version sollte Entscheidungstabellen und SMT/CSP-Ausführung hinzufügen, mit unzufriedener Kernzuordnung und gegen Beispielgesteuerter semantischer Verfeinerung. Diese Pressemitteilung kann sich mit Förderfähigkeitsrichtlinien, Zeitplänen, Zuweisungsbeschränkungen, quantitativer Konsistenz und begrenzter Verfahrensdurchführbarkeit befassen. Jedes Solver-Ergebnis muss, soweit möglich, durch einen unabhängigen Prüfer oder einen kleineren Checker wiedergegeben werden.

Mutationstests sollten Einschränkungen und Beobachtungen ändern, um zu zeigen, dass Benchmark-Suiten semantische Regressionen erkennen.

Die vierte Version sollte einen Temporal/Modell-Checking-Adapter und einen pathsensitiven Symbol-Execution-Adapter für eine konkrete Domäne, wie Workflow-Spezifikationen oder Smart-Contract-Regeln, hinzufügen. Der Zweck ist keine breite Abdeckung, sondern ein Beweis dafür, dass das Formalismusregister und die kanonische Zertifikatsschnittstelle wirklich unterschiedliche Semantik unterstützen. Cross-Formalism-Konsistenztests sollten überprüfen, ob gemeinsame fundierte Fakten Identität, Zeit, Modalität und Herkunft über Senkungen hinweg beibehalten.

Die fünfte Version sollte Planung und kontrollierte Realisierung hinzufügen. Eine Idee wird in einem kanonischen Plan zusammengestellt, mit formalen Einschränkungen überprüft, in Prosa umgesetzt und vom selben Regelsystem geprüft. Die Bewertung muss die Gültigkeit des Plans, die Realisierungstreue, die sprachliche Qualität und die Reparatureffizienz unterscheiden. Der Plan wird nicht vertrauenswürdig, nur weil Prosa aus einem strukturierten Objekt generiert wurde; er wird revisionssicherer, weil das Objekt und seine Überprüfungen persistent sind.

Artifact stack for a replayable executable-language runtime

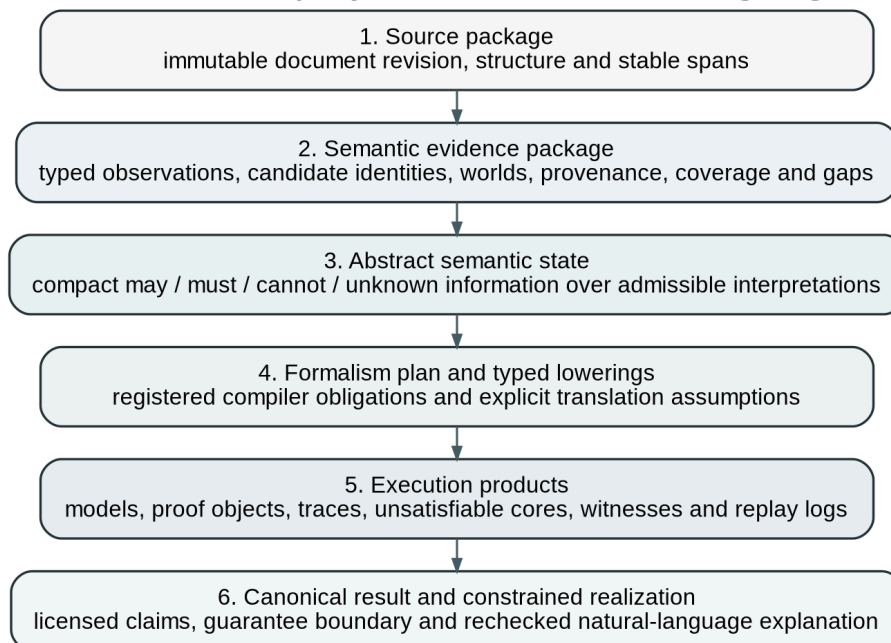


Abbildung 4. Artefakt-Stack für eine wiederholbare ausführbare Sprachlaufzeit. Jede Ebene ist versioniert, quellenzugeordnet und unabhängig inspizierbar.

Diese inszenierte Implementierung verkörpert ein wichtiges methodisches Prinzip: Semantisches Ehrgeiz sollte nur wachsen, wenn das System erkennen kann, dass seine bisherige Darstellung unzureichend ist. Die Laufzeit sollte die nicht unterstützte Sprache hinter dem Vertrauen des Modells nicht verbergen. Es sollte Kompatibilitätslücken als erstklassige Ergebnisse darstellen und wiederkehrende Lücken in neue Produzenten, Ontologie, Motor und Benchmark-Arbeit verwandeln. Ausführbare natürliche Sprache schreitet durch die geregelte Erweiterung einer Soliditätshülle voran, nicht durch einen Sprung zum universellen Verständnis.

8.2 Forschungsfragen, Benchmarks und ein Publikationsprogramm

Die Architektur wird erst dann zu einem wissenschaftlichen Beitrag, wenn ihre Behauptungen widerlegbar sind. Eine Demonstration, bei der ein LLM ein plausibles formales Artefakt erzeugt und ein Löser eine Antwort zurückgibt, ist unzureichend. Das gleiche Modell kann den Benchmark, die Übersetzung und die Erklärung generieren und einen geschlossenen Kreislauf korrelierter Fehler erzeugen. Eine seriöse Bewertung muss die Pipeline zerlegen und jede Verpflichtung unabhängig prüfen. Sie muss auch Misserfolgsbedingungen enthalten, unter denen der vorgeschlagene Ansatz abgelehnt oder verengt werden sollte.

Die erste Forschungsfrage betrifft die Darstellung: Verbessert die Aufrechterhaltung mehrerer zulässiger semantischer Interpretationen die Zuverlässigkeit bei der Auswahl einer Übersetzung? Die entsprechende Hypothese ist, dass die Kandidaten-Set-Semantik, kombiniert mit May/Must-Analyse, die falsche Sicherheit auf mehrdeutige und unterspezifizierte Inputs reduziert. Es kann eine niedrigere scheinbare Antwortrate, weil mehr Fälle bedingt oder unbekannt werden. Das ist nicht unbedingt ein Defekt. Die relevanten Vergleichsmaßnahmen, ob akzeptierte Must-Schlussfolgerungen häufiger richtig sind und ob die Ergebnisse wesentlich plausible Alternativen ohne überwältigende Gutachter erfassen können.

Ein Maßstab für diese Frage sollte Texte mit fachmännisch kommentierten Interpretationssätzen und nicht mit einer goldlogischen Form enthalten. Minimale Paare sollten den Quantifizierungsumfang, die Negation, die Ausnahmebindung, die Koordination, die Koreferenz, die zeitlichen Grenzen, die Modalkraft und die Rechtsprechung variieren. Einige Elemente sollten eine eindeutig beabsichtigte Lesung, einige mehrere legitime Lesungen und einige unzureichende Beweise für eine bestimmte Antwort haben. Die Systeme werden anhand der Erfassung zulässiger Messwerte, des Ausschlusses unzulässiger Messwerte, der Kalibrierung des Mai-/Mussstatus und der nachgelagerten Urteile bewertet. Die exakte Graphenübereinstimmung ist zu spröde; die semantische Äquivalenz sollte über Abfrage-Suiten oder bidirektionales Einbringen unter der Benchmark-Ontologie getestet werden.

Die zweite Forschungsfrage betrifft die Verfeinerung: Sind gelöste Konflikte nützlicher für die Reparatur von Formalisierungen als die Selbstkritik des generischen Modells? Die Hypothese ist, dass Gegenbeispiele, unzufriedene Kerne und fehlgeschlagene Beweisverpflichtungen zu lokalisierten, stabileren und probeneffizienteren Reparaturen führen, weil sie die für das Ergebnis verantwortliche Abhängigkeit identifizieren. Ein kontrolliertes Experiment sollte vier Bedingungen vergleichen: direkte Wiederholung, verbale Selbstreflexion, Exekutionsfehler-Feedback und gegen Beispielgesteuerte semantische Verfeinerung mit Source-Mapping. Das gleiche Modell, die gleiche Temperatur und das gleiche Budget sollten nach Möglichkeit verwendet werden.

Die Reparaturqualität sollte nicht nur daran gemessen werden, ob das Zielbeispiel korrekt wird. Die Reparatur muss nicht zusammenhängende Fälle aufbewahren, die semantische Differenzierung reduzieren und nicht unterstützte Annahmen vermeiden. Mutationsbasierte Regressions-Suiten sind unerlässlich. Eine Reparatur, die ein Beispiel löst, indem sie die Regel weltweit schwächt, ist kein Erfolg. Das Experiment sollte eine Anzahl von Iterationen, Token, menschlichen Eingriffen, überlebenden Mutanten, veränderten Beobachtungen und ob das endgültige Artefakt unabhängig widerspielbar war, aufgezeichnet werden.

Die dritte Forschungsfrage betrifft den Beweisvertrag. Verbessert eine typisierte Trennung von Quellenspannen, Beobachtungen, Ontologie-Asservisationen, Domänenaxiomen und

Weltwissensprämissen den Transfer und die Auditierbarkeit im Vergleich zu einem flachen Wissensgraphen oder einer Freiform-Gedankenkette? Die Hypothese ist, dass typisierte Verträge die Einführung von stillen Prämissen reduzieren und die domänenübergreifende Wiederverwendung vorhersehbarer machen. Die wahrscheinlichen Kosten sind die Verfassung von Overhead und mehr gestoppte Läufe.

Der Benchmark sollte die gleiche Regel enthalten, die für Dokumente aus mehreren Bereichen angewendet wird, mit kontrollierten Variationen in Wortschatz und Ontologie. Zum Beispiel kann eine allgemeine Anforderung, dass jede akzeptierte Behauptung rückverfolgbare Beweise hat, auf Forschungspapieren, Grundsatzdokumenten, technischen Spezifikationen und Plänen getestet werden. Strukturelle Beobachtungen können direkt übertragen werden; domänenspezifische Entitäten und Beziehungshersteller dürfen dies nicht tun. Die Auswertungsaufzeichnungen, welche Kompatibilität aktiviert wird, ob der Stop-Grund korrekt ist und wie viel neues semantisches Material erforderlich ist, um die Ausführung wieder aufzunehmen. Erfolgreiche Übertragung ist nicht nur eine Antwort zu erhalten; sie ist genau zu erkennen, was wiederverwendet werden kann und was nicht.

Die vierte Forschungsfrage betrifft die Wahl des Formalismus. Übertrifft das Routing zu einem Portfolio spezialisierter Motoren eine universelle Zwischensprache unter gleicher semantischer Aufsicht? Beiser und Penz zeigen, dass die Zwischensprachwahl die Gesamt- und Ausführungsgenauigkeit bei neurosymbolischen Argumentationsaufgaben [Beiser and Penz 2025] wesentlich verändern kann. Die Hypothese hier ist spezifischer: Ein typisierter Router, der den einfachsten angemessenen Formalismus auswählt, verbessert den Erfolg der Zusammenstellung, die Erklärungsqualität und die Effizienz, ohne die Kreuzaufgabenkonsistenz zu beeinträchtigen.

Ein aussagekräftiger Vergleich sollte dem Portfoliosystem keine versteckteren Informationen ermöglichen. Jede Aufgabe erhält die gleichen Quellenbeweise und Ontologie. Die Single-Formalism-Baseline kann Logik erster Ordnung, ASP, SMT oder eine allgemeine Graphenabfragesprache verwenden. Das Portfoliosystem kann relationale Abfragen, Datalog, Entscheidungstabellen, SMT, zeitliche Logik oder symbolische Ausführung auswählen. Zu den Kennzahlen gehören die Gültigkeit der Formalisierung, die semantische Äquivalenz, der Erfolg der Solver, die Laufzeit, die Qualität der Zeugen und die Wartungskosten. Formalismus-Kreise sollten zwei Motoren erfordern und testen, ob gemeinsame Symbole konsistent bleiben.

Das Experiment sollte auch auf Gegenbeweise achten. Eine universelle Darstellung kann das Portfolio in engen Domänen übertreffen, da das Routing Fehler verursacht und mehrere Compiler die vertrauenswürdige Basis erhöhen. Die erwartete Schlussfolgerung ist nicht, dass Portfolios immer überlegen sind. Es ist so, dass die Formalismusvielfalt nur dann gerechtfertigt ist, wenn die Aufgabenverteilung wirklich unterschiedliche Rechenstrukturen enthält und wenn Schnittstellen die Semantik bewahren.

Die fünfte Forschungsfrage betrifft die endgültige Sprachschicht. Reduziert die nachweisbeschränkte Realisierung nicht unterstützte oder semantisch gestärkte Ansprüche, während eine akzeptable Lesbarkeit erhalten bleibt? Die Hypothese ist, dass ein kanonisches Ergebnis plus Claim-Lizenzierungsgeneration die Halluzination und Modalitätsdrift relativ zur direkten Antwortzeugung oder Antwortzeugung aus einer rohen Solver-Spur erheblich senkt. Die Kosten können weniger natürliche Prosa und mehr Regeneration sein.

Der Benchmark sollte kanonische Ergebnisse mit subtiler Unsicherheit, widersprüchliche Beweise, verschachtelte Bedingungen, Mengen und Daten enthalten. Menschliche Bewerter

beurteilen sachliche Treue, Vollständigkeit der materiellen Begrenzungen, Lesbarkeit und Nützlichkeit. Deterministische Prüfer messen Entität, Anzahl, Datum, Polarität und Modalitätserhaltung. Ein kontradiktorisches Set sollte den Renderer dazu verleiten, eine stärkere, einfachere Schlussfolgerung zu erzielen, als das Ergebnis unterstützt. Die zentrale Metrik ist keine stilistische Präferenz, sondern die Geschwindigkeit, mit der die Prosa die Autorität des symbolischen Produkts übersteigt.

Die sechste Forschungsfrage betrifft die föderative Architektur selbst. Die systematische Überprüfung des neurosymbolischen NLP im Jahr 2026 berichtet vielversprechende Gewinne für föderative Systeme, warnt jedoch davor, dass die Beweise durch Aufgabenunterschiede [Chatzikyriakidis and Lappin 2026] verwechselt werden. Ein direktes Experiment sollte eine föderative Pipeline mit einer injektiven oder durchgängigen gelernten Alternative auf den gleichen Aufgaben, Daten und Modellskala vergleichen. Die relevanten Vorteile können weniger in der Rohgenauigkeit als bei der Prüfbarkeit, Enthaltung, Fehlerlokalisierung und Reparatur erscheinen.

Die Studie sollte daher mehrere Dimensionen melden. Die Genauigkeit der rohen Aufgaben bleibt notwendig. Es sollte von semantischer Genauigkeit, löserbedingter Genauigkeit, unbekannter Kalibrierung, Reproduzierbarkeit über Modellversionen, Erklärungstreue, menschlicher Überprüfungszeit und dem Bruchteil der Fehler begleitet werden, die auf Extraktion, Erdung, Zusammenstellung, Ausführung oder Realisierung zurückzuführen sind. Ein modulares System, das eher der End-to-End-Genauigkeit entspricht als schlägt, kann für hochsichere Arbeiten immer noch vorzuziehen sein, wenn es leise Fehler in diagnostizierbare Artefakte verwandelt. Umgekehrt sollte die Modularität nicht als intrinsische Tugend verteidigt werden, wenn sie lediglich Bürokratie hinzufügt, ohne die Beweise zu verbessern.

Eine siebte Forschungsfrage betrifft die abstrakte Interpretation über semantische Alternativen. Kann eine nützliche abstrakte Domäne definiert werden, die im Vergleich zu einer Benchmark-Interpretation konservativ und skalierbarer als explizite Aufzählung ist? Die ersten Experimente können endliche Sätze als konkrete Referenz verwenden. Abstrakte Domänen für Identität, zeitliche Intervalle, Modalität, Größen und Abdeckung werden separat implementiert und dann kombiniert. Die Schalligkeit wird mechanisch gegen die Aufzählung auf Kleinkästen überprüft. Die Präzision wird an der Geschwindigkeit der Mai-Ergebnisse gemessen, die durch die Suche oder Verfeinerung von Zeugen beseitigt werden können.

Dieses Programm sollte mit dem Wort Solidness vorsichtig sein. Wenn die Expertenankündigung eine endliche Interpretationsmenge definiert, kann der abstrakte Interpretierer relativ zu dieser Menge solide sein. Es kann dabei nicht beweisen, dass das Set die menschliche Bedeutung erschöpft. Der Beitrag ist eine ausführbare bedingte Semantik und ein Beweis für seine Nützlichkeit. Eine spätere theoretische Arbeit kann die Zulässigkeitsbeziehung, Abstraktions- und Konkretisierungsfunktionen, Transferbedingungen und Verfeinerungsoperatoren definieren. Das empirische System sollte nicht auf eine vollständige Semantik der Sprache warten, sondern die bedingte Natur seiner Behauptungen deutlich machen.

Eine achte Forschungsfrage betrifft die symbolische Ausführung von semantischen Programmen. Verbessert die Pfad-sensible Zeugenkonstruktion das Verständnis und die Fehlererkennung von Prüfern im Vergleich zu statischen Regelergebnissen? In einem Workflow oder Richtliniendomäne kann die Laufzeit einen einfachen Regelauswerter mit der Erforschung symbolischer Pfade vergleichen. Der Bewerter sagt, dass ein Verstoß möglich ist; der Executor

gibt eine konkrete Abfolge von Ereignissen und Aufgaben zurück. Menschliche Gutachter beurteilen, ob der Zeuge verständlich und nützlich ist. Grund-Wahrheits-Defekte und ausgesäte Mutanten bestimmen, ob der Zeuge einem realen Fall entspricht.

Die Pfadexplosion muss Teil der Bewertung sein und nicht als Implementierungsunannehmlichkeiten behandelt werden. Die Studie sollte Verzweigung, Schleifengrenzen, Identitätsalternativen und zeitliche Entscheidungen variieren. Die Anleitung von LLM kann die Pfade priorisieren, wie die jüngsten LLM-gesteuerten symbolischen Ausführungsarbeiten nahelegen, aber die Engine muss die heuristische Abdeckung von der formalen Vollständigkeit unterscheiden. Abdeckungsmetriken sollten untersuchte Zustände, beschnittene Zustände, Lösezeitausfälle und nicht unterstütztes externes Verhalten melden. Ein Zeuge ist ein starker Beweis für die Existenz; das Versäumnis, einen zu finden, ist kein Beweis für Abwesenheit, es sei denn, die Forschung gebunden und das Modell rechtfertigen diese Schlussfolgerung.

Die Benchmark-Architektur sollte geschichtet werden. Die Quellschicht testet die Einnahme und Verankerung. Die Beobachtungsschicht prüft Extraktion und Status. Die Erdungsschicht testet Identität, Zeit, Modalität, Mengen und Autorität. Die Formalisierungsschicht testet die semantische Äquivalenz und die zielsprachliche Validität. Die Ausführungsschicht testet Solver-Ausgänge und Zertifikate. Die Ergebnisschicht testet Urteilszusammensetzung und Unsicherheit. Die Realisierungsschicht testet die Genauigkeit der natürlichen Sprachantworten. Jedes Beispiel sollte die früheste Schicht identifizieren, bei der ein injizierter Fehler erkannt werden sollte.

Benchmark-Fälle sollten klein genug sein, um manuell zu validieren und in längere Dokumente zusammensetzbar zu sein. Kleine Beispiele liefern vertrauenswürdige Bodenwahrheit für subtile Logik. Lange Verbundwerkstoffe testen Abruf, Indexierung, Identitätskontinuität und Skalierbarkeit. Die gleiche zugrunde liegende Regel kann in mehreren sprachlichen Formen und Dokumentstrukturen erscheinen. Paraphrasenfamilien Testinvarianz; Kontrastsätze verändern ein semantisches Merkmal und Testempfindlichkeit. Der Benchmark muss das gemeinsame Versagen vermeiden, bei dem ein System gut funktioniert, indem es Oberflächenmuster auswendig lernt, während die beabsichtigte Unterscheidung ignoriert wird.

Mutationstests sollten von zentraler Bedeutung sein. Mutationen können einen Quantifizierer ändern, eine Bedingung negieren, eine Ausnahme verschieben, eine Datumsgrenze ändern, Identitäten zusammenführen, Beweise löschen, die Modalkraft ändern, Akteur und Objekt tauschen, eine Anforderung duplizieren oder eine Schaltungsschwelle ändern. Eine robuste Suite sollte die erwartete Veränderung erkennen und nicht verwandte Ergebnisse stabil lassen. Mutations-Score ist kein vollständiges Qualitätsmaß, aber es zeigt Tests, die nur einen glücklichen Weg bestätigen. Für nllAgent würden ausführbare Mutationen direkt auf eine der anerkannten Lücken des Repositorys [AssistOS-AI 2026] adressieren.

Die Bewertung sollte eine schnelle Injektion und semantische Gegner umfassen. Dokumente können Anweisungen für das Modell, irreführende Überschriften, zitierten Richtlinien text, kontradiktorischen Unicode, versteckte Inhalte oder Aussagen enthalten, die Ontologie-Deklarationen ähneln. Der Quellparser muss diese als Daten beibehalten. Semantische Hersteller sollten unter fester Systemautorität arbeiten. Der Benchmark sollte testen, ob Dokumenttext die Schaltung verändern, Befunde unterdrücken oder privilegierte Werkzeugaufrufe verursachen kann. Sicherheitsergebnisse sollten getrennt von der semantischen Genauigkeit aufgezeichnet werden.

Die Live-Modell-Evaluierung muss reproduzierbar genug sein, um wissenschaftliche Behauptungen zu stützen. Jeder Lauf sollte den Anbieter, die Modellkennung, das Anforderungsschema, die Temperatur, das Saatgut bei der Unterstützung, die Kontextkonstruktion und die Wiederholungsrichtlinie anheften. Ein- und Ausgänge sollten vorbehaltlich der Datenschutzbeschränkungen erfasst werden. Wiederholte Studien schätzen die Varianz. Deterministische Doppel bleiben für Softwaretests nützlich, können aber keine Übersetzungsqualität feststellen. Mindestens zwei Modellfamilien sollten getestet werden, soweit dies möglich ist, um architektonische Effekte vom Verhalten eines Anbieters zu unterscheiden.

Die menschliche Bewertung ist notwendig, sollte aber eng gestaltet werden. Experten sollten nicht gefragt werden, ob eine Antwort „gut aussieht“. Sie sollten spezifische Verpflichtungen beurteilen: ob die Formalisierung die Quelle bewahrt, ob Identitäten gerechtfertigt sind, ob ein Zeuge machbar ist, ob die Erklärung alle materiellen Annahmen vorgibt und ob eine Unbekannte angemessen zurückgegeben wird. Inter-Rater-Uneinigkeit ist selbst ein Beweis für semantische Zweideutigkeit und sollte nicht ohne Analyse zu einem einzigen Goldlabel gezwungen werden.

Die quantitativen Kernmetriken sollten die Genauigkeit des Quell-Ankers umfassen; Beobachtungsgenauigkeit und -abruf nach Status; Ontologie und Typgültigkeit; Identitäts-Link-Präzision, Rückruf und Kalibrierung; zeitliche und modale Beziehungsgenauigkeit; Candidate-Set-Berichterstattung; kann/muss-Klanglichkeit relativ zu kommentierten Interpretationen; Formalisierungssyntax und semantische Äquivalenz; Solver-Erfolg; Zeugnis-Gegenleistung; Beweisprüfungs-Erfolg. Ein einziger Aggregat-Score würde den Mechanismus verbergen und sollte zweitrangig sein.

Falsche Zusicherung verdient besondere Betonung. Sie misst Fälle, in denen das System eine bestimmte verifizierte Schlussfolgerung darstellt, obwohl die Quelle, die Grundlage oder die Auslegung dies nicht rechtfertigt. Bei hochsicheren Anwendungen kann die Reduzierung der falschen Sicherheit mehr als die Maximierung beantworteter Fragen von Bedeutung sein. Das System sollte eine Gutschrift für einen präzisen gestoppten Zustand erhalten, wenn ein direkter LLM eine Brücke sicher erfindet. Entsprechend unbekannt ist ein positives Ergebnis.

Baselines sollten direkte LLM-Antworten, Chain-of-Thought- oder gleichwertige Argumentationsaufforderung, werkzeuggestützte LLM-Ausführung ohne persistente formale Artefakte, Einzelformalismus-Übersetzung, Candidate-Set-Übersetzung ohne abstrakte Interpretation, symbolische Ausführung ohne semantische Verfeinerung und die gesamte Architektur umfassen. Für die Fallstudie nllAgent kann die aktuelle grenzende Foundation Release als implementierte Basislinie dienen, während vorgeschlagene Identitäts-, Abstract-Semantik- und Multiformalismus-Erweiterungen schrittweise ausgewertet werden. Dadurch wird vermieden, eine vollständige Forschungsvision nur mit schwächeren externen Systemen zu vergleichen.

Das Publikationsprogramm kann um fünf Beiträge statt um einen übergroßen Anspruch organisiert werden. Das erste Papier sollte ein Positions- und Architekturpapier sein, das ausführbare natürliche Sprache, die Vertrauensgrenze, die Formalisierungslücke, die Solidness-Hüllkurve und das Multiformalismus-Design definiert. Sein Beitrag ist konzeptionelle Klarheit und eine Referenzarchitektur, unterstützt durch eine systematische Übersicht und einen kleinen Arbeitsdemonstrator.

Die zweite Arbeit sollte den provenienztragenden semantischen Drehpunkt und die bedingte Solidität formalisieren. Es definiert Quellobjekte, typisierte Beobachtungen, Kandidateninterpretationen, Möglichkeitssemantik und Kompatibilität. Es beweist die Solidität der abstrakten Berechnung in Bezug auf ein endlich zulässiges Interpretationsset und berichtet von Experimenten zu Identität, Zeit, Modalität und Berichterstattung. Das ist der Programmiersprachen- und Formalsemantikkern.

Das dritte Papier sollte eine gegenbeispielgeführte semantische Verfeinerung einführen. Es definiert Zuordnungen von Solver-Konflikten bis hin zu quellengebundenen semantischen Verpflichtungen und vergleicht Verfeinerung mit Selbstreflexion und generischem Ausführungsfeedback. Das stärkste Ergebnis wäre eine geringere falsche Sicherheit, kleinere Reparaturen, eine bessere Regressionserhaltung und eine Verringerung der menschlichen Anstrengung. Selbst ein negatives Ergebnis wäre wertvoll, wenn es identifiziert, welches Solver-Feedback zu niedrig ist, um die semantische Reparatur zu steuern.

Das vierte Papier sollte das Formalismus-Portfolio und die typisierten Motorverträge vorlegen. Es bewertet relationale, Datalog, Entscheidungstabelle, SMT, zeitliche und symbolische Ausführungsadapter auf einem gemeinsamen Artefaktmodell. Das Papier sollte eher die semantische Konservierung und Zertifikate als die Anzahl der Integrationen hervorheben. Ein kleiner Satz von wirklich unterschiedlichen Motoren ist überzeugender als viele flache Verpackungen.

Das fünfte Papier sollte das nllAgent-System und das Evaluierungspapier sein. Es sollte die implementierte Veröffentlichung klar von der breiteren Laufzeit unterscheiden. Es kann unveränderliche Quellenrevisionen, LongTextJS, CircuitJS, bedarfsgesteuerte Beobachtungen, Kompatibilität, Abdeckung, eingeschränkte Ausführung, Lern- und Publikations-Governance, Replay-Verifizierung, kanonische Audit- und Planprodukte und den Ernststellenprozess dokumentieren. Die Bewertung sollte ausführbare Mutation, Live-Modell-Datensätze, Identitätsfälle, die Governance des aktuellen Weltpakets, Planungsbenchmarks und einen unabhängigen oder separat implementierten Prüfer umfassen, soweit dies möglich ist.

Ein sechstes späteres Papier könnte eine nachweisbeschränkte Realisierung untersuchen. Es würde die direkte Erzeugung, das Template-Rendering, das eingeschränkte LLM-Rendering und die Post-hoc-Verifizierung vergleichen. Der zentrale Beitrag wäre eine Behauptungslizenzdarstellung und ein Beweis dafür, dass lesbare Sprache erzeugt werden kann, ohne die Modalität, Unsicherheit oder den sachlichen Inhalt zu ändern. Diese Arbeit verbindet die formale Ausführung mit dem praktischen Bedürfnis nach zugänglicher menschlicher Kommunikation.

Das Projekt sollte explizite Fehlerkriterien definieren. Der Kandidaten-Set-Ansatz sollte überdacht werden, wenn er so viele Alternativen hervorbringt, dass die Ergebnisse möglicherweise uninformativ sind und die Verfeinerung keine Präzision zurückgewinnt. Das Formalismus-Portfolio sollte verengt werden, wenn Routing- und Cross-Formalismus-Bugs die Vorteile der Spezialisierung übersteigen. Die LLM-gestützte Formalisierung sollte auf die Unterstützung durch die Autorisierung beschränkt werden, wenn die semantische Gleichwertigkeit unter realistischen Dokumenten schlecht bleibt. Die nachweisbeschränkte Realisierung sollte auf Vorlagen zurückgreifen, wenn die modellgenerierte Prosa wiederholt ihre Lizenz überschreitet. Die Argumentation der aktuellen Welt sollte außerhalb des vertrauenswürdigen Kerns bleiben, wenn Quellen-, Zeit-, Gerichtsbarkeits- und Konfliktkontrollen nicht durchgesetzt werden können.

Wissenschaftliche Ehrlichkeit erfordert auch einen Vergleich mit einfacheren Alternativen. Viele Dokumentenprüfungen können durch deterministische Parser, reguläre Ausdrücke, Schemavalidierung, Datenbankabfragen oder von Menschen verfasste Regeln gelöst werden. Ein LLM ist gerechtfertigt, wenn Sprachvariation, Ontologie-Mapping oder semantischer Vorschlag die Erstellungskosten erheblich reduzieren oder die Abdeckung erhöhen. Ein symbolischer Kern ist gerechtfertigt, wenn genaue Staat, Wiederholung, Zeugen oder unabhängige Überprüfungsangelegenheiten. Eine Forschungsarbeit sollte berichten, wann ein einfacheres System gleichermaßen gut funktioniert. Ausführbare natürliche Sprache ist kein Grund, jede redaktionelle Präferenz zu formalisieren oder einen Solver in Aufgaben einzufügen, die nur einen zuverlässigen Parser benötigen.

Das wahrscheinliche praktische Ergebnis ist keine Maschine, die alle natürliche Sprache ausführt. Es ist eine Familie von geregelten semantischen Compilern für Dokument- und Agentendomänen, Freigabequellen, Erdung, Ausführung und Zertifikatinfrastruktur. Jeder Compiler hat einen deklarierten semantischen Umschlag. LLMs machen diese Compiler einfacher zu autorisieren, anzupassen und zu erklären. Formale Methoden machen ihre akzeptierten Ergebnisse reproduzierbar und anfechtbar. Die Kombination ist gerade deshalb sinnvoll, weil keines der beiden Komponenten gefragt wird, was es nicht zuverlässig alleine liefern kann.

Das Forschungsprogramm ist daher sowohl bescheidener als auch folgenreicher als der Slogan „natürliche Sprache als Code“. Natürliche Sprache bleibt ein offenes, soziales, kontextabhängiges Medium. Die Laufzeit erstellt ausführbare Projektionen davon für deklarierte Fragen. Diese Projektionen können streng genug sein, um echte Audits, Pläne, Protokolle und wissenschaftliche Überprüfungen zu unterstützen, vorausgesetzt, das System zeichnet die Entfernung zwischen Quelle und formalem Objekt auf, anstatt so zu tun, als wäre die Entfernung verschwunden.

Schlussfolgerung: von probabilistischer Form zu symbolischer Konsequenz

Die jüngste Wiederbelebung des neurosymbolischen AI hat ein erkennbares architektonisches Muster ergeben. Große Sprachmodelle werden dort eingesetzt, wo der Suchraum sprachlich, heterogen oder schwer aufzuzählen ist. Formale Systeme werden verwendet, wenn genau Zustand, Konsistenz, erschöpfende begrenzte Suche, Zeugen oder Beweise wichtig sind. Die interessante Frage ist nicht mehr, ob neuronale und symbolische Methoden kombiniert werden können. Das können sie deutlich. Die wichtige Frage ist, wo Autorität liegt und was unabhängig geprüft wird.

Die Umfrage zeigt, dass die glaubwürdigsten Systeme dem Sprachmodell keine endgültige Autorität über das Ergebnis gewähren. Kontrollierte Sprachen übersetzen eine eingeschränkte Oberfläche in formale Semantik. Semantische Parser generieren ausführbare Abfragen oder Programme. PAL delegiert Berechnung zu Python. SatLM delegiert Einschränkungen an einen Solver. Logic-LM und LINC delegieren Deduktion auf logische Motoren. NL2TL und nl2spec übersetzen Anforderungen in zeitliche Vorgaben. Planungssysteme verwenden formale Modelle und Verifizierungstools, um die Konsistenz des Long-Horizonts zu erhalten. SymGPT kompiliert natursprachliche ERC-Regeln in Einschränkungen und verwendet symbolische Ausführung, um konkrete Vertragspfade zu konstruieren. SAIL verwendet LLMs, um nach abstrakten Transformatoren zu suchen, gibt aber nur Kandidaten zu, die mechanische Verpflichtungen erfüllen. Diese Systeme unterscheiden sich stark, aber sie konvergieren nach dem Prinzip, das die Erzeugung vorschlägt, und ein unabhängiger Mechanismus entscheidet.

Diese Konvergenz sollte nicht überbewertet werden. Die Verwendung von Solver überprüft nicht das Verständnis. Die Formalisierungslücke bleibt das zentrale Problem. Ein Theorem-Prover kann aus Prädikaten, die den Text falsch darstellen, eine gültige Schlussfolgerung ziehen. Ein Programm kann die erwartete Antwort über einen falschen Weg zurückgeben. Ein SMT-Modell kann Einschränkungen erfüllen, die die entscheidende Anforderung des Benutzers auslassen. Ein symbolischer Testamentsvollstrecker kann einen echten Softwarepfad für eine falsch übersetzte Regel freilegen. Die stärksten aktuellen Systeme reduzieren deduktive und computergestützte Fehler und verlagern gleichzeitig das Risiko für semantisches Parsen, Erdung, Ontologie und Umfang.

Die richtige Antwort besteht nicht darin, formelle Werkzeuge aufzugeben, da die Übersetzung unvollkommen ist. Es geht darum, die Übersetzung zu einem persistenten, typisierten, überprüfbaren Artefakt zu machen. Die Quelle muss verfügbar bleiben. Jede semantische Beobachtung muss ihren Produzenten, Beweise, Status und Ontologie identifizieren. Identität, Zeit, Modalität, Größen und Autorität müssen bei jeder Antwort dargestellt und nicht neu abgeleitet werden. Alternative Interpretationen müssen Alternativen bleiben, wenn die Unterscheidung das Ergebnis beeinflusst. Eine symbolische Engine sollte nur geerdete Objekte ausführen, die explizite Verträge erfüllen.

Die abstrakte Interpretation bietet eine nützliche Theorie für die globale Seite dieser Architektur. Sie fragt, was konservativ aus einer Vielzahl möglicher Staaten geschlossen werden kann. Angewandt auf die natürliche Sprache sind die konkreten Zustände keine undefinierten „Bedeutungen“, sondern begrenzte semantische Strukturen, die unter einer erklärten Grammatik-, Ontologie-, Quellrevisions- und Erdungspolitik zulässig sind. Ein

abstrakter Zustand kann dann darstellen, was halten muss, kann halten, nicht halten, Konflikte oder unbekannt bleiben. Die Garantie ist vom Interpretationssatz abhängig, aber bedingte Solidität ist immer noch sinnvoll, wenn die Bedingungen explizit und geprüft sind.

Die symbolische Ausführung bietet die lokale Ergänzung. Wenn eine abstrakte Analyse einen möglichen Verstoß meldet, kann eine symbolische Ausführung oder eine andere Constraint-Engine nach einer konkreten Interpretation und einem Pfad suchen. Eine erfolgreiche Suche gibt einen Zeugen zurück, den ein Rezensent inspizieren kann. Eine fehlgeschlagene Suche zeigt eine übermäßig grobe Abstraktion oder eine inkompatible semantische Wahl. Das Kartieren dieses Versagens auf die Quellenspanne erzeugt eine beispielgesteuerte semantische Verfeinerung: Das Modell oder der Mensch überarbeitet das kleinste relevante formale Engagement, und die Analyse wird wiederholt. Dies ist eine diszipliniertere Verwendung von LLM Feedback als generische Selbstkritik, da das Fehlersignal an eine formale Abhängigkeit gebunden ist.

Keine einzelne Zielsprache wird wahrscheinlich jede nützliche Form von ausführbarer Prosa unterstützen. Die Literatur zeigt bereits die Abhängigkeit von der Zwischensprachwahl, und zeitgenössische Systeme gelingen mit Python, SQL, Logik erster Ordnung, zeitlicher Logik, PDDL, Zentrainsprachen oder domänenspezifischen Grammatiken nach der Aufgabe. Eine praktische Laufzeit sollte daher einen provenienzhaltigen semantischen Drehpunkt und ein typisiertes Portfolio an formalen Motoren verwenden. Relationale und Datalog-Auswertungshandle-Dokumentabfragen und -schließung; Entscheidungstabellen behandeln endliche Richtlinien; SAT/SMT-Handling-interaktionsbeschränkungen; Modellprüfung behandelt zeitliche Verhaltensweisen; symbolische Ausführung behandelt Pfadzeugen; automatisierte geordnete Muster; natürliche Logik behandelt ausgewählte Includements; Beweisassistenten behandeln hochwertige formelle Ansprüche.

Pluralität bringt eigene technische Risiken ein. Jeder Compiler kann die Semantik verzerren, jede Engine erweitert die vertrauenswürdige Basis und das Routing kann fehlschlagen. Das Portfolio ist nur gerechtfertigt, wenn Schnittstellen typisiert werden, gemeinsame Symbole Identität und Herkunft behalten, Motor Garantien explizit sind und die Ergebnisse zu einem kanonischen zertifikattragenden Produkt normalisiert werden. Eine universelle Logik kann in einem schmalen Bereich vorzuziehen bleiben. Der architektonische Anspruch ist nicht, dass Pluralität immer besser ist; es ist, dass eine breite ausführbare Sprache unterschiedliche Rechenstrukturen erfordert und sie nicht hinter einem vagen DSL verkleiden sollte.

Die Rückkehr von Symbolen zur Sprache ist Teil der Vertrauensgrenze. Ein endgültiges LLM darf ein bedingtes Ergebnis nicht in eine bedingungslose Empfehlung, eine mögliche Verletzung in einen bestätigten Verstoß oder fehlende Beweise in Abwesenheitsbeweise verwandeln. Das kanonische Ergebnis sollte eine endliche Reihe von lizenzierten Ansprüchen mit Modalität, Mengen, Einheiten, Annahmen und Herkunft definieren. Der Renderer kann die Lesbarkeit verbessern, sollte aber dieser Autorität untergeordnet bleiben. Die nachweisbeschränkte Realisierung ist die natürliche Vollendung der Architektur.

Die öffentliche nllAgent-Implementierung ist von Bedeutung, da sie bereits mehrere dieser Disziplinen verkörpert. LongTextJS bewahrt ein quellenbasiertes Dokumentprogramm; CircuitJS stellt wiederverwendbare Theorien dar; Beobachtungsbedarf wird aus Schaltkreisen abgeleitet; Kompatibilität und Abdeckung sind explizit; eingeschränkte Module und überprüfte Erweiterungen begrenzen die versteckte Ausführung; Lernen findet in der Einweg-Staging statt; Veröffentlichung ist manuell und unveränderlich; Replay-Verifikationskontrollen kanonische

Ergebnisse; und fehlendes Wissen wird nicht als Compliance behandelt. Diese Entscheidungen machen das Projekt zu einer glaubwürdigen Plattform für die Forschung, obwohl sie kein allgemeines semantisches Verständnis herstellen.

Die gleichen öffentlichen Materialien identifizieren die wichtigen unfertigen Arbeiten: allgemeine Identitätsmaterialisierung, vollständige typisierte primitive Verträge, ausführbare semantische Mutation, unabhängige Schattenbewertung, Live-Modell-Benchmarking, Planungs- und Realisierungs-Benchmarks und geregelte Wissenspakete der aktuellen Welt. Für diese Ausgabe wurde keine unabhängige Peer-Review-Bewertung von nllAgent gefunden. Das hier vorgebrachte Urteil ist daher ein architektonisches: Die Umsetzung hat ungewöhnlich solide Grenzen gewählt, aber ihre stärksten vertretbaren Ansprüche bleiben strukturelle und kontrolliersprachige Analysen gebunden. Sein Wert wird steigen, wenn die semantische Breite nur durch explizite Produzenten, Verträge, Benchmarks und Beweise erweitert wird.

Der vorgeschlagene nächste Schritt besteht nicht darin, LongTextJS oder CircuitJS durch eine größere Universalsprache zu ersetzen. LongTextJS sollte als endliche, abfragbare, provenienztragende semantische Welt reifen. CircuitJS sollte eine eingeschränkte Orchestrierung und eine Evidenzauftragsschicht bleiben. Spezialisierte registrierte Engines sollten eine relationale, eingeschränkte, zeitliche, Datenfluss-, Automatisierungs- oder pfadsensitive Ausführung durchführen. Die May/Must-Semantik sollte in Betrieb genommen werden. Gegenbeispiele sollten fokussierte semantische Verfeinerung vorantreiben. Die kanonischen CNL-Produkte sollten die endgültige Sprachausgabe steuern.

Ein strenges Forschungsprogramm kann jeden Teil testen. Die Candidate-Set-Semantik kann mit Single-Translation-Systemen verglichen werden. Die gelöste Verfeinerung kann mit der Selbstreflexion verglichen werden. Typisierte Beweisverträge können auf Übertragung und falsche Zusicherung getestet werden. Ein Formalismus-Portfolio kann mit einer universellen Zwischensprache verglichen werden. Abstrakte Domänen können mit expliziten Interpretationssätzen überprüft werden. Symbolische Zeugen können auf Fehlerfindung und Beurteilungskraft bewertet werden. Eine nachweisbeschränkte Realisierung kann auf Treue geprüft werden. Modulare und End-to-End-Architekturen können bei den gleichen Aufgaben und nicht bei verschiedenen Benchmark-Familien verglichen werden.

Die entscheidende Kennzahl sollte falsche Sicherheit sein, nicht nur die Antwortrate. Ein System, das „unbekannt zurückgibt, weil Identität ungelöst ist“, kann wertvoller sein als eines, das ein selbstbewusstes Urteil hervorbringt, indem es zwei Menschen stillschweigend zusammenführt. Ein System, das aufhört, weil Beweise der aktuellen Welt abgestanden sind, ist vertrauenswürdiger als eines, das eine nicht unterstützte Tatsache abrufen. Ein System, das drei Interpretationen aufdeckt, ist besser als eines, das die fließendsten auswählt. Entsprechende Enthaltung ist Teil der Hinrichtungssemantik.

Ausführbare natürliche Sprache sollte daher als eine geregelte Zusammenstellungsbeziehung verstanden werden, nicht als intrinsische Eigenschaft der Prosa. Der Quelltext bleibt reicher als jede ausführbare Projektion. Das System wählt eine Aufgabe aus, materialisiert relevante Semantik, bewahrt Unsicherheit, Grundsymbole, kompiliert zu einem geeigneten Formalismus, führt unter expliziten Grenzen aus, überprüft das Ergebnis und gibt eine eingeschränkte Erklärung zurück. Was ausführbar wird, ist nicht die Sprache in ihrer Gesamtheit, sondern eine erklärte semantische Projektion, deren Annahmen und Grenzen sichtbar bleiben.

Diese schmalere Konzeption ist ausreichend für eine wesentliche praktische Wirkung. Wissenschaftliche Angaben können auf Herkunft und Konsistenz überprüft werden. Anforderungen können mit Tests und Lebenszykluszuständen verknüpft werden. Richtlinien können in Entscheidungsstrukturen zusammengestellt werden. Pläne können vor der Realisierung überprüft werden. Protokollregeln können auf Zustands- und Pfadanalysen abgebildet werden. Lange Dokumente können durch wiederverwendbare Theorien gesäumt werden, ohne ein Modell zu bitten, sich an jedes Detail zu erinnern. Agenten können die natürliche Sprache als Autoren- und Koordinationsschicht verwenden und gleichzeitig die genaue Konsequenz auf symbolische Laufzeiten delegieren.

Das zentrale Prinzip kann ohne formale Symbolik angegeben werden: Lassen Sie den LLM eine mögliche Form entdecken; lassen Sie den Erdungskern entscheiden, was die Symbole bezeichnen dürfen; lassen Sie formale Engines bestimmen, was folgt; und lassen Sie den endgültigen LLM nur erklären, was die symbolischen Datensatzlizenzen. Dieses Prinzip löst nicht jedes Problem der Sprache. Sie schafft eine vertretbare Grenze zwischen Interpretation und Konsequenz, die die notwendige Grundlage für die natürliche Sprache ist, um sicher an ausführbaren Systemen teilzunehmen.

Literatur

- Abzianidze, Lasha, Rik van Noord, Hessel Haagsma und Johan Bos. 2019. „Die erste gemeinsame Aufgabe zur Diskursrepräsentationsstruktur Parsing und Bewertung“. Proceedings der IWCS Shared Task on Semantic Parsing. Verein für Computerlinguistik. <https://aclanthology.org/W19-1201/>
- Abzianidze, Lasha. 2017. „LangPro: Natürlicher Satzsatzprüfer“. Proceedings der 2017 Konferenz über empirische Methoden in der Verarbeitung natürlicher Sprache: Systemdemonstrationen. Verein für Computerlinguistik. <https://aclanthology.org/D17-2022/>
- Angeli, Gabor und Christopher D. Manning. 2014. „NaturalLI: Natürliche Logik-Inferenz für Common Sense Reasoning“. Tagungsband von EMNLP 2014. Verein für Computerlinguistik. <https://aclanthology.org/D14-1209/>
- AssistOS-AI. 2026. „NaturalLanguageLinterAgent (nllAgent): LongTextJS–CircuitJS Architektur, Dokumentation und Ernste Probleme“. Öffentliches Repository und generierte Dokumentation, geprüft 2 August 2026. <https://github.com/assistos-ai/nllAgent>
- Badreddine, Samy, Artur d'Avila Garcez, Luciano Serafini und Michael Spranger. 2022. „Logic Tensor Netzwerke“. Künstliche Intelligenz 303: 103649. <https://doi.org/10.1016/j.artint.2021.103649>
- Baldoni, Roberto, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu und Irene Finocchi. 2018. „Eine Übersicht über symbolische Ausführungstechniken“. ACM Computer-Umfragen 51(3): 1–39. <https://doi.org/10.1145/3182657>
- Beg, Arshad, Diarmuid P. O'Donoghue und Rosemary Monahan. 2025. „Eine kurze Umfrage zur Formalisierung von Softwareanforderungen mit großen Sprachmodellen“. arXiv:2506.11874. <https://doi.org/10.48550/arXiv.2506.11874>
- Beiser, Alexander und David Penz. 2025. „LLMs Grund? Das Intermediate-Sprachproblem in neurosymbolischen Ansätzen“. arXiv:2502.17216. <https://arxiv.org/abs/2502.17216>
- Bos, Johan. 2008. „Weitversicherte semantische Analyse mit Boxer“. Proceedings der 2008 Konferenz über Semantik in der Textverarbeitung. <https://aclanthology.org/W08-2222/>
- Chatzikyriakidis, Stergios und Shalom Lappin. 2026. „Neuro-Symbolische NLP: Taxonomie, Bewertung und Richtungen“. Grenzen in Künstlicher Intelligenz 9. <https://doi.org/10.3389/frai.2026.1797587>
- Chen, Lingfeng, Tao Xiao, Masanari Kondo und Yasutaka Kamei. 2026. „Gezielte symbolische Ausführung für die Schwachstellenerkennung: Ein LLM-gesteuerter Ansatz in KLEE“. arXiv:2607.21676. <https://arxiv.org/abs/2607.21676>
- Chen, Yongchao, Rujul Gandhi, Yang Zhang und Chuchu Fan. 2023. „NL2TL: Umwandlung natürlicher Sprachen in temporäre Logiken mit großen Sprachmodellen“. Proceedings der 2023 Konferenz über empirische Methoden in der Verarbeitung natürlicher Sprache. Verein für Computerlinguistik. <https://aclanthology.org/2023.emnlp-main.985/>
- Clarke, Edmund, Orna Grumberg, Somesh Jha, Yuan Lu und Helmut Veith. 2000. „Gegenbeispielgeführte Abstraktionsverfeinerung“. Computergestützte Verifizierung, LNCS 1855, 154–169. https://doi.org/10.1007/10722167_15
- Cosler, Matthias, Christopher Hahn, Daniel Mendoza, Frederik Schmitt und Caroline Trippel. 2023. „nl2spec: Interaktive Übersetzung unstrukturierter natürlicher Sprache in temporäre Logiken mit großen Sprachmodellen“. Computergestützte Verifizierung, LNCS 13964, 383–396. https://doi.org/10.1007/978-3-031-37703-7_18
- Cousot, Patrick und Radhia Cousot. 1977. „Zusammenfassung der Interpretation: Ein einheitliches Gittermodell für die statische Analyse von Programmen durch Konstruktion oder Annäherung an Fixpunkte“. Tagungsband von POPL 1977, 238–252. <https://doi.org/10.1145/512950.512973>
- Delvecchio, Giovanni Pio, Lorenzo Molfetta und Gianluca Moro. 2025. „Neurosymbolische Künstliche Intelligenz: Eine aufgabengesteuerte Umfrage in der Black-Box-Modelle-Ära“. Tagungsband von IJCAI 2025, Survey Track, 10418–10426. <https://doi.org/10.24963/ijcai.2025/1157>
- Dong, Wenkai und Yifan Wang. 2026. „Von der Interpretation zur Zusammenstellung: Zusammenstellungsbasierte Ausführung von semantischen Operatoren“. arXiv:2607.13407. <https://doi.org/10.48550/arXiv.2607.13407>

- Ferrari, Alessio und Paola Spoletini. 2025. „Formal Requirements Engineering und Large Language Models: Eine Zwei-Wege-Roadmap“. Informations- und Softwaretechnik 181: 107697. <https://doi.org/10.1016/j.infsof.2025.107697>
- Fuchs, Norbert E. und Rolf Schwitter. 1996. „Attempto Controlled English (ACE) (englisch)“. Proceedings of the First International Workshop on Controlled Language Applications. <https://attempto.ifi.uzh.ch/site/pubs/papers/ace96.pdf>
- Fuchs, Norbert E., Kaarel Kaljurand und Tobias Kuhn. 2008. „Attempto Controlled English für Wissensdarstellung“. Reasoning Web, LNCS 5224, 104–124. https://doi.org/10.1007/978-3-540-85658-0_3
- Gao, Luyu, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan und Graham Neubig. 2022. „PAL: Programmgestützte Sprachmodelle“. arXiv:2211.10435. <https://arxiv.org/abs/2211.10435>
- Goldman, Omer, Veronica Latcinnik, Udi Naveh, Amir Globerson und Jonathan Berant. 2018. „Schwach überwacht semantisches Parsing mit abstrakten Beispielen“. Tagungsband von ACL 2018. Verein für Computerlinguistik. <https://aclanthology.org/P18-1168/>
- Grimaldi, Pasquale. 2026. „Pamaldi bei SemEval-2026 Task 11: Neuro-Symbolisches syllogistisches Denken über LLM-geführte Strukturextraktion und deterministische Validierung“. Proceedings des 20th International Workshop on Semantic Evaluation. Verein für Computerlinguistik. <https://doi.org/10.18653/v1/2026.semeval-1.186>
- Gu, Qiuhan, Avaljot Singh und Gagandeep Singh. 2026. „SAIL: Klangabstrakte Interpreter mit LLMs“. Tagungsband des ACM zu Programmiersprachen 10 (PLDI): 1534–1559. <https://doi.org/10.1145/3808308>
- Guu, Kelvin, Panupong Pasupat, Evan Liu und Percy Liang. 2017. „Von der Sprache zu den Programmen: Überbrückung von Verstärkungslernen und maximaler marginaler Wahrscheinlichkeit“. Tagungsband von ACL 2017. Verein für Computerlinguistik. <https://aclanthology.org/P17-1016/>
- Hamilton, Kyle, Aparna Nayak, Bojan Božić und Luca Longo. 2024. „Erfüllt Neurosymbolische AI seine Versprechen in der Verarbeitung natürlicher Sprache? Eine strukturierte Überprüfung“. Semantisches Web 15(4): 1265–1306. <https://doi.org/10.3233/SW-223228>
- Han, Simeng, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Kryściński, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying und Dragomir Radev. 2024. „FOLIO: Natürliche Sprache mit Logik erster Ordnung“. Tagungsband von EMNLP 2024. Verein für Computerlinguistik. <https://aclanthology.org/2024.emnlp-main.1229/>
- Hao, Yilun, Yongchao Chen, Yang Zhang und Chuchu Fan. 2025. „Große Sprachmodelle können die reale Planung mit formalen Verifizierungstools streng lösen“. Tagungsband von NAACL 2025, 3434–3483. <https://aclanthology.org/2025.naacl-long.176/>
- Hu, Ruikang, Shaoyu Lin, Yeliang Xiu und Yongmei Liu. 2025. „LTRAG: Verbesserung der Autoformalisierung und Selbstveredelung für logisches Denken mit dem Gedankengesteuerten RAG“. Ergebnisse von ACL 2025, 2483–2493. <https://aclanthology.org/2025.findings-acl.126/>
- Huang, Pei, Jiayu Li, Yuhao Zhang, Changliu Liu und Shih-hsi Alex Liu. 2026. „Parametrisierte abstrakte Interpretation für die Transformatorverifikation“. Tagungsband von AAAI 2026. <https://doi.org/10.1609/aaai.v40i42.40860>
- Jia, Robin, Aditi Raghunathan, Kerem Göksel und Percy Liang. 2019. „Zertifizierte Robustheit gegenüber widrigen Wortsatzstoffen“. Tagungsband von EMNLP-IJCNLP 2019. Verein für Computerlinguistik. <https://aclanthology.org/D19-1423/>
- Karthikeyan, T, Om Dehlan, Mausam und Manish Gupta. 2025. „LRPLAN: Eine Multi-Agent-Kooperation von großen Sprach- und Argumenting-Modellen für die Planung mit impliziten und expliziten Einschränkungen“. Ergebnisse von EMNLP 2025, 8280–8310. <https://doi.org/10.18653/v1/2025.findings-emnlp.440>
- Kartáč, Ivan, Kristýna Onderková, Jan Bronec, Zdeněk Kasner, Mateusz Lango und Ondřej Dušek. 2026. „UFAL-CUNI bei SemEval-2026 Task 11: Eine Effiziente Modulare Neuro-Symbolische Methode für die syllogistische Argumentation“. Proceedings des 20th International Workshop on Semantic Evaluation. Verein für Computerlinguistik. <https://aclanthology.org/2026.semeval-1.418/>
- King, James C. 1976. „Symbolische Ausführung und Programmprüfung“. Kommunikation des ACM 19(7): 385–394. <https://doi.org/10.1145/360248.360252>

- Kirtania, Shashank, Priyanshu Gupta und Arjun Radhakrishna. 2024. „LOGIC-LM++: Mehrstufige Verfeinerung für symbolische Formulierungen“. Proceedings of the 2nd Workshop on Natural Language Reasoning and Structured Explanations, 56–63. <https://aclanthology.org/2024.nlrse-1.6/>
- Kuhn, Tobias. 2014. „Eine Übersicht und Klassifizierung von kontrollierten natürlichen Sprachen“. Computerlinguistik 40(1): 121–170. https://doi.org/10.1162/COLI_a_00168
- Lalwani, Vivek, et al. 2025. „Automatische Formulierung der natürlichen Sprache zur Logik erster Ordnung: Eine Fallstudie zur logischen Fehlererkennung“. Ergebnisse von IJCNLP-AACL 2025, 132–147. <https://aclanthology.org/2025.findings-ijcnlp.8/>
- Lee, Kang-il, Segwang Kim und Kyomin Jung. 2023. „Schwach überwachtes semantisches Parsing mit exekutionsbasierter Spurious-Programm-Filterung“. Tagungsband von EMNLP 2023, 6884–6894. <https://aclanthology.org/2023.emnlp-main.425/>
- Li, Yihe, Ruijie Meng und Gregory J. Duck. 2025. „Großsprachige Modellgestützte Symbolische Ausführung“. Proceedings des ACM on Programmiersprachen 9 (OOPSLA2), Article 385, 3148–3176. <https://doi.org/10.1145/3763163>
- Liang, Chen, Jonathan Berant, Quoc Le, Kenneth D. Forbus und Ni Lao. 2017. „Neuronale Symbolmaschinen: Semantische Parser auf Freebase mit schwacher Aufsicht lernen“. Tagungsband von ACL 2017. Verein für Computerlinguistik. <https://aclanthology.org/P17-1003/>
- Liu, Jiangming, Shay B. Cohen und Mirella Lapata. 2018. „Diskurs-Repräsentationsstruktur Parsing“. Tagungsband von ACL 2018. Verein für Computerlinguistik. <https://aclanthology.org/P18-1040/>
- Locascio, Nicholas, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman und Regina Barzilay. 2018. „Neuronale Erzeugung von regulären Ausdrücken aus natürlicher Sprache mit minimalem Domänenwissen“. Tagungsband von EMNLP 2018. Verein für Computerlinguistik. <https://aclanthology.org/D18-1436/>
- MacCartney, Bill und Christopher D. Manning. 2007. „Natürliche Logik für Textinferenz“. Proceedings des ACL-PASCAL Workshops über Textual Entailment und Paraphrasing. <https://aclanthology.org/W07-1401/>
- MacCartney, Bill und Christopher D. Manning. 2009. „Ein erweitertes Modell der natürlichen Logik“. Tagungsband von IWCS 2009. <https://aclanthology.org/W09-3714/>
- Manhaeve, Robin, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester und Luc De Raedt. 2018. „DeepProbLog: Neuronale Probabilistische Logikprogrammierung“. Fortschritte in neuronalen Informationsverarbeitungssystemen 31. <https://proceedings.neurips.cc/paper/2018/hash/dc5d637ed5e62c36ecb73b654b05ba2a-Abstract.html>
- Mao, Ziyu, Jingyi Wang, Jun Sun, Shengchao Qin und Jiawen Xiong. 2025. „LLM-gestützte automatische Modellierung für die Sicherheitsprotokollverifizierung“. Proceedings der 47. IEEE/ACM International Conference on Software Engineering, 2025. https://ink.library.smu.edu.sg/sis_research/10297/
- Mitchell, Jacqueline L., Brian Hyeongseok Kim, Chenyu Zhou und Chao Wang. 2025. „Kann LLMs formal als abstrakte Interpreter für die Programmanalyse argumentieren?“. arXiv:2503.12686. <https://arxiv.org/abs/2503.12686>
- Mou, Lili, Zhengdong Lu, Hang Li und Zhi Jin. 2017. „Kopplung verteilter und symbolischer Ausführung für natürliche Sprachabfragen“. Tagungsband von ICML 2017. <https://proceedings.mlr.press/v70/mou17a.html>
- Olausson, Theo X., Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum und Roger Levy. 2023. „LINC: Ein neurosymbolischer Ansatz für logisches Denken durch Kombination von Sprachmodellen mit Logikprüfern erster Ordnung“. Tagungsband von EMNLP 2023. Verein für Computerlinguistik. <https://aclanthology.org/2023.emnlp-main.313/>
- Pan, Liangming, Alon Albalak, Xinyi Wang und William Yang Wang. 2023. „Logic-LM: Ermächtigung von großen Sprachmodellen mit symbolischen Lösern für originalgetreues logisches Denken“. Ergebnisse von EMNLP 2023. Verein für Computerlinguistik. <https://aclanthology.org/2023.findings-emnlp.248/>
- Pasupat, Panupong und Percy Liang. 2016. „Ableiten von logischen Formen aus Denotationen“. Tagungsband von ACL 2016. Verein für Computerlinguistik. <https://aclanthology.org/P16-1004/>
- Pei, Yu, Yongping Du und Xingnan Jin. 2025. „FoVer: Logikverifizierung erster Ordnung für natürliche Sprachvernichtung“. Transaktionen der Association for Computational Linguistics 13: 1340–1359. <https://aclanthology.org/2025.tacl-1.61/>

- Tafjord, Oyvind, Bhavana Dalvi Mishra und Peter Clark. 2021. „ProofWriter: Generierung von Implikationen, Beweisen und abduktiven Aussagen über natürliche Sprache“. Ergebnisse von ACL-IJCNLP 2021. Verein für Computerlinguistik. <https://aclanthology.org/2021.findings-acl.317/>
- Tantakoun, Nat, Christian Muise und Yilun Zhu. 2025. „Große Sprachmodelle als Planungsformalisatoren: Eine Übersicht“. Ergebnisse von ACL 2025. <https://aclanthology.org/2025.findings-acl.1291/>
- Tziafas, Georgios und Hamidreza Kasaei. 2026. „Neurosymbolische Aufgabenplanung und Objektmanipulation mit großen Sprachmodellen“. Autonome Roboter 50. <https://doi.org/10.1007/s10514-026-10230-6>
- Wang, Bailin, Richard Shin, Xiaodong Liu, Oleksandr Polozov und Matthew Richardson. 2018. „Ausführungsgesteuerte neuronale Programm-Dekodierung“. Proceedings of the First Workshop on Learned Program Synthesis. <https://arxiv.org/abs/1807.03100>
- Wang, Boxin, Chejian Xu, Xiangyu Liu, Yu Cheng und Bo Li. 2023. „Robustheitsbewusste Worteinbettung verbessert die zertifizierte Robustheit für schädliche Wortersetzungen“. Ergebnisse von ACL 2023. Verein für Computerlinguistik. <https://aclanthology.org/2023.findings-acl.620/>
- Wang, Jiayimei, Tao Ni, Wei-Bin Lee und Qingchuan Zhao. 2025. „Eine zeitgenössische Übersicht über die Analyse des großen Sprachmodells“. Trends in der Künstlichen Intelligenz. <https://arxiv.org/abs/2502.18474>
- Wang, Wenguan, Yi Yang und Fei Wu. 2025. „Auf dem Weg zu Daten- und Wissensgetriebener AI: Eine Übersicht zum Neuro-Symbolic Computing“. IEEE Transaktionen auf Musteranalyse und Maschinenintelligenz 47(2): 878–899. <https://doi.org/10.1109/TPAMI.2024.3483273>
- Wang, Yu, You Zhang, Hao Zhang und Dan Xu. 2026. „YNU-NLP bei SemEval-2026 Task 11: Ein neurosymbolischer Ansatz mit Mechanismus, der Inhalte und formales Denken in Sprachmodellen entwirrt“. Proceedings des 20th International Workshop on Semantic Evaluation. Verein für Computerlinguistik. <https://doi.org/10.18653/v1/2026.semeval-1.126>
- Weng, Ke, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu und Tiancheng Zhang. 2025. „Autoformalisierung in der Ära der großen Sprachmodelle: Eine Übersicht“. arXiv:2505.23486. <https://arxiv.org/abs/2505.23486>
- Xia, Shihao, Mengting He, Shuai Shao, Tingting Yu, Yiyang Zhang, Nobuko Yoshida und Linhai Song. 2026. „SymGPT: Prüfung von Smart Contracts durch Kombination von symbolischer Ausführung mit großen Sprachmodellen“. Tagungsband des ACM zu Programmiersprachen 10 (OOPSLA1), Article 109. <https://doi.org/10.1145/3798217>
- Yang, Zonglin, et al. 2024. „Autoformalisierung von Problemen mit der Argumentation der natürlichen Sprache für die Logik erster Ordnung“. Tagungsband von EMNLP 2024. Verein für Computerlinguistik. <https://aclanthology.org/2024.emnlp-main.1132/>
- Ye, Mao, Chengyue Gong und Qiang Liu. 2020. „SAFER: Ein strukturfreier Ansatz für zertifizierte Robustheit gegenüber schädlichen Wortersetzungen“. Tagungsband von ACL 2020. Verein für Computerlinguistik. <https://aclanthology.org/2020.acl-main.317/>
- Ye, Xi, Qiaochu Chen, Isil Dillig und Greg Durrett. 2023. „SatLM: durch Erfüllbarkeit unterstützte Sprachmodelle mit deklarativem Prompting“. Fortschritte in neuronalen Informationsverarbeitungssystemen 36. <https://arxiv.org/abs/2305.09656>
- Zheng, Zihan, Tianle Cui, Chuwen Xie, Jiahui Pan, Qianglong Chen und Lewei He. 2025. „PlanningArena: Ein modularer Benchmark für die mehrdimensionale Bewertung von Planung und Werkzeuglernen“. Tagungsband von ACL 2025, 31047–31086. <https://doi.org/10.18653/v1/2025.acl-long.1499>